



acm International Collegiate Programming Contest



event
sponsor

图论

Ciallo~($\angle \cdot \omega <$) $\frown$ ☆

目 录

图论	4
树和图的存储	4
邻接矩阵	4
邻接表	4
树和图的遍历	5
DFS	5
DFS序列	7
连通块划分	7
BFS	9
拓扑排序	10
最短路	11
单源最短路	13
边权非负	13
Dijkstra朴素	13
Dijkstra堆优化	14
含负权边	15
Bellman-Ford	15
SPFA	16
多源最短路	18
Floyd	18
传递闭包	19
Johnson	19
(LCA)	22
分层图最短路	23
树上问题	25
树的重心	25
树的中心	28
树的直径	29
最近公共祖先	32
倍增求LCA	32
链剖求LCA	34
DFS序求LCA	35

Tarjan求LCA	36
树上差分	37
树链剖分	39
重链剖分	39
树哈希	47
生成树	49
最小生成树	49
Kruskal	49
Prim朴素	50
Boruvka	51
唯一性	51
最小瓶颈路	53
次小生成树	56
生成树计数	60
最小树形图	63
二分图	67
二分图判定	68
最大匹配	69
匈牙利算法	70
HK 算法	73
转为网络流模型	75
常见问题模型	77
最大权匹配	79
KM算法	79
转为费用流模型	85
最小点覆盖	87
最大独立集	89
最小路径覆盖	90
一般图最大匹配	93
一般图最大权匹配	95
基环树	100
负环	101
SPFA判负环	101
bellman-ford判负环	102

差分约束	103
连通性相关	105
割点和割边	105
双连通分量	108
边双连通分量	108
点双连通分量	110
强连通分量	111
2-SAT	113
网络流	115
最大流	116
Edmonds-Karp	116
Dinic	117
最小割	119
最大权闭合图	123
费用流	124
欧拉图	129

图论

[图论部分简介 - OI Wiki \(oi-wiki.org\)](#)

[作图工具Graph Editor \(csacademy.com\)](#)

树和图的存储

邻接矩阵

空间复杂度 n^2 ，适合存储稠密图，可以快速查询一条边是否存在

```
1 int arr[N][N];
2
3 void add(int a, int b){
4     arr[a][b] = x; //无权值则x为bool值代表是否连通，有权值则x为权值
5 }
6 //单向 add(a,b);
7 //双向 add(b,a);
```

邻接表

适合存稀疏图，不支持随机访问

初始 h[u] -> -1
add(u,1) h[u] -> 1 -> -1
add(u,2) h[u] -> 2 -> 1 -> -1
add(u,3) h[u] -> 3 -> 2 -> 1 -> -1

```
1 //数组模拟链表实现 ---推荐
2 //若为无向图，则边i和边i^1互为一条反边
3 int h[N], e[M], ne[M], idx;
4
5 //存边
6 void add(int a, int b) {
7     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
8 }
9
10 //遍历边
11 for(int i = h[x]; i != -1; i = ne[i]){
12     int k = e[i];
13     cout << k << ' ';
14 }
15
16 int main() {
17     memset(h, -1, sizeof h); //注意h需要初始化为-1
```

```
18 } }
```

```
1 //STL vector实现
2 //在内存和时间上都比数组模拟链表能差，可能被卡时间
3 vector<int>e[N];
4
5 void add(int a, int b) { //存边
6     e[a].push_back(b);
7 }
8
9 //遍历边
10 for(int i = 0; i < v[x].size(); i++){
11     cout << v[x][i] << ' ';
12 }
```

```
1 //STL list实现 由于内存不连续遍历不如vector但支持删除操作
2 list<int>e[N];
3
4 //存边
5 void add(int a, int b) {
6     e[a].push_back(b);
7 }
8
9 //遍历边
10 for(int &k:e[u]){
11     cout << k << " ";
12 }
```

树和图的遍历

二叉树的遍历	
前序遍历	根-左-右
中序遍历	左-根-右
后序遍历	左-右-根

DFS

```
1 #include <iostream>
2 #include <algorithm>
3 #include <cstring>
4 using namespace std;
```

```

5  const int N = 100010, M = N * 2;
6  int n;
7  int h[N], e[M], ne[M], idx;
8  bool st[N]; //存哪些点已经遍历过了
9
10 void add(int a, int b) {
11     e[idx] = b; ne[idx] = h[a]; h[a] = idx++;
12 }
13
14 void dfs(int u) {
15     st[u] = 1; //标记这个点遍历过了
16     for (int i = h[u]; i != -1; i = ne[i]) { //搜索下一条未被遍历过的条边
17         int k = e[i];
18         if (!st[k]) {
19             dfs(k);
20         }
21     }
22 }
23
24 void dfs2(int u, int fa) { //（无环图）可以省下一个vis数组
25     for (int i = h[u]; ~i; i = ne[i]) {
26         int k = e[i];
27         if (k != fa) {
28             dfs2(k, u);
29         }
30     }
31 }
32
33 int main() {
34     memset(h, -1, sizeof h);
35
36     dfs(1);
37     //dfs2(1, 0); 诺根节点编号从0开始，则应dfs2(0, -1)
38 }

```

```

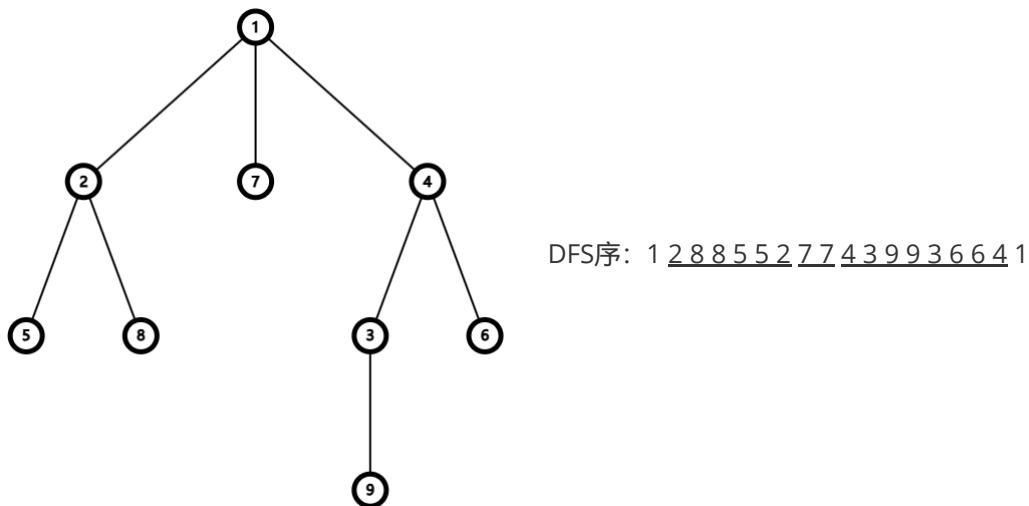
1  //dfs 无向图叶子节点
2  void dfs(int u, int fa) { //u为当前节点，fa为当前节点的父节点
3      if (q[u].size() == 1 && q[u][0] == fa) {
4          leaf.emplace_back(u); //此时u为叶子节点
5      }
6      st[u] = 1;
7      for (auto &k: q[u]) {
8          if (!st[k]) {
9              dfs(k, u);
10             st[k] = 1;
11         }
12     }
13 }

```

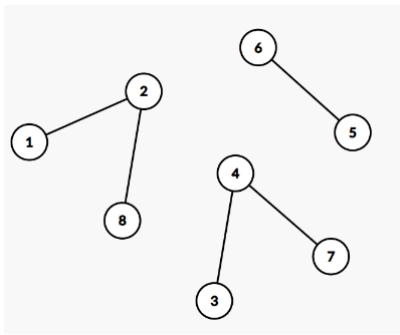

DFS序列

```
1 void dfs(int u){
2     d[++p] = u;
3     st[u] = 1; //每个点第一次被标记为走过时的p即为该点的时间戳
4     for(int i = h[u]; ~i; i = ne[i]){
5         int k = e[i];
6         if(!st[k]){
7             dfs(k);
8         }
9     }
10    d[++p] = u; //每个点来回会被标记两次,最后长度为2*n
11 }
12
13 for(int i = 1; i <= 2*n; i++) cout << d[i] << ' ';
```

每个节点 x 的编号在dfs序中恰好出现两次, 假设这两次的位置为 $L[x]$ 和 $R[x]$, 则闭区间 $L[x] \sim R[x]$ 就是以 x 为根的子树的dfs序。可以通过dfs序把子树统计转化为序列上的区间统计。例题: 线段树维护子树信息[Assign the task - HDU 3974 - Virtual Judge \(vjjudge.net\)](#).



连通块划分



对一个森林dfs/bfs可以划分出每一棵树，并查集可以达到类似效果

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  const int N = 100005;
5  int n,m;
6  int h[N],e[N],ne[N],idx;
7  int id[N],cnt;
8
9  void add(int a,int b){
10     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
11 }
12
13 void dfs(int u){
14     id[u] = cnt;
15     for(int i = h[u];~i;i = ne[i]){
16         int k = e[i];
17         if(!id[k]){
18             dfs(k);
19         }
20     }
21 }
22
23
24 int main(){
25     memset(h,-1,sizeof h);
26     cin >> n >> m;
27     while(m--){
28         int a,b;cin >> a >> b;
29         add(a,b);
30         add(b,a);
31     }
32
33     for(int i = 1;i <= n;i++){
34         if(!id[i]){
35             cnt++;
36             dfs(i);
37         }
38         cout << i << ' ' << id[i] << '\n';
39     }
40 }

```

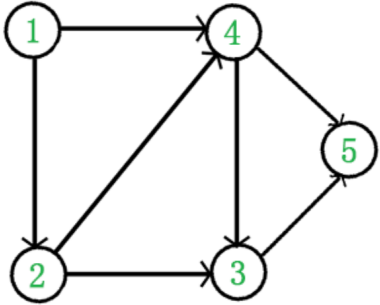
BFS

```
1 //1号点到n号点的最短距离 https://www.acwing.com/problem/content/849/
2 #include <iostream>
3 #include <cstring>
4 #include <queue>
5 using namespace std;
6 const int N = 100010;
7 int n, m;
8 queue<int>q; //队列实现
9 int h[N], e[N], ne[N], idx, d[N]; //d计算距离
10 void add(int a, int b) { //插入有向边a->b
11     e[idx] = b;
12     ne[idx] = h[a];
13     h[a] = idx++;
14 }
15
16 int bfs() {
17     q.push(1);
18     d[1] = 0;
19     while (q.size()) {
20         int t = q.front(); //取队头
21         q.pop();
22         for (int i = h[t]; i != -1; i = ne[i]) { //扩展t
23             int k = e[i];
24             if (d[k] == -1) { //如果没走过
25                 d[k] = d[t] + 1; //则标记为走过，距离+1
26                 q.push(k); //该点入队
27             }
28         }
29     }
30     return d[n];
31 }
32 int main() {
33     memset(h, -1, sizeof h); //初始化邻接表指向-1
34     memset(d, -1, sizeof d); //初始化距离为-1
35     cin >> n >> m;
36     while (m--) {
37         int a, b; cin >> a >> b;
38         add(a, b);
39     }
40     cout << bfs();
41 }
```

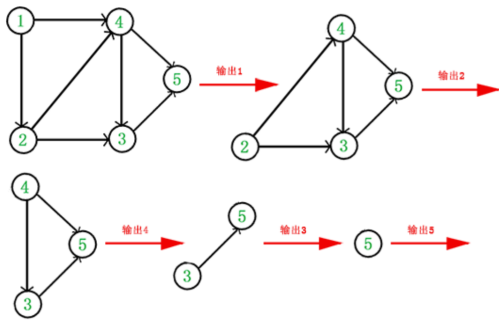
拓扑排序

在图论中，拓扑排序（Topological Sorting）是一个**有向无环图**（DAG, Directed Acyclic Graph）的所有顶点的线性序列。且该序列必须满足下面两个条件：

1. 每个顶点出现且只出现一次。
2. 若存在一条从顶点 A 到顶点 B 的路径，那么在序列中顶点 A 出现在顶点 B 的前面。



所有入度为0的点都可以作为起点，一个有向无环图至少存在一个入度为0的点，答案一般不唯一



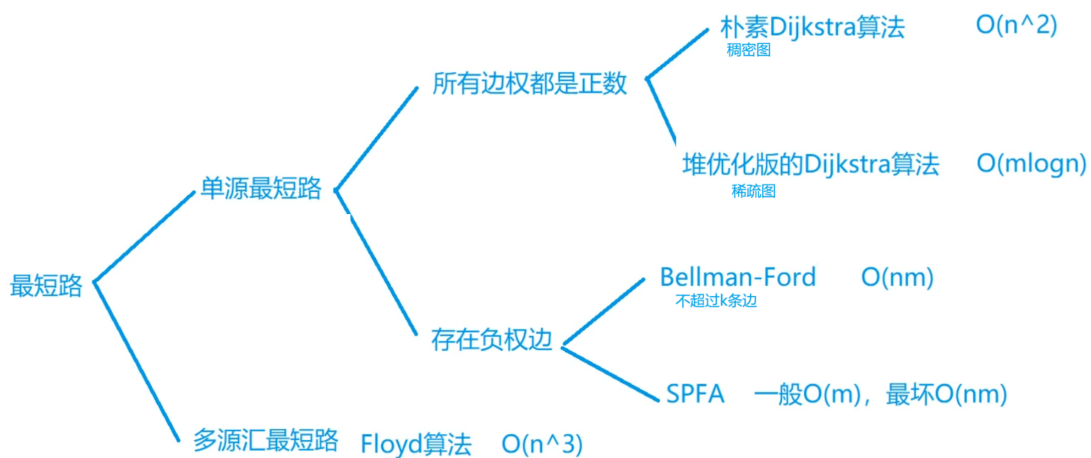
```
1  #include <iostream>
2  #include <cstring>
3  #include <queue>
4  using namespace std;
5  const int N = 100010;
6  int n, m;
7  queue<int>q;
8  int h[N], e[N], ne[N], idx;
9  int d[N], arr[N], cnt; //d[N] 存每条边的入度, arr[cnt] 记录每次入队的点
10 void add(int a, int b) {
11     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
12 }
13 bool topsort() {
14     for (int i = 1; i <= n; i++) {
15         if (d[i] == 0) { //先将所有入度为0的点入队
16             q.push(i);
17             arr[cnt++] = i;
18         }
19     }
20     while (q.size()) {
21         int t = q.front(); //取队头
```

```

22     q.pop();
23     for (int i = h[t]; i != -1; i = ne[i]) { //拓展t
24         int k = e[i];
25         d[k]--; //指向的边度数-1
26         if (d[k] == 0) { //如果入度为0则入队
27             q.push(k);
28             arr[cnt++] = k;
29         }
30     }
31 }
32 return cnt == n; //不相等说明存在重边或自环
33 }
34
35 int main() {
36     memset(h, -1, sizeof h);
37     cin >> n >> m;
38     while (m--) {
39         int a, b; cin >> a >> b;
40         add(a, b);
41         d[b]++; //b边入度+1
42     }
43     if (topsort()) {
44         for (int i = 0; i < n; i++) {
45             cout << arr[i] << ' ';
46         }
47     }
48     else cout << -1;
49
50     return 0;
51 }

```

最短路



记录路径

邻接表用 `pre[a] = b` 表示a是由b过来的，邻接矩阵用 `pre[i][j] = k`

```
1 //对于邻接表 https://codeforces.com/problemset/problem/20/c
2 if(dist[k] > dist[t] + w[i]){
3     pq.push({dist[y],y});
4     dist[k] = dist[t] + w[i];
5     pre[t] = k;//每次更新最短路时记录路径
6 }
7
8 //输出最短路时由终点往前推
9 for(int i = n;i != 0;i = pre[n]){
10     cout << i << " ";//此输出为反向,若要正向输出路径,将i记录下来逆序输出即可
11 }
```

```
1 //对于邻接矩阵
2 if(dist[i][j] > dis[i][k] + dist[k][j]){
3     dist[i][j] = dis[i][k] + dist[k][j];
4     path[i][j] = k;//更新时记录路径
5 }
6
7 void print(int a,int b){
8     if(path[a][b] == 0) return;
9     print(a,path[a][b]);//前半部
10    cout<< path[a][b] << " ";//输出该点
11    print(path[a][b],b);//后半部
12 }
13
14 cout << x << ' ';//输出从x到y的最短路,无需逆序输出。
15 print(x,y);
16 cout << y;
```

最短路条数

求最短路有多少条

```
1 //https://www.luogu.com.cn/problem/P1144
2 //初始时cnt[1] = 1;
3 if(dist[y] > dist[x] + w[i]){
4     dist[y] = dist[x] + w[i];
5     pq.push({dist[y],y});
6     cnt[y] = cnt[x];//更新y节点时,最短路条数继承自x
7 }
8 else if(dist[y] == dist[x] + w[i]){
9     cnt[y] = (cnt[y] + cnt[x]) % mod;//若有多条则累加
10 }
```

若要求最多同时划分成多少条最短路，他们之间不含公共边。则可以将所有最短路径上的边保留下来，(如何确定一条边是否为最短路：正反图各跑一次，若 $\text{起点} \rightarrow a + a \rightarrow b + b \rightarrow \text{终点} == \text{最短路}$ ，则边(a,b)为最短路上的边)，跑最大流即为答案。[4264. 规划最短路 - AcWing题库](#)

单源最短路

边权非负

Dijkstra朴素

适合稠密图，邻接矩阵存 $O(n^2)$

```
1 //https://www.acwing.com/activity/content/problem/content/918/
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5 const int N = 510;
6 int n, m;
7 int g[N][N]; //邻接矩阵存稠密图
8 int dist[N]; //记录每个点距离第一个点的距离
9 bool st[N]; //记录每个点的最短距离是否已经确认
10
11 int Dijkstra() {
12     memset(dist, 0x3f, sizeof dist); //初始化距离为无限大
13     dist[1] = 0; //第一个点到自身距离为0
14
15     for (int i = 0; i < n; i++) { //n个点进行n次迭代
16         int t = -1; //t存储当前访问的点
17
18         for (int j = 1; j <= n; j++) { //从1号点开始到n号点
19             if (!st[j] && (t == -1 || dist[t] > dist[j])) //t=寻找所有st=0中dist最
                小的点
20                 t = j;
21         }
22         //if(t == n) break;
23
24         st[t] = 1;
25
26         for (int j = 1; j <= n; j++) { //再用t依次更新每个点所到相邻的点路径值
27             dist[j] = min(dist[j], dist[t] + g[t][j]);
28             //min(1~j, 1~t + t~j)    d[j]取最短路径值
29         }
30     }
31
32     if (dist[n] == 0x3f3f3f3f) return -1; //如果第n号点距离为无穷,则不存在最短路
33     return dist[n];
34 }
35
```

```

36 int main() {
37     memset(g, 0x3f, sizeof g); //求最短,稠密图初始化为无限大
38     cin >> n >> m;
39     while (m--) {
40         int a, b, c; cin >> a >> b >> c;
41         g[a][b] = min(g[a][b], c); //处理重边,保留最短的一条
42     }
43     cout << Dijkstra();
44 }

```

Dijkstra堆优化

适合稀疏图, 优先队列小根堆实现 $O(m\log n)$

```

1 //https://www.acwing.com/problem/content/description/852/
2 #include <iostream>
3 #include <cstring>
4 #include <queue>
5 using namespace std;
6 using PII = pair<int, int>; //first为距离, 队列根据距离排序, second为对应点
7 const int N = 150005;
8 int n, m;
9 priority_queue<PII, vector<PII>, greater<PII>> pq;
10 int h[N], e[N], ne[N], idx;
11 int w[N]; //存权重
12 int dist[N];
13 bool st[N]; //如果为true说明这个点的最短路径已经确定
14
15 void add(int a, int b, int c) {
16     w[idx] = c;
17     e[idx] = b;
18     ne[idx] = h[a];
19     h[a] = idx++;
20 }
21
22 int Dijkstra() {
23     memset(dist, 0x3f, sizeof dist);
24     dist[1] = 0;
25     pq.push({0, 1});
26
27     while (pq.size()) {
28         auto t = pq.top(); //取队头t
29         pq.pop();
30         int ver = t.second, distance = t.first;
31
32         if (st[ver]) continue;
33         st[ver] = 1;
34         //如果距离已经确定,则跳过该点(st数组非必要,加上可以略微优化)

```



```

35 //或者st数组去掉,优化为 if (distance > dist[ver]) continue; //说明当前数据过
    时
36
37     for (int i = h[ver]; i != -1; i = ne[i]) { //拓展t
38         int k = e[i];
39         if (dist[k] > distance + w[i]) {
40             dist[k] = distance + w[i];
41             pq.push({dist[k], k});
42         }
43     }
44 }
45 if (dist[n] == 0x3f3f3f3f) return -1; //不存在输出-1
46 return dist[n];
47 }
48 int main() {
49     memset(h, -1, sizeof h);
50     cin >> n >> m;
51     while (m--){
52         int a, b, c; cin >> a >> b >> c;
53         add(a, b, c);
54     }
55
56     cout << Dijkstra();
57 }

```

含负权边

在图中如果存在负环，则从该环上任意一点出发，沿着环重复行走可以使路径总权值无限减小，因此通常意义上的“最短路”在这种情况下是没有定义的（路径长度可以无限小）。然而，根据具体问题的需求，我们仍然可以通过一些方法处理含有负环的最短路问题。

Bellman-Ford

可以解决**有边数限制**的问题（若问题允许忽略负环，如限制路径边数时，可以处理负权环）

时间复杂度 $O(mn)$ **结构体存边**

```

1 //https://www.acwing.com/problem/content/855/
2 //1~n号点，最多经过k条边的最短距离，边权可能为负数
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6 const int N = 505, M = 10004;
7 int n, m, k; //k代表最多经过k条边，若是没限制k的最短路，则取n-1
8 int dist[N], backup[N]; //backup备份防止串联

```

```

9 struct Edge { int a, b, w; }edges[M];//使用结构体存储
10
11 int bellman_ford() {
12     memset(dist, 0x3f, sizeof dist);
13     dist[1] = 0;
14
15     for (int i = 0; i < k; i++) {//k次循环
16         memcpy(backup, dist, sizeof backup);
17         for (int j = 0; j < m; j++) {//遍历所有边
18             auto &[a, b, w] = edges[j];
19             dist[b] = min(dist[b], backup[a] + w);//使用上一次备份数据防止节点最短距
20             离串联
21         }
22     }
23     return dist[n];
24 }
25
26 int main() {
27     cin >> n >> m >> k;
28     for (int i = 0; i < m; i++) {
29         int a, b, w; cin >> a >> b >> w;
30         edges[i] = { a,b,w };
31     }
32     int k = bellman_ford();
33     if (k > 0x3f3f3f3f/2) cout << "impossible";//最后一次松弛后可能会在0x3f3f3f3f附
34     近
35     else cout << k;
36 }

```

SPFA

也称为 队列优化的Bellman-Ford

一般 $O(m)$,最坏 $O(mn)$, **邻接表存, 队列实现**

如果不被卡(SPFA已经死了.jpg), 可以用来替代Dijkstra堆优化版

```

1 //https://www.acwing.com/problem/content/853/
2 #include <iostream>
3 #include <cstring>
4 #include <queue>
5 using namespace std;
6 const int N = 100005;
7 int n, m;
8
9 int h[N], e[N], ne[N], idx;
10 int dist[N],w[N];
11 bool st[N];
12

```

```

13 void add(int a, int b, int c) {
14     w[idx] = c, e[idx] = b, ne[idx] = h[a], h[a] = idx++;
15 }
16
17 int SPFA() {
18     memset(dist, 0x3f, sizeof dist); //距离初始化为正无穷
19     queue<int> q; //队列存最小距离变小的点，再用它扩展到邻接表中其它相邻的点
20     dist[1] = 0; //起点到自身距离为0
21     q.push(1);
22
23     while (q.size()){
24         auto t = q.front();
25         q.pop();
26         st[t] = 0; //从队列中取出来之后该节点st被标记为false
27
28         for (int i = h[t]; i != -1; i = ne[i]) {
29             int k = e[i];
30             if (dist[k] > dist[t] + w[i]) {
31                 dist[k] = dist[t] + w[i];
32                 if (!st[k]) { //将当前未加入队列的结点入队
33                     q.push(k);
34                     st[k] = 1;
35                 }
36             }
37         }
38     }
39     return dist[n];
40 }
41
42 int main() {
43     memset(h, -1, sizeof h); //链表头节点初始化为-1
44     cin >> n >> m;
45     while (m--){
46         int a, b, c; cin >> a >> b >> c;
47         add(a, b, c);
48     }
49     int t = SPFA();
50     if (t == 0x3f3f3f3f) puts("impossible");
51     else cout << t;
52 }

```

多源最短路

Floyd

可以有负权边，不能有负权环 $O(n^3)$

邻接矩阵存储

```
1 | (k:1~n) (i:1~n) (j:1~n) {d[i,j] = min(d[i,j], d[i,k] + d[k,j]);}
```

```
1 //https://www.acwing.com/problem/content/description/856/
2 #include <iostream>
3 using namespace std;
4 const int inf = 0x3f3f3f3f;
5 const int N = 210;
6 int d[N][N];
7 int n, m, q;
8
9 void floyd() {
10     for (int k = 1; k <= n; k++) { //在枚举第k层前,已经得到了前k-1个点之间(路径不包含k)
        的最短路
11         for (int i = 1; i <= n; i++) {
12             for (int j = 1; j <= n; j++) {
13                 d[i][j] = min(d[i][j], d[i][k] + d[k][j]); //核心代码
14             }
15         }
16     }
17 }
18
19 int main() {
20     cin >> n >> m >> q;
21     for (int i = 1; i <= n; i++) {
22         for (int j = 1; j <= n; j++) {
23             if (i != j) d[i][j] = inf; //初始化为无穷,处理自环
24         }
25     }
26
27     while (m--) {
28         int a, b, c; cin >> a >> b >> c; //a到b的距离为c
29         d[a][b] = min(d[a][b], c); //处理重边
30     }
31
32     floyd();
33
34     while (q--) {
35         int a, b; cin >> a >> b;
36         if (d[a][b] > inf / 2) cout << "impossible" << endl;
37         //因为存在负权边,可能会把inf更新变小,导致d[a][b]略小于inf
38         else cout << d[a][b] << endl;
39     }
40 }
```

传递闭包

在一张点数为 n 的有向图的邻接矩阵中，给出任意两点间是否有直接连边的关系，让你求出任意两点之间是否有直接连边或间接连边的关系。

如果 i 能到达 k ，且 k 能到达 j ，则 i 也能到达 j

```
1 //https://www.luogu.com.cn/problem/B3611
2 //给定一张点数为  $n$  的有向图的邻接矩阵，图中不包含自环，求该有向图的传递闭包
3 #include <iostream>
4 using namespace std;
5 const int N = 105;
6 int n;
7 int a[N][N];
8
9 void floyd(){
10     for(int k = 1;k <= n;k++){
11         for(int i = 1;i <= n;i++){
12             for(int j = 1;j <= n;j++){
13                 a[i][j] |= a[i][k] & a[k][j];
14             }
15         }
16     }
17 }
18
19 int main(){
20     cin >> n;
21     for(int i = 1;i <= n;i++){
22         for(int j = 1;j <= n;j++){
23             cin >> a[i][j];
24         }
25     }
26     floyd();
27     for(int i = 1;i <= n;i++){
28         for(int j = 1;j <= n;j++){
29             cout << a[i][j] << ' ';
30         }
31         cout << '\n';
32     }
33 }
```

Johnson

可以有负权边，不能有负权环

时间复杂度 $O(NM \log M)$

- 新建一个虚拟的0号节点，从这个点向其它点连一条边权为0的边。然后用Bellman-Ford求出0号节点到其它所有点的最短路，记为 $h[]$ 。

- 假设存在一条从x到y边权为w的边，则我们重新设置边权为 $w+p[x]-p[y]$ (类似于物理学中的势能概念)，消除负权边的影响。
- 接下来以每个点为起点跑n轮Dijkstra算法，再消除势能差，即可求出任意两点间的最短路。

```

1 //https://www.luogu.com.cn/problem/P5905
2 #include <iostream>
3 #include <queue>
4 #include <cstring>
5 using namespace std;
6 const long long INF = 1e9;
7 const int N = 3003,M = 10004;
8 int n,m;
9 int h[N],e[M],ne[M],w[M],idx;
10 long long dist[N][N],p[N];
11 bool vis[N];
12 int cnt[N];
13
14 void add(int a,int b,int c){
15     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
16 }
17
18 bool spfa(){
19     memset(p,0x3f,sizeof p);
20     queue<int>q;
21     q.push(0);
22     p[0] = 0;
23     vis[0] = 1;
24     while(q.size()){
25         int t = q.front();
26         q.pop();
27         vis[t] = 0;
28         for(int i = h[t];~i;i = ne[i]){
29             int k = e[i];
30             if(p[k] > p[t] + w[i]){
31                 p[k] = p[t] + w[i];
32                 if(!vis[k]){
33                     q.push(k);
34                     vis[k] = 1;
35                     cnt[k]++;
36                     if(cnt[k] == n+1) return 1;
37                 }
38             }
39         }
40     }
41     return 0;
42 }
43
44 void dijkstra(int bg){
45     memset(vis,0,sizeof vis);
46
47     priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>>pq;

```

```

47     pq.push({0,bg});
48     dist[bg][bg] = 0;
49     while(pq.size()){
50         auto [distance,ver] = pq.top();
51         pq.pop();
52         if(vis[ver]) continue;
53         else vis[ver] = 1;
54         for(int i = h[ver];~i;i = ne[i]){
55             int k = e[i];
56             if(dist[bg][k] > dist[bg][ver] + w[i]){
57                 dist[bg][k] = dist[bg][ver] + w[i];
58                 if(!vis[k]){
59                     pq.push({dist[bg][k],k});
60                 }
61             }
62         }
63     }
64 }
65
66 int main(){
67     memset(h,-1,sizeof h);
68     cin >> n >> m;
69     for(int i = 1;i <= m;i++){
70         int a,b,c;cin >> a >> b >> c;
71         add(a,b,c);
72     }
73     for(int i = 1;i <= n;i++) add(0,i,0);
74
75     if(spfa()) {cout << -1;return 0;}
76
77     for(int u = 1;u <= n;u++){
78         for(int i = h[u];~i;i = ne[i]){
79             w[i] += p[u] - p[e[i]];
80         }
81     }
82
83     memset(dist,0x3f,sizeof dist);
84     for(int i = 1;i <= n;i++) { dijkstra(i); }
85
86     for(int i = 1;i <= n;i++){
87         long long ans = 0;
88         for(int j = 1;j <= n;j++){
89             dist[i][j] += p[j] - p[i]; //最后消除势能差，此时dist[i][j]即为i到j的最短
路
90             if(dist[i][j] < INF ) ans += j * dist[i][j];
91             else ans += j * INF;
92         }
93         cout << ans << '\n';
94     }
95 }

```

(LCA)

求**无向树**上任意两点的距离

$O(N\log N)$ 预处理, $O(\log N)$ 查询

设两个节点分别为 x,y , $\text{dist}[i]$ 为根节点到点 i 的距离, $\text{lca}(x,y)$ 表示两点的最近公共祖先。

则 $\text{ans} = \text{dist}[x] + \text{dist}[y] - 2 * \text{dist}[\text{lca}(x,y)]$

```
1 //例题:How far away? https://vjudge.net/problem/HDU-2586#author=GPT\_zh
2 #include <iostream>
3 #include <queue>
4 #include <vector>
5 using namespace std;
6
7 void sol(){
8     int n,q;scanf("%d %d",&n,&q);
9     vector<vector<int>>>fa(n+1,vector<int>(20));
10    vector<vector<pair<int,int>>>e(n+1);
11    vector<int>dist(n+1,0x3f3f3f3f),st(n+1),deep(n+1);
12    for(int i = 1;i < n;i++){
13        int a,b,c;scanf("%d %d %d",&a,&b,&c);
14        e[a].push_back({b,c});
15        e[b].push_back({a,c});
16    }
17    auto uuz = [&]() {
18
19        priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>>pq;
20        dist[1] = 0;
21        pq.push({0,1});
22        while(pq.size()){
23            auto [distance,ver] = pq.top();pq.pop();
24            if(st[ver]) continue;
25            else st[ver] = 1;
26            for(auto &[k,w]:e[ver]){
27                if(dist[k] > distance + w){
28                    dist[k] = distance + w;
29                    pq.push({dist[k],k});
30                }
31            }
32        }
33        auto dfs = [&](auto &dfs,int u,int father)->void{
34            deep[u] = deep[father] + 1;
35            fa[u][0] = father;
36            for(int i = 1;(1 << i) <= deep[u];i++){
37                fa[u][i] = fa[fa[u][i-1]][i-1];
38            }
39            for(auto [k,w]:e[u]){
40                if(k == father) continue;
```



```

41         dfs(dfs,k,u);
42     }
43 };
44 auto lca = [&](int x,int y){
45     if(deep[x] < deep[y]) swap(x,y);
46     for(int i = 20;i >= 0;i--){
47         if(deep[fa[x][i]] >= deep[y]) x = fa[x][i];
48     }
49     if(x == y) return x;
50     for(int i = 20;i >= 0;i--){
51         if(fa[x][i] != fa[y][i]){
52             x = fa[x][i];
53             y = fa[y][i];
54         }
55     }
56     return fa[x][0];
57 };
58 uuz();
59 dfs(dfs,1,0);
60 while(q--){
61     int a,b;scanf("%d %d",&a,&b);
62     int k = lca(a,b);
63     printf("%d\n",dist[a] + dist[b] - 2*dist[k]);
64 }
65 }
66
67 int main(){
68     int t;scanf("%d",&t);
69     while(t--) sol();
70 }

```

分层图最短路

有 k 次机会改变某一条路径的权值，求最短路。

对于这种类型的题目，我们可以构建 $k + 1$ 层平行图，第 $0 \leq j \leq k$ 层分别代表使用了 j 次机会。每层按照原图建图，第 $j - 1$ 层到第 j 层之间，每对边连一条权值为 val 的有向边（ val 按题目要求而定）。以第 0 层的起点开始跑最短路，答案取每层终点的最小值。

[P4568 [JLOI2011](#)] 飞行路线 - 洛谷

N 个点 M 条边的无向带权图，一共有 K 次机会可以 0 代价经过一条边，求给定起点和终点，求最短路。

```
1 #include <bits/stdc++.h>
```

```

2  const int N = 10004 * 11, M = 50004 * 11 * 4; //节点数至少  $N*(K+1)$ , 边数  $M*K*4$ 
3  int n, m, k;
4  int s1, s2;
5
6  int h[N], e[M], ne[M], w[M], idx;
7  void add(int a, int b, int c) {
8      w[idx] = c, e[idx] = b, ne[idx] = h[a], h[a] = idx++;
9  }
10
11 int id(int u, int d) {
12     return u + d * n;
13 }
14
15 int dist[N];
16 bool vis[N];
17
18 int uuz() {
19     std::memset(dist, 0x3f, sizeof dist);
20     std::priority_queue<std::pair<int, int>, std::vector<std::pair<int, int>>,
std::greater<std::pair<int, int>>> pq;
21     pq.push({0, s1});
22     dist[id(s1, 0)] = 0;
23     while (pq.size()) {
24         int x = pq.top().second;
25         pq.pop();
26         if (vis[x]) continue;
27         else vis[x] = 1;
28         for (int i = h[x]; ~i; i = ne[i]) {
29             int y = e[i];
30             if (dist[y] > dist[x] + w[i]) {
31                 dist[y] = dist[x] + w[i];
32                 pq.push({dist[y], y});
33             }
34         }
35     }
36     int ans = 0x3f3f3f3f;
37     for (int i = 0; i <= k; i++) {
38         ans = std::min(ans, dist[id(s2, i)]); //每层终点中取最小
39     }
40     return ans;
41 }
42
43 int main() {
44     std::memset(h, -1, sizeof h);
45     std::cin >> n >> m >> k;
46     std::cin >> s1 >> s2;
47
48     for (int i = 1; i <= m; i++) {
49         int a, b, c; std::cin >> a >> b >> c;
50         for (int j = 0; j <= k; j++) {
51             add(id(a, j), id(b, j), c);
52             add(id(b, j), id(a, j), c);

```

```

53         if (j >= 1) { // 第j-1层到第j层有向边,权值设为0
54             add(id(a, j - 1), id(b, j), 0);
55             add(id(b, j - 1), id(a, j), 0);
56         }
57     }
58 }
59 std::cout << uuz();
60 }

```

还有一种方法是多开一维记录决策信息。`dist[i][j]`表示到达*i*用了*j*次机会的最小花费, `vis[i][j]`标记该状态是否出现过。类似于dp, 在最短路模版更新时

- 不使用机会 `dist[y][j] = min(dist[y][j], dist[x][j] + w[i])`
- 使用机会 `dist[y][j] = min(dist[y][j], dist[y][j - 1] + val)`

树上问题

树的重心

重心是指树中的一个结点, 如果将这个点删除后, 剩余各个连通块中点数的最大值最小, 那么这个结点被称为树的重心。

1. 树的重心如果不唯一, 则**至多有两个且相邻**
2. 以树的重心为根时, 所有子树的大小都不超过整棵树大小的一半。
3. 树中所有点到某个点的距离和中, **到重心的距离和最小**; 如果有两个重心, 那么到它们的距离和一样。
4. 把两棵树通过一条边相连得到一棵新的树, 那么新的树的重心在连接原来两棵树的重心的路径上。
5. 在一棵树上添加或删除一个叶子, 那么它的重心最多只移动一条边的距离。

```

1  #include <iostream>
2  #include <vector>
3  #include <cstring>
4  using namespace std;
5  const int N = 200005;
6  int n,m;
7  int h[N],e[N],ne[N],idx;
8  bool st[N];
9  int siz[N],weight[N]; //siz[i]表示dfs(x)选x为树根时,所有根节点为i的子树的大小(包括i)
10                          //weight[i]表示第i个节点的最大的子树的大小,与dfs(x)的x选取无关
11  vector<int>cent; //重心可能有两个
12
13  void add(int a,int b){
14      e[idx] = b,ne[idx] = h[a],h[a] = idx++;
15  }
16

```

```

17 void dfs(int u){
18     st[u] = 1;
19     siz[u] = 1;
20     int &now = weight[u];
21     for(int i = h[u]; ~i; i = ne[i]){
22         int k = e[i];
23         if(!st[k]){
24             dfs(k);
25             siz[u] += siz[k];
26             now = max(now, siz[k]);
27         }
28     }
29     now = max(now, n - siz[u]);
30     if(now <= n/2){//根据定义2:以树的重心为根时,所有子树的大小都不超过整棵树大小的一半
31         cent.emplace_back(u);
32     }
33 }
34
35 int main(){
36     memset(h, -1, sizeof h);
37     cin >> n >> m;
38     for(int i = 1; i <= m; i++){
39         int a, b; cin >> a >> b;
40         add(a, b);
41         add(b, a);
42     }
43
44     dfs(1); //任选一个点当做树根dfs均可
45
46     cout << cent[0] << endl;
47     //cout << cent[1] << endl; //重心可能有两个
48
49     //cout << weight[cent[0]] << endl;
50 }

```

```

1 //给定一棵有根树, 求出每一棵子树 (有根树意义下且包含整棵树本身) 的重心是哪一个节点。O(N)
2 //https://codeforces.com/contest/685/problem/B
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6 const int N = 300005;
7 int n, q;
8 int h[N], e[N], ne[N], idx;
9 bool st[N];
10
11 int fa[N]; //存父节点
12 int ans[N]; //当前节点的重心
13 int siz[N]; //子树大小: 所有子树上节点数 + 该节点
14 int weight[N]; //节点重量: 即所有子树「大小」的最大值
15
16 void add(int a, int b){

```

```

17     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
18 }
19
20 void dfs(int u){
21     siz[u] = 1;
22     ans[u] = u;
23     for(int i = h[u]; ~i; i = ne[i]){
24         int k = e[i];
25         dfs(k);
26         siz[u] += siz[k];
27         weight[u] = max(weight[u], siz[k]);
28     }
29     for(int i = h[u]; ~i; i = ne[i]){
30         int k = e[i];
31         int p = ans[k];
32         while(p != u){
33             if(max(weight[p], siz[u] - siz[p]) <= siz[u] / 2){
34                 ans[u] = p;
35                 break;
36             }
37             else{
38                 p = fa[p];
39             }
40         }
41     }
42 }
43
44 int main(){
45     memset(h, -1, sizeof h);
46     cin >> n >> q;
47     for(int i = 2; i <= n; i++){
48         int a; cin >> a;
49         fa[i] = a;
50         add(a, i);
51     }
52     dfs(1);
53     while(q--){
54         int u; cin >> u;
55         cout << ans[u] << endl;
56     }
57 }

```

树的中心

在树中，如果节点 x 作为根节点时，从 x 出发的最长链最短，那么称 x 为这棵树的中心。

性质：

- 树的中心不一定唯一，但最多有 2 个，且这两个中心是相邻的。
- 树的中心一定位于树的直径上。
- 树上所有点到其最远点的路径一定交会于树的中心。
- 当树的中心为根节点时，其到达直径端点的两条链分别为最长链和次长链。
- 当通过在两棵树间连一条边以合并为一棵树时，连接两棵树的中心可以使新树的直径最小。
- 树的中心到其他任意节点的距离不超过树直径的一半。

```
1 //树形dp实现 https://www.acwing.com/problem/content/1075/
2 //给定一颗树，找到一个点，使得该点到树中其它节点的最远距离最近
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6 const int N = 200005, INF = 0x3f3f3f3f;
7 int n;
8 int h[N], e[N], ne[N], w[N], idx;
9 int d1[N], d2[N], p1[N], p2[N], up[N];
10 //d1,d2表示向下的最大值和次大值，p1,p2表示d1,d2是由哪个子节点更新过来的
11 //up表示向上的最大值
12
13 void add(int a, int b, int c) {
14     w[idx] = c, e[idx] = b, ne[idx] = h[a], h[a] = idx++;
15 }
16
17 void dfs_d(int u, int fa) { //子节点向父节点更新，获得向下的d1,d2
18     d1[u] = d2[u] = -INF; //初始化为-INF可以处理负权边
19     for (int i = h[u]; ~i; i = ne[i]) {
20         int k = e[i];
21         if (k == fa) continue;
22         dfs_d(k, u);
23         int t = d1[k] + w[i];
24         if (t > d1[u]) {
25             d2[u] = d1[u]; d1[u] = t;
26             p2[u] = p1[u]; p1[u] = k;
27         }
28         else if (t > d2[u]) {
29             d2[u] = t;
30             p2[u] = k;
31         }
32     }
33     if (d1[u] == -INF) { d1[u] = d2[u] = 0; } //特殊处理叶子节点
34 }
35
36 void dfs_u(int u, int fa) { //父节点向子节点更新，获得向上的up
37     for (int i = h[u]; ~i; i = ne[i]) {
```

```

38     int k = e[i];
39     if(k == fa)continue;
40     if(p1[u] == k){ up[k] = max(up[u],d2[u])+w[i]; }
41     else { up[k] = max(up[u],d1[u])+w[i]; }
42     dfs_u(k,u);
43 }
44 }
45
46 int main(){
47     memset(h,-1,sizeof h);
48     cin >> n;
49     for(int i = 1;i < n;i++){
50         int a,b,c;cin >> a >> b >> c ;
51         add(a,b,c);add(b,a,c);
52     }
53
54     dfs_d(1,0);
55     dfs_u(1,0);
56
57     int ans = INF;
58     for(int i = 1;i <= n;i++){
59         ans = min(ans,max(d1[i],up[i]));
60     }
61     cout << ans;
62 }

```

树的直径

树上任意两节点之间最长的简单路径即为「树的直径」。(可能有多条，所有直径经过相同的中点)

树形DP实现：我们记录当 1 为树的根时，每个节点作为子树的根向下，所能延伸的最长路径长度 d_1 与次长路径（与最长路径无公共边）长度 d_2 ，那么直径就是对于每一个点，该点 $d_1 + d_2$ 能取到的值中的最大值。

```

1 //https://www.acwing.com/problem/content/1074/
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5 const int N = 200005;
6 int n;
7
8 int h[N],e[N],ne[N],w[N],idx;
9 int d1[N],d2[N],ans;//d1记录最长路径，d2记录次长路径，初始为0，避免负权边的影响
10
11 void add(int a,int b,int c){
12     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
13 }
14

```

```

15 void dfs(int u,int fa){
16     for(int i = h[u];~i;i = ne[i]){
17         int k = e[i];
18         if(k == fa) continue;
19         dfs(k,u);
20         int t = d1[k] + w[i];
21         if(t > d1[u]){//若t>d1同时更新d1,d2
22             d2[u] = d1[u];
23             d1[u] = t;
24         }
25         else if(t > d2[u]){//else if若t>d2只需更新d2
26             d2[u] = t;
27         }
28     }
29     ans = max(ans,d1[u]+d2[u]);
30 }
31
32 int main(){
33     memset(h,-1,sizeof h);
34     cin >> n;
35     for(int i = 1;i < n;i++){
36         int a,b,c;cin >> a >> b >> c;
37         add(a,b,c);
38         add(b,a,c);
39     }
40     dfs(1,0);
41     cout << ans;
42 }

```

两次DFS/BFS实现(边权必须非负): 首先从任意节点 y 开始进行第一次 DFS, 到达距离其最远的节点, 记为 z , 然后再从 z 开始做第二次 DFS, 到达距离 z 最远的节点, 记为 z' , 则 $dist(z, z')$ 即为树的直径。

定理: 在一棵树上, 从任意节点 y 开始进行一次 DFS, 到达的距离其最远的节点 z 必为直径的一端。

```

1 //https://www.luogu.com.cn/problem/B4016
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5 using ll = long long;
6 const int N = 400005;
7
8 int n;
9 int h[N],e[N],ne[N],w[N],idx;
10 ll dist[N];
11 ll maxn;//记录最远点能到的点
12
13 void add(int a,int b,int c){
14     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
15 }
16

```



```

17 void dfs(int u,int fa){
18     for(int i = h[u];i != -1;i = ne[i]){
19         int k = e[i];
20         if(k == fa) continue;
21         dist[k] = dist[u] + w[i];
22         if(dist[k] > dist[maxn]){maxn = k;}//更新最远能到的点
23         dfs(k,u);
24     }
25 }
26
27 int main(){
28     memset(h,-1,sizeof h);
29     cin >> n;
30     for(int i = 1;i < n;i++){
31         int a,b,c = 1;cin >> a >> b;
32         add(a,b,c);
33         add(b,a,c);
34     }
35     dfs(1,0);
36     dist[maxn] = 0;
37     dfs(maxn,0);
38     cout << dist[maxn] << endl;;
39 }

```

```

1 //bfs(1)
2 void bfs(int u){
3     queue<pair<int,int>>q;
4     memset(st,0,sizeof st);
5     q.push({u,0});
6     st[u] = 1;
7     while(q.size()){
8         auto [v,dist] = q.front();
9         q.pop();
10        for(int i = h[v];~i;i = ne[i]){
11            int k = e[i];
12            if(st[k]) continue;
13            st[k] = 1;
14            q.push({k,dist+w[i]});
15            if(ans < dist + w[i]){
16                ans = dist + w[i];
17                maxn = k;
18            }
19        }
20    }
21 }

```

最近公共祖先

性质

1. $LCA(\{u\}) = u$;
2. u 是 v 的祖先, 当且仅当 $LCA(u, v) = u$;
3. 如果 u 不为 v 的祖先并且 v 不为 u 的祖先, 那么 u, v 分别处于 $LCA(u, v)$ 的两棵不同子树中;
4. 前序遍历中, $LCA(S)$ 出现在所有 S 中元素之前, 后序遍历中 $LCA(S)$ 则出现在所有 S 中元素之后;
5. 两点集并的最近公共祖先为两点集分别的最近公共祖先的最近公共祖先, 即 $LCA(A \cup B) = LCA(LCA(A), LCA(B))$;
6. 两点的最近公共祖先必定处在树上两点间的最短路上;
7. $d(u, v) = h(u) + h(v) - 2h(LCA(u, v))$, 其中 d 是树上两点间的距离, h 代表某点到树根的距离。

编号 l 到 编号 r 的所有节点的 LCA, 就是这些节点中, dfn 序最小的节点与 dfn 序最大的节点的 LCA, 对 dfn 序建立 ST 表快速求区间最值。 pog loves szh III - HDU 5266 - Virtual Judge

算法	类型	时间复杂度	空间复杂度	特点
倍增	在线	$O(n \log n)$ 预处理 $O(\log n)$ 查询	$O(n \log n)$	灵活, 易理解
树链剖分	在线	$O(n)$ 预处理 $O(\log n)$ 查询	$O(n)$	功能全面
DFS序	在线	$O(n \log n)$ 预处理 $O(1)$ 查询	$O(n \log n)$	查询快
Tarjan	离线	$O(n+q)$	$O(n)$	常数小

倍增求LCA

$O(N \log N)$ 预处理, $O(\log N)$ 查询, 空间复杂度 $O(N \log N)$

dfs 预处理 $fa[i, j]$, 表示节点 i 的第 2^j 个祖先

求 LCA 时, 让深度最小的节点跳到与另一个节点同一个深度, 若此时两个节点相同直接返回当前节点。再让两个节点同时往上跳到最后一个非公共祖先节点, 该点的父亲节点即为 LCA 节点

```
1 //https://www.luogu.com.cn/problem/P3379
2 #include <iostream>
3 #include <vector>
4 using namespace std;
```

```

5  const int N = 500005;
6  int n,q,root;
7  vector<int> e[N];
8  int fa[N][22],deep[N]; //深度从1开始
9
10 void dfs(int u,int father){
11     fa[u][0] = father;
12     deep[u] = deep[father] + 1;
13     for(int i = 1;i <= 20;i++){
14         fa[u][i] = fa[fa[u][i-1]][i-1];
15     }
16     for(auto k:e[u]){
17         if(k == father) continue;
18         dfs(k,u);
19     }
20 }
21
22 int lca(int x,int y){
23     if(deep[x] < deep[y]) swap(x,y); //一般以x作为深度较深的点
24     for(int i = 20;i >= 0;i--){ //再让x跳到与y同一深度
25         if(deep[fa[x][i]] >= deep[y]) x = fa[x][i];
26     }
27     if(x == y) return x;
28     for(int i = 20;i >= 0;i--){
29         if(fa[x][i] != fa[y][i]){
30             x = fa[x][i];
31             y = fa[y][i];
32         }
33     }
34     return fa[x][0];
35 }
36
37 int main(){
38     cin >> n >> q >> root;
39     for(int i = 1;i < n;i++){
40         int a,b;cin >> a >> b;
41         e[a].emplace_back(b);
42         e[b].emplace_back(a);
43     }
44     dfs(root,0);
45     while(q--){
46         int a,b;cin >> a >> b;
47         cout << lca(a,b) << '\n';
48     }
49 }

```

链剖求LCA

$O(N)$ 预处理, $O(\log N)$ 查询, 空间复杂度 $O(N)$, 常数较小

不断让链顶深度较深的节点往上跳, 直到二者在同一条重链。此时深度较小的节点即为LCA。

```
1  #include <vector>
2  #include <iostream>
3  #include <cstring>
4
5  const int N = 500005;
6  int n,q,root = 1;
7  std::vector<int>e[N];
8
9  int fa[N],dep[N],son[N],siz[N];
10 void dfs1(int x,int father){
11     fa[x] = father;
12     dep[x] = dep[father] + 1;
13     siz[x] = 1;
14     for (auto &y: e[x]) {
15         if(y == father) continue;
16         dfs1(y,x);
17         siz[x] += siz[y];
18         if(siz[y] > siz[son[x]]) son[x] = y;
19     }
20 }
21
22 int top[N],id[N],idn,rnk[N];
23 void dfs2(int x,int father){
24     top[x] = father;
25     id[x] = ++idn;
26     rnk[idn] = x;
27     if (son[x] == 0) return;
28     dfs2(son[x],father);
29     for(auto &y : e[x]) {
30         if(y == son[x] || y == fa[x]) continue;
31         dfs2(y,y);
32     }
33 }
34
35 int lca(int x,int y){
36     while(top[x] != top[y]) {
37         if(dep[top[x]] > dep[top[y]]) x = fa[top[x]]; //选择链顶深度较深的一条链往上跳
38         else y = fa[top[y]];
39     }
40     return dep[x] < dep[y] ? x : y; //处于同一条链时,深度较浅的节点即为最近公共祖先
41 }
42
43 int main(){
44     std::cin >> n >> q >> root;
45     for (int i = 1; i < n; i++) {
46         int a,b; std::cin >> a >> b;
```

```

47     e[a].emplace_back(b);
48     e[b].emplace_back(a);
49 }
50
51     dfs1(root,0);
52     dfs2(root,root);
53
54     while(q--) {
55         int a,b; std::cin >> a >> b;
56         std::cout << lca(a,b) << '\n';
57     }
58 }

```

DFS序求LCA

$O(N \log N)$ 预处理, $O(1)$ 查询, 空间复杂度 $O(N \log N)$

若 $x = y$, 则他们的 LCA 等于 x 。

否则, 他们的 LCA 等于 DFS序 上位置在 $[dfn_x + 1, dfn_y]$ 的深度最小的任意结点的父亲。

```

1  #include <iostream>
2  #include <cmath>
3  #include <vector>
4
5  const int N = 500005;
6  int n,q,root;
7  std::vector<int>e[N];
8
9  int mi[N][20];
10 int dfn[N],idn;
11
12 int get(int x,int y) {
13     return dfn[x] < dfn[y] ? x : y;
14 }
15
16 void dfs(int x,int father){
17     dfn[x] = ++idn;
18     mi[idn][0] = father;
19     for (auto &y: e[x]) {
20         if (y != father) dfs(y, x);
21     }
22 }
23
24 int lca(int x,int y) {
25     if(x == y) return x;
26     x = dfn[x],y = dfn[y];
27     if(x > y) std::swap(x,y);
28     int d = std::__lg(y-x);
29     x++;
30     return get(mi[x][d],mi[y-(1<<d)+1][d]);

```

```

31 }
32
33 int main(){
34     std::cin >> n >> q >> root;
35     for (int i = 1; i < n; i++) {
36         int a, b; std::cin >> a >> b;
37         e[a].emplace_back(b);
38         e[b].emplace_back(a);
39     }
40
41     dfs(root,0);
42
43     for (int j = 1; j <= std::__lg(n); j++) {
44         for (int i = 1; i + (1 << j) - 1 <= n; i++) {
45             mi[i][j] = get(mi[i][j - 1], mi[i + (1 << j - 1)][j - 1]);
46         }
47     }
48
49     while (q--) {
50         int a,b; std::cin >> a >> b;
51         std::cout << lca(a,b) << '\n';
52     }
53 }

```

Tarjan求LCA

离线算法，使用 DFS+并查集 实现

时间复杂度 $O(N + Q)$

```

1  #include <iostream>
2  #include <vector>
3  #include <cstring>
4  using namespace std;
5  const int N = 1000006;
6  int n,q,root;
7  int h[N],e[N],ne[N],idx;
8  int dfn[N],low[N],dn;
9  vector<pair<int,int>>ask[N];
10 int ans[N];
11 int p[N];
12
13 void add(int a,int b){
14     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
15 }
16
17 int find(int x){
18     if(p[x] != x) p[x] = find(p[x]);
19     return p[x];
20 }

```

```

21
22 void tarjan(int x,int fa){
23     for(int i = h[x];~i;i = ne[i]){
24         int y = e[i];
25         if(y == fa) continue;
26         tarjan(y,x);
27         p[find(y)] = find(x);
28     }
29     for(auto [y,i]:ask[x]){
30         ans[i] = find(y);
31     }
32 }
33
34 int main(){
35     memset(h,-1,sizeof h);
36     cin >> n >> q >> root;
37     for(int i = 1;i <= n;i++) p[i] = i;
38     for(int i = 1;i < n;i++){
39         int a,b;scanf("%d %d",&a,&b);
40         add(a,b);add(b,a);
41     }
42     for(int i = 1;i <= q;i++){
43         int a,b;scanf("%d %d",&a,&b);
44         if(a == b) ans[i] = a;
45         else{
46             ask[a].push_back({b,i});
47             ask[b].push_back({a,i});
48         }
49     }
50     tarjan(root,0);
51     for(int i = 1;i <= q;i++){
52         printf("%d\n",ans[i]);
53     }
54 }

```

树上差分

最近公共祖先的应用，树上差分可以理解为对树上的某一段路径进行差分操作，这里的路径可以类比一维数组的区间进行理解。适合多次修改($O(\log N)$)，单次查询($O(N)$ dfs统计信息)。

点差分

[P3128 [USACO15DEC\] Max Flow P - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P3128)

每次将 a 到 b 简单路径上的点权加上 x ，则 $val[a] += x, val[b] += x, val[lca] -= x, val[fa[lca]] -= x$ 。因为回溯统计时 lca 会被加上 $2x$ ，所以我们需要让 lca 减去 x ，再让 lca 的父节点减去 x 。

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  const int N = 50004,M = 100005;
5  int n,q,ans;
6  int h[N],e[M],ne[M],idx;
7  int fa[N][22],deep[N];
8  int val[N];
9
10 void add(int a,int b){
11     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
12 }
13
14 void dfs(int u,int father){
15     fa[u][0] = father;
16     deep[u] = deep[father] + 1;
17     for(int i = 1;(1 << i) <= deep[u];i++){
18         fa[u][i] = fa[fa[u][i-1]][i-1];
19     }
20     for(int i = h[u];~i;i = ne[i]){
21         int k = e[i];
22         if(k != father) dfs(k,u);
23     }
24 }
25
26 int lca(int x,int y){
27     if(deep[x] < deep[y]) swap(x,y);
28     for(int i = 20;i >= 0;i--){
29         if(deep[fa[x][i]] >= deep[y]) x = fa[x][i];
30     }
31     if(x == y) return x;
32     for(int i = 20;i >= 0;i--){
33         if(fa[x][i] != fa[y][i]){
34             x = fa[x][i];
35             y = fa[y][i];
36         }
37     }
38     return fa[x][0];
39 }
40
41 void dfs2(int u,int father){
42     for(int i = h[u];~i;i = ne[i]){
43         int k = e[i];
44         if(k != father){
45             dfs2(k,u);
46             val[u] += val[k];
47         }
48     }
49     ans = max(ans,val[u]);
50 }
51
52 int main(){

```



```

53     memset(h,-1,sizeof h);
54     cin >> n >> q;
55     for(int i = 1;i < n;i++){
56         int a,b;cin >> a >> b;
57         add(a,b);add(b,a);
58     }
59     dfs(1,0);
60     while(q--){
61         int a,b;cin >> a >> b;
62         int father = lca(a,b);
63         val[a]++;val[b]++;
64         val[father]--;val[fa[father][0]]--;
65     }
66     dfs2(1,0);
67     cout << ans;
68 }

```

边差分

与点差分略有不同，将边差分转换为点差分。

若要将a到b上简单路径的边权加x，则 $\text{dist}[a] += x, \text{dist}[b] += x, \text{dist}[\text{lca}] -= 2x$ 。

树链剖分

树链剖分（Heavy-Light Decomposition, HLD）用于将树分割成若干条链的形式，从而将树上路径、子树问题转化为序列上的区间问题，使得可以用线段树等数据结构进行处理。

重链剖分

定义 **重子节点** 表示其子节点中子树最大的子结点。如果有多个子树最大的子结点，取其一。如果没有子节点，就无重子节点。

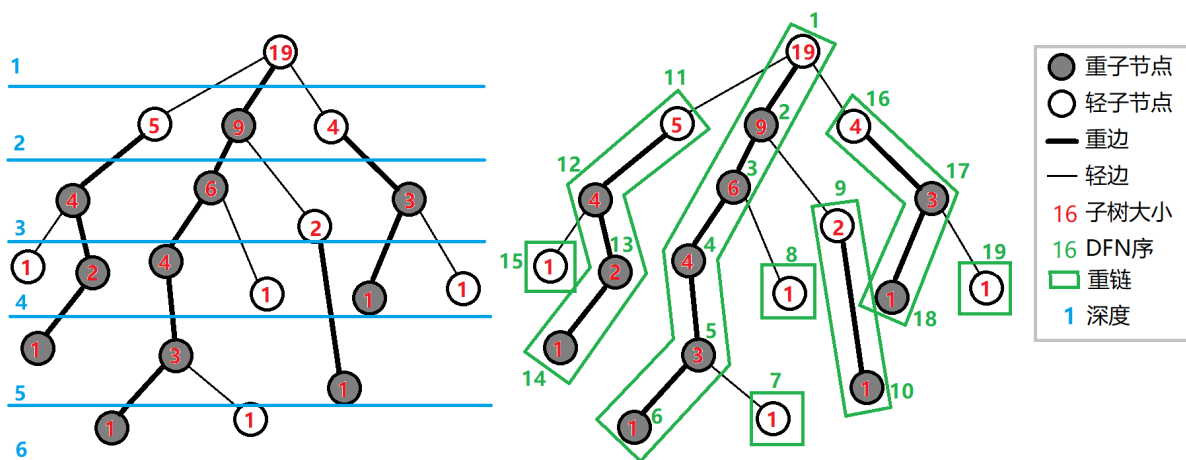
定义 **轻子节点** 表示剩余的所有子结点。

从这个结点到重子节点的边为 **重边**。

到其他轻子节点的边为 **轻边**。

若干条首尾衔接的重边构成 **重链**。

把落单的结点也当作重链，那么整棵树就被剖分成若干条重链。



实现

第一次DFS记录每个节点的父节点(father)、深度(deep)、子树大小(size)、重子节点(hson)。

第二次DFS记录所在链的链顶(top, 应初始化为本身)、重边优先遍历时的DFS序(dfnc)、DFS序对应的节点编号(rank)

重链剖分的性质

树上每个节点都属于且仅属于一条重链。

重链开头的节点不一定是重子节点（因为重边是对于每一个节点都有定义的）。

所有的重链将整棵树 完全剖分。

在剖分时 重边优先遍历，最后树的DFS序上，重链内的DFS序是连续的。按DFN排序后的序列即为剖分后的链。

一颗子树内的DFS序是连续的。

可以发现，但我们向下经过一条轻边时，所在子树的大小至少会除以二。

因此，对于树上任意一条路径，把它拆分成从LCA分别向两边往下走，分别最多走 $O(\log n)$ 次，因此，树上的每条路径都可以被拆分成不超过 $O(\log n)$ 条重链。

P3384 【模板】重链剖分/树链剖分 - 洛谷

给定一颗包含 n 个节点的数，根为 $root$ ，执行 q 次以下操作，输出结果对 mod 取模。

- 1 $x\ y\ z$ 将树从 x 到 y 节点最短路径上的所有节点的值都加上 z 。
- 2 $x\ y$ 求树从 x 到 y 节点最短路径上的所有节点的值之和。
- 3 $x\ z$ 将以 x 为根节点的子树内所有节点的值都加上 z 。
- 4 x 求以 x 为根节点的子树内所有节点值之和。

```
1 #include <bits/stdc++.h>
```

```

2
3  const int N = 100005, M = N << 1;
4  int n,q,root,mod;
5  long long a[N];
6
7  int h[N],e[M],ne[M],idx;
8  void add(int a,int b){
9      e[idx] = b,ne[idx] = h[a],h[a] = idx++;
10 }
11
12 int son[N],siz[N],fa[N],dep[N];
13 int id[N],idn,top[N],rnk[N];
14 void dfs1(int x,int father){
15     fa[x] = father;
16     dep[x] = dep[father] + 1;
17     siz[x] = 1;
18     for(int i = h[x];~i;i = ne[i]){
19         int y = e[i];
20         if(y == father) continue;
21         dfs1(y,x);
22         siz[x] += siz[y];
23         if(siz[y] > siz[son[x]]) { son[x] = y; } //求重儿子
24     }
25 }
26 void dfs2(int x,int father){
27     top[x] = father; //指向链顶
28     id[x] = ++idn;
29     rnk[idn] = x;
30     if(son[x] == 0) return;
31     dfs2(son[x],father); // 优先对重儿子进行 DFS,可以保证同一条重链上的点 DFS 序连续
32     for(int i = h[x];~i;i = ne[i]){
33         int y = e[i];
34         if(y == son[x] || y == fa[x]) continue;
35         dfs2(y,y);
36     }
37 }
38
39 int lca(int x,int y){ //链剖求LCA
40     while(top[x] != top[y]) {
41         if(dep[top[x]] > dep[top[y]]) x = fa[top[x]]; //选择链顶深度较深的一条链往上
42         //跳
43         else y = fa[top[y]];
44     }
45     return dep[x] < dep[y] ? x : y; //处于同一条链时,深度较浅的节点即为最近公共祖先
46 }
47
48 int dist(int x,int y) { //求两点之间距离
49     return dep[x] + dep[y] - 2 * dep[lca(x,y)];
50 }
51
52 struct Segtree{
53     int n;

```

```

53     struct ST{
54         int l,r;
55         long long add;
56         long long sum;
57     };
58
59     std::vector<ST>t;
60     Segtree(){}
61
62     Segtree(int n_){
63         n = n_;
64         t.assign((n<<2)+2,{});
65         build(1,1,n);
66     }
67
68     void build(int p,int l,int r){
69         t[p] = {l,r};
70         if(l == r){build(t[p],l);return;}
71         int mid = l + r >> 1;
72         build(p<<1,l,mid);build(p<<1|1,mid+1,r);
73         pushup(t[p],t[p<<1],t[p<<1|1]);
74     }
75
76     void modify(int p,int l,int r,long long x){
77         if(l <= t[p].l && r >= t[p].r) {update(t[p],x);return;}
78         pushdown(t[p],t[p<<1],t[p<<1|1]);
79         int mid = t[p].l + t[p].r >> 1;
80         if(l <= mid) modify(p<<1,l,r,x);
81         if(r > mid) modify(p<<1|1,l,r,x);
82         pushup(t[p],t[p<<1],t[p<<1|1]);
83     }
84
85     ST query(int p,int l,int r){
86         if(l <= t[p].l && r >= t[p].r){return t[p];}
87         pushdown(t[p],t[p<<1],t[p<<1|1]);
88         int mid = t[p].l + t[p].r >> 1;
89         if(r <= mid) return query(p<<1,l,r);
90         if(l > mid) return query(p<<1|1,l,r);
91         ST pl = query(p<<1,l,r),pr = query(p<<1|1,l,r),ans;
92         pushup(ans,pl,pr);
93         return ans;
94     }
95
96     void modify(int l,int r,long long x){modify(1,l,r,x);}
97
98     ST query(int l,int r){return query(1,l,r);}
99
100    void pushdown(ST &p,ST &pl,ST &pr){
101        auto &x = p.add;
102        if(x) {
103            update(pl,x);
104            update(pr,x);

```

```

105         x = 0;
106     }
107 }
108
109 void update(ST &p, long long x){
110     p.sum = (p.sum + (p.r - p.l + 1) * x) % mod;
111     p.add = (p.add + x) % mod;
112 }
113
114 void pushup(ST &p, ST &pl, ST &pr){
115     p.sum = (pl.sum + pr.sum) % mod;
116 }
117
118 void build(ST &p, int i){
119     p.sum = a[rnk[i]] % mod; //初始化为a[rnk[i]],而非a[i]
120 }
121 }t;
122
123 long long path_sum(int x, int y){
124     long long ans = 0;
125     while(top[x] != top[y]){
126         if(dep[top[x]] < dep[top[y]]) std::swap(x, y); //选择链顶深度较深的一条链
127         ans = (ans + t.query(id[top[x]], id[x]).sum) % mod; //区间[x到其链顶]快速计
算
128         x = fa[top[x]]; //往上跳到链顶的父节点,直到两点在同一条链上
129     }
130     if(id[x] > id[y]) std::swap(x, y); //保证查询时 x为左端点
131     ans = (ans + t.query(id[x], id[y]).sum) % mod; //同一条链快速计算
132     return ans;
133 }
134
135 void path_add(int x, int y, int z){
136     while(top[x] != top[y]) {
137         if(dep[top[x]] < dep[top[y]]) std::swap(x, y);
138         t.modify(id[top[x]], id[x], z);
139         x = fa[top[x]];
140     }
141     if(id[x] > id[y]) std::swap(x, y);
142     t.modify(id[x], id[y], z);
143 }
144
145 int subtree_sum(int x) { //子树必为连续一段区间[id[x], id[x]+siz[x]-1]
146     return t.query(id[x], id[x]+siz[x]-1).sum;
147 }
148
149 void subtree_add(int x, int z){
150     t.modify(id[x], id[x]+siz[x]-1, z);
151 }
152
153 int main(){
154     std::memset(h, -1, sizeof h);
155     std::cin >> n >> q >> root >> mod;

```

```

156     for(int i = 1; i <= n; i++) std::cin >> a[i];
157     for(int i = 1; i < n; i++) {
158         int a, b; std::cin >> a >> b;
159         add(a, b); add(b, a);
160     }
161
162     dfs1(root, 0);
163     dfs2(root, root);
164     t = Segtree(n); //dfs后再建线段树
165
166     while(q--) {
167         int op; std::cin >> op;
168         if(op == 1){
169             int x, y, z; std::cin >> x >> y >> z;
170             path_add(x, y, z);
171         }
172         if(op == 2){
173             int x, y; std::cin >> x >> y;
174             std::cout << path_sum(x, y) << '\n';
175         }
176         if(op == 3){
177             int x, z; std::cin >> x >> z;
178             subtree_add(x, z);
179         }
180         if(op == 4){
181             int x; std::cin >> x;
182             std::cout << subtree_sum(x) << '\n';
183         }
184     }
185 }

```

若要对边权进行操作，可以将边权存在dfn序较大的点权中<https://vjudge.net/problem/FZU-2082>

```

1 //修改第i条边的边权, 查询路径x到y的边权和
2 #include <iostream>
3 #include <vector>
4 #include <cstring>
5
6 const int N = 50004;
7 int n, q;
8 int a[N];
9
10 struct Edge{
11     int a, b, c;
12 }edge[N];
13
14 const int M = N << 1;
15 int h[N], e[M], ne[M], idx;
16 void add(int a, int b) {
17     e[idx] = b, ne[idx] = h[a], h[a] = idx++;

```

```

18 }
19
20 int siz[N], dep[N], son[N], fa[N];
21 void dfs1(int x, int father) {
22     fa[x] = father;
23     dep[x] = dep[father] + 1;
24     siz[x] = 1;
25     for (int i = h[x]; ~i; i = ne[i]) {
26         int y = e[i];
27         if (y == father) continue;
28         dfs1(y, x);
29         siz[x] += siz[y];
30         if (siz[y] > siz[son[x]]) son[x] = y;
31     }
32 }
33
34 int id[N], idn, rnk[N], top[N];
35 void dfs2(int x, int father) {
36     top[x] = father;
37     id[x] = ++idn;
38     rnk[idn] = x;
39
40     if (!son[x]) return;
41     dfs2(son[x], father);
42
43     for (int i = h[x]; ~i; i = ne[i]) {
44         int y = e[i];
45         if (y == son[x] || y == fa[x]) continue;
46         dfs2(y, y);
47     }
48 }
49
50 struct ST{
51     int l, r;
52     long long sum;
53 }t[N << 2];
54
55 void pushup(ST &p, ST &pl, ST &pr){
56     p.sum = pl.sum + pr.sum;
57 }
58
59 void pushup(int p) {
60     pushup(t[p], t[p << 1], t[p << 1 | 1]);
61 }
62
63 void build(int p, int l, int r) {
64     t[p] = {l, r};
65     if (l == r) {
66         t[p].sum = a[rnk[l]];
67         return;
68     }
69     int mid = l + r >> 1;

```

```

70     build(p << 1, l, mid);
71     build(p << 1 | 1, mid + 1, r);
72     pushup(p);
73 }
74
75 void modify(int p, int l, int r, int x) {
76     if (l <= t[p].l && r >= t[p].r) {
77         t[p].sum = x;
78         return;
79     }
80     int mid = t[p].l + t[p].r >> 1;
81     if (l <= mid) modify(p << 1, l, r, x);
82     if (r > mid) modify(p << 1 | 1, l, r, x);
83     pushup(p);
84 }
85
86 ST query(int p, int l, int r) {
87     if (l <= t[p].l && r >= t[p].r) {
88         return t[p];
89     }
90     int mid = t[p].l + t[p].r >> 1;
91     if (r <= mid) return query(p << 1, l, r);
92     if (l > mid) return query(p << 1 | 1, l, r);
93     ST pl = query(p << 1, l, r), pr = query(p << 1 | 1, l, r), ans;
94     pushup(ans, pl, pr);
95     return ans;
96 }
97
98 long long path_query(int x, int y) {
99     long long ans = 0;
100     while (top[x] != top[y]) {
101         if (dep[top[x]] < dep[top[y]]) std::swap(x, y);
102         ans += query(1, id[top[x]], id[x]).sum;
103         x = fa[top[x]];
104     }
105     if (id[x] > id[y]) std::swap(x, y);
106     if (id[x] + 1 > id[y]) return ans;
107     ans += query(1, id[x]+1, id[y]).sum; //跳到同一个链时,累加上区间id[x]+1到id[y]
    的值即为答案
108     return ans;
109 }
110
111 void sol(){
112     idx = idn = 0;
113     std::memset(h, -1, sizeof h);
114     std::memset(son, 0, sizeof son);
115     for (int i = 1; i < n; i++) {
116         int x, y, z; std::cin >> x >> y >> z;
117         add(x, y);
118         add(y, x);
119         edge[i] = {x, y, z};
120     }

```



```

121
122     dfs1(1,0);
123     dfs2(1,1);
124     for (int i = 1; i < n; i++) {
125         auto &[x, y, z] = edge[i];
126         if (id[x] < id[y]) std::swap(x,y);
127         a[x] = z;
128     }
129     build(1,1,n);
130
131     while (q--) {
132         int op; std::cin >> op;
133         if (op == 0) {
134             int i, k; std::cin >> i >> k;
135             if (i >= n) continue;
136             auto &[x,y,z] = edge[i];
137             modify(1, id[x], id[x], k);
138         }
139         else {
140             int x, y; std::cin >> x >> y;
141             std::cout << path_query(x, y) << '\n';
142         }
143     }
144 }
145
146 int main(){
147     std::ios::sync_with_stdio(false); std::cin.tie(0);
148     while (std::cin >> n >> q) sol();
149 }

```

树哈希

判断一些树是否同构的时，我们常常把这些树转成哈希值储存起来，以降低复杂度。

这里介绍一个实现简单且不容易被卡的做法，这类方法需要一个多重集的哈希函数。以某个结点为根的子树的哈希值，就是以它的所有儿子为根的子树的哈希值构成的多重集的哈希值，即：

$$h_x = f(\{h_i \mid i \in \text{son}(x)\})$$

其中 h_x 表示以 x 为根的子树的哈希值， f 是多重集的哈希函数。

以代码中使用的哈希函数为例：

$$f(S) = \left(c + \sum_{x \in S} g(x) \right) \bmod m$$

其中 c 为常数，一般使用 1 即可。 m 为模数，一般使用 2^{32} 或 2^{64} 进行自然溢出，也可使用大素数。 g 为整数到整数的映射，代码中使用 xor shift，也可以选用其他的函数，但是不建议使用多项式。为了预防出题人对着 xor hash 卡，还可以在映射前后异或一个随机常数。

这种哈希十分好写，比每次使用 `mt19937_64(hs[k])()` 生成随机数要快。如果需要换根，第二次 DP 时只需把子树哈希减掉即可。

```
1 //有根树哈希 https://uoj.ac/problem/763
2 //给定一棵以1为根的树，你需要输出这棵树中最多能选出多少个互不同构的子树。
3 #include <iostream>
4 #include <set>
5 #include <cstring>
6 #include <random>
7 using namespace std;
8 const int N = 1000006, M = N << 1;
9 int n;
10 int h[N], e[M], ne[M], idx;
11 unsigned long long hs[N];
12 set<unsigned long long> se;
13 const unsigned long long mask = mt19937_64(time(0))();
14
15 void add(int a, int b){
16     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
17 }
18
19 unsigned long long shift(unsigned long long x){
20     x ^= mask;
21     x ^= x << 13;
22     x ^= x >> 7;
23     x ^= x << 17;
24     x ^= mask;
25     return x;
26 }
27
28 void dfs(int u, int fa){
29     hs[u] = 1;
30     for(int i = h[u]; ~i; i = ne[i]){
31         int k = e[i];
32         if(k == fa) continue;
33         dfs(k, u);
34         hs[u] += shift(hs[k]);
35     }
36     se.insert(hs[u]);
37 }
38
39 int main(){
40     memset(h, -1, sizeof h);
41     cin >> n;
42     for(int i = 1; i < n; i++){
43         int a, b; cin >> a >> b;
44         add(a, b); add(b, a);
45     }
```

```

46     dfs(1,0);
47     cout << se.size();
48 }

```

生成树

最小生成树

无向连通图的 **最小生成树** (Minimum Spanning Tree, MST) 为边权和最小的生成树。

从图(一般为无向有环图)中生成一棵树($n-1$ 条边)时, 这棵树每条边的权重相加之和最小。

只有连通图才有生成树, 而对于非连通图, 只存在生成森林。

Kruskal

适合稀疏图 $O(M \log M)$ **结构体存边** **并查集实现**

将边权从小到大排序, 遍历边, 若当前边的两个端点属于不同的树, 则合并这两颗树

Kruskal 重构树的一些性质

- 是一棵二叉树。
- 如果是按最小生成树建立的话是一个大根堆。
- 原图中两个点间所有路径上的边最大权值的最小值 = 最小生成树上两点简单路径的边最大权值 = Kruskal 重构树上两点 LCA 的点权。

```

1  //https://www.acwing.com/problem/content/description/861/
2  #include <iostream>
3  #include <algorithm>
4  using namespace std;
5  const int N = 100005, M = 2 * N;
6  int n, m;
7  int p[N]; //保存并查集
8  int res, cnt;
9  struct Edge {
10     int a, b, w;
11 } edges[M];
12
13 bool cmp(Edge E1, Edge E2) { //自定义sort排序, 以结构体w值为依据排序
14     return E1.w < E2.w;
15 }
16
17 int find(int x) { //并查集
18     if (p[x] != x) p[x] = find(p[x]);
19     return p[x];
20 }
21

```

```

22 void kruskal() {
23     for (int i = 0; i < m; i++) {
24         int pa = find(edges[i].a), pb = find(edges[i].b);
25         if (pa != pb) { //如果这个边与之前选择的所有边不会成环, 就选择这条边
26             p[pa] = pb; //合并
27             res += edges[i].w;
28             cnt++;
29         }
30     }
31 }
32
33 int main() {
34     cin >> n >> m;
35     for (int i = 1; i <= n; i++) { p[i] = i; } //初始化并查集
36
37     for (int i = 0; i < m; i++) {
38         int a, b, w; cin >> a >> b >> w;
39         edges[i] = { a,b,w };
40     }
41     sort(edges, edges + m, cmp); //将所有m条边按照权值的大小进行升序排序
42
43     kruskal();
44
45     if (cnt < n - 1) puts("impossible");
46     else cout << res << endl;
47 }

```

Prim朴素

适合稠密图 $O(N^2)$

```

1 //https://www.acwing.com/problem/content/860/
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5 const int N = 510, inf = 0x3f3f3f3f;
6 int n, m;
7 int dist[N], g[N][N]; //dist存点到集合s的距离
8 bool vis[N];
9
10 int prim(){ //l_idx
11     std::memset(dist, 0x3f, sizeof dist);
12     std::memset(vis, 0, sizeof vis);
13     int ans = 0;
14     for(int i = 0; i < n; i++){
15         int t = 0;
16         for(int j = 1; j <= n; j++){
17             if(!vis[j] && (t == 0 || dist[t] > dist[j])) {

```

```

18         t = j; //寻找离集合s最近的点t
19     }
20 }
21 vis[t] = 1;
22 if(i && dist[t] == 0x3f3f3f3f) return -1; //判断是否连通
23 if(i) ans += dist[t]; //加入点t
24 for(int j = 1; j <= n; j++){
25     dist[j] = std::min(dist[j], g[t][j]); //再用点t更新其它点到集合的距离
26 }
27 }
28 return ans;
29 }
30
31 int main() {
32     cin >> n >> m;
33     memset(g, 0x3f, sizeof g);
34     while (m--){
35         int a, b, c; cin >> a >> b >> c;
36         g[a][b] = g[b][a] = min(g[a][b], c); //处理重边
37     }
38     int t = prim();
39     if (t == -1) puts("impossible");
40     else cout << t;
41 }

```

若prim要输出具体选择的边，可以开一个辅助数组 last[]，更新 dist[] 时记录当前点从哪个点转移的，ans+=dist[t] 时即选择了边 (t, last)。

Prim堆优化

比较复杂不常用，直接用Kruskal算法

Boruvka

Kruskal和Prim的结合，在边具有较多特殊性质的问题中，Boruvka算法具有优势。

唯一性

判断最小生成树的唯一性。如果一条边 **不在最小生成树的边集中**，并且可以替换与其 **权值相同、并且在最小生成树边集** 的另一条边。那么，这个最小生成树就是不唯一的。

对于 Kruskal 算法，只要计算为当前权值的边可以放几条，实际放了几条，如果这两个值不一样，那么就说明这几条边与之前的边产生了一个环（这个环中至少有两条当前权值的边，否则根据并查集，这条边是不能放的），即最小生成树不唯一。

寻找权值与当前边相同的边，我们只需要记录头尾指针，用单调队列即可在 $O(\alpha(m))$ (m 为边数) 的时间复杂度里优秀解决这个问题（基本与原算法时间相同）。

```
1 //https://vjudge.net/problem/OpenJ_Bailian-1679#author=GPT_zh
2 #include <iostream>
3 #include <algorithm>
4 using namespace std;
5 const int N = 105,M = 10004;
6 int n,m;
7 int p[N];
8
9 struct edge{
10     int a,b,w;
11     bool operator < (edge &e2){
12         return w < e2.w;
13     }
14 }e[M];
15
16 int find(int x){
17     if(p[x] != x) p[x] = find(p[x]);
18     return p[x];
19 }
20
21 void soviet(){
22     cin >> n >> m;
23     for(int i = 1;i <= n;i++) p[i] = i;
24     for(int i = 1;i <= m;i++){
25         cin >> e[i].a >> e[i].b >> e[i].w;
26     }
27     sort(e+1,e+m+1);
28
29     int ans = 0,cnt = 0;
30     bool flag = 1;
31     int tail = 0,sum1 = 0,sum2 = 0;
32     for(int i = 1;i <= m+1;i++){
33         if(i > tail){
34             if(sum1 != sum2){//如果sum1 != sum2 则最小生成树不唯一
35                 flag = 0;
36                 break;
37             }
38             sum1 = sum2 = 0;
39             for(int j = i;j <= m+1;j++){
40                 if(j > m || e[j].w != e[i].w){
41                     tail = j-1;
42                     break;
43                 }
44                 if(find(e[j].a) != find(e[j].b)) sum1++;//sum1计算当前权值的边可以
放几条
45             }
46         }
47         if(i > m) break;
```

```

48     int pa = find(e[i].a), pb = find(e[i].b);
49     if(pa != pb && cnt != n-1){
50         sum2++; //sum2计算当前权值的边实际放了几条
51         cnt++;
52         p[pa] = pb;
53         ans += e[i].w;
54     }
55 }
56 if(flag) cout << ans << '\n';
57 else cout << "Not Unique!\n";
58 }
59
60 int main(){
61     int mt; cin >> mt;
62     while(mt--) soviet();
63 }

```

最小瓶颈路

[#136. 最小瓶颈路 - 题目 - LibreOJ](#)

给定一张无向图，每次询问点对 (u, v) ，求 u 到 v 的简单路径中，经过的边权最大值最小的一条，输出该值。

我们可以求出最小生成树，重新建图，答案即为两点之间所经过边权的最大值。

时间复杂度： $O(m \log m)$ 排序(求最小生成树)， $O(n \log n)$ 预处理 f_a, mx ， $O(q \log n)$ 回答询问。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int N = 1003, M = 200005;
5
6  int h[N], e[M], ne[M], w[M], idx;
7  void add(int a, int b, int c) {
8      w[idx] = c, e[idx] = b, ne[idx] = h[a], h[a] = idx++;
9  }
10
11 struct Edge {
12     int u, v, w;
13     bool operator < (const auto &o) {
14         return w < o.w;
15     }
16 };
17
18 struct DSU {
19     int n;
20     vector<int> p;
21
22     DSU(int _n): n(_n) {

```

```

23     p.resize(n + 1);
24     for (int i = 0; i <= n; i++) p[i] = i;
25 }
26
27 int find(int x) {
28     if (p[x] != x) p[x] = find(p[x]);
29     return p[x];
30 }
31
32 void merge(int x, int y) {
33     x = find(x), y = find(y);
34     if (x == y) return;
35     p[x] = y;
36 }
37 };
38
39 int fa[N][21], deep[N], mx[N][21]; //预处理fa, mx
40 void dfs(int x, int father, int weight) {
41     fa[x][0] = father;
42     deep[x] = deep[father] + 1;
43     mx[x][0] = weight;
44
45     for (int i = 1; (1 << i) <= deep[x]; i++) {
46         mx[x][i] = max(mx[fa[x][i - 1]][i - 1], mx[x][i - 1]);
47         fa[x][i] = fa[fa[x][i - 1]][i - 1];
48     }
49     for (int i = h[x]; ~i; i = ne[i]) {
50         int y = e[i];
51         if (y == father) continue;
52         dfs(y, x, w[i]);
53     }
54 }
55
56
57 int path_query(int x, int y) { //返回x到y路径的最大边权
58     int ans = 0;
59     if (deep[x] < deep[y]) swap(x, y);
60     for (int i = 20; i >= 0; i--) {
61         if (deep[fa[x][i]] >= deep[y]) {
62             ans = max(ans, mx[x][i]);
63             x = fa[x][i];
64         }
65     }
66     if (x == y) return ans;
67     for (int i = 20; i >= 0; i--) {
68         if (fa[x][i] != fa[y][i]) {
69             ans = max({ans, mx[x][i], mx[y][i]});
70             x = fa[x][i];
71             y = fa[y][i];
72         }
73     }
74     ans = max({ans, mx[x][0], mx[y][0]});

```



```

75     return ans;
76 }
77
78 int main() {
79     memset(h, -1, sizeof h);
80     int n, m, q;
81     cin >> n >> m >> q;
82     vector<Edge> edge(m);
83     for (int i = 0; i < m; i++) {
84         int u, v, w; cin >> u >> v >> w;
85         edge[i] = {u, v, w};
86     }
87
88     //求最小生成树,重新建图
89     sort(edge.begin(), edge.end());
90     DSU dsu(n);
91     for (auto &[x, y, w]: edge) {
92         int px = dsu.find(x), py = dsu.find(y);
93         if (px == py) continue;
94         else {
95             add(x, y, w);
96             add(y, x, w);
97             dsu.merge(x, y);
98         }
99     }
100
101     for (int i = 1; i <= n; i++) {
102         if (!deep[i]) { //图不一定连通,可能为若干个森林
103             dfs(i, 0, 0);
104         }
105     }
106
107     while (q--) {
108         int x, y; cin >> x >> y;
109         int px = dsu.find(x), py = dsu.find(y);
110         if (px != py) {
111             cout << -1 << '\n';
112             continue;
113         }
114         cout << path_query(x, y) << '\n';
115     }
116 }

```

次小生成树

非严格次小生成树

求解方法

- 求出无向图的最小生成树 T ，设其权值和为 M 。
- 遍历每条未被选中的边 $e = (u, v, w)$ ，找到 T 中 u 到 v 的路径上边权最大的一条边 $e' = (s, t, w')$ ，在 T 中用 e 替换 e' ，即可得到一颗生成树 T' ，其权值为 $M' = M + w - w'$ 。
- 对所有的 M' 取最小值即为答案。

其中求 u, v 路径上边权最大的值可以在倍增求 LCA 的过程中求得。

```
1 //ACM Contest and Blackout https://vjudge.net/problem/UVA-10600
2 //模版题,分别输出无向图的最小生成树和非严格次小生成树的权值,本题保证答案存在
3 #include <iostream>
4 #include <cstring>
5 #include <vector>
6 #include <algorithm>
7
8 const int N = 105, M = N*N;
9 int n, m;
10 int fa[N][22], deep[N], mx[N][22];
11 int h[N], e[M], ne[M], w[M], idx;
12
13 void add(int a, int b, int c) {
14     w[idx] = c, e[idx] = b, ne[idx] = h[a], h[a] = idx++;
15 }
16
17 int p[N];
18 int find(int x) {
19     if (p[x] != x) p[x] = find(p[x]);
20     return p[x];
21 }
22
23 struct Edge {
24     int a, b, c;
25     bool operator < (const Edge &e2) const {
26         return c < e2.c;
27     }
28 };
29
30 void dfs(int u, int father, int weight) { //mx[u][i]表示u到其第2^i个父节点的最大权值
31     deep[u] = deep[father] + 1;
32     fa[u][0] = father;
33     mx[u][0] = weight;
34
35     for (int i = 1; (1 << i) <= deep[u]; i++) {
36         mx[u][i] = std::max(mx[fa[u][i-1]][i-1], mx[u][i-1]);
37         fa[u][i] = fa[fa[u][i-1]][i-1];
38     }
```

```

39     for(int i = h[u]; ~i; i = ne[i]){
40         if(e[i] != father) dfs(e[i], u, w[i]);
41     }
42 }
43
44 int lca(int a, int b){ //求a,b路径上的最大权值
45     int ans = 0;
46     if(deep[a] < deep[b]) std::swap(a, b);
47     for(int i = 20; i >= 0; i--){
48         if(deep[fa[a][i]] >= deep[b]){
49             ans = std::max(ans, mx[a][i]); //注意先更新ans再更新a
50             a = fa[a][i];
51         }
52     }
53     if(a == b) return ans;
54     for(int i = 20; i >= 0; i--){
55         if(fa[a][i] != fa[b][i]){
56             ans = std::max({ans, mx[a][i], mx[b][i]});
57             a = fa[a][i];
58             b = fa[b][i];
59         }
60     }
61     ans = std::max({ans, mx[a][0], mx[b][0]});
62     return ans;
63 }
64
65 void sol(){
66     std::memset(h, -1, sizeof h);
67     std::memset(fa, 0, sizeof fa); //多测需清空fa数组
68     idx = 0;
69     std::cin >> n >> m;
70     for(int i = 1; i <= n; i++) p[i] = i;
71     std::vector<Edge> edge;
72     for(int i = 1; i <= m; i++){
73         int a, b, c; std::cin >> a >> b >> c;
74         edge.push_back({a, b, c});
75     }
76     std::sort(edge.begin(), edge.end());
77
78     int ans1 = 0;
79     std::vector<Edge> T2;
80     for(auto &[a, b, c]: edge){
81         int pa = find(a), pb = find(b);
82         if(pa == pb) { T2.push_back({a, b, c}); }
83         else{
84             p[pa] = pb;
85             ans1 += c;
86             add(a, b, c); add(b, a, c);
87         }
88     }
89     //if(cnt != n-1) {return;} 不存在最小生成树
90     //if(T2.empty()) {return;} 不存在次小生成树

```

```

91     dfs(1,0,0);
92     int ans2 = 1e9;
93     for(auto &[a,b,c]:T2){//遍历所有未选择的边
94         int now = ans1 + c - lca(a,b);//替换最小生成树中(a,b)路径上权值最大的一条边
95         ans2 = std::min(ans2,now);//答案取最小
96     }
97     std::cout << ans1 << ' ' << ans2 << '\n';
98 }
99
100 int main(){
101     int mt;std::cin >> mt;
102     while(mt--) sol();
103 }

```

严格次小生成树

在用未选择的边 $e = (u, v, w)$ 替换最小生成树 T 上的边时，找出 T 上 (u, v) 路径中边权严格小于 w 的最大的一条边进行替换。

同时维护节点到祖先路径上的最大边权和**严格次大边权**即可。

```

1 //https://www.luogu.com.cn/problem/P4180
2 //模版,本题保证严格次小生成树存在
3 #include <iostream>
4 #include <cstring>
5 #include <vector>
6 #include <algorithm>
7
8 const int INF = 0x3f3f3f3f;
9 const int N = 100005,M = 600005;
10 int n,m;
11 int h[N],e[M],ne[M],w[M],idx;
12 int fa[N][20],deep[N],mx[N][20],mx2[N][20];//mx:最大边权,mx2:次大边权
13
14 struct Edge{
15     int a,b,c;
16     bool operator < (const Edge &e2)const{
17         return c < e2.c;
18     }
19 };
20
21 void add(int a,int b,int c){
22     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
23 }
24
25 int p[N];
26 int find(int x){

```

```

27     if(p[x] != x) p[x] = find(p[x]);
28     return p[x];
29 }
30
31 void dfs(int u,int father,int weight){
32     deep[u] = deep[father] + 1;
33     fa[u][0] = father;
34     mx[u][0] = weight;
35     mx2[u][0] = -INF; //-INF代表不存在次大边权
36     for(int i = 1;(1 << i) <= deep[u];i++){
37         mx[u][i] = std::max(mx[fa[u][i-1]][i-1],mx[u][i-1]);
38         mx2[u][i] = std::max(mx2[fa[u][i-1]][i-1],mx2[u][i-1]);
39
40         if(mx[u][i-1] > mx[fa[u][i-1]][i-1]) mx2[u][i] = std::max(mx2[u]
[i],mx[fa[u][i-1]][i-1]);
41         else if(mx[u][i-1] < mx[fa[u][i-1]][i-1]) mx2[u][i] = std::max(mx2[u]
[i],mx[u][i-1]);
42
43         fa[u][i] = fa[fa[u][i-1]][i-1];
44     }
45     for(int i = h[u];~i;i = ne[i]){
46         if(e[i] != father) dfs(e[i],u,w[i]);
47     }
48 }
49
50 int lca(int a,int b,int c){
51     int ans = -INF;
52     if(deep[a] < deep[b]) std::swap(a,b);
53     for(int i = 18;i >= 0;i--){
54         if(deep[fa[a][i]] >= deep[b]) {
55             if(mx[a][i] < c) ans = std::max(ans,mx[a][i]);
56             else ans = std::max(ans,mx2[a][i]);
57             a = fa[a][i];
58         }
59     }
60     if(a == b) return ans;
61     for(int i = 18;i >= 0;i--){
62         if(fa[a][i] != fa[b][i]){
63             ans = std::max(ans,mx[a][i] < c ? mx[a][i] : mx2[a][i]);
64             ans = std::max(ans,mx[b][i] < c ? mx[b][i] : mx2[b][i]);
65             a = fa[a][i];
66             b = fa[b][i];
67         }
68     }
69     ans = std::max(ans,mx[a][0] < c ? mx[a][0] : mx2[a][0]);
70     ans = std::max(ans,mx[b][0] < c ? mx[b][0] : mx2[b][0]);
71     return ans;
72 }
73
74 int main(){
75     std::memset(h,-1,sizeof h);
76     std::cin >> n >> m;

```

```

77     for(int i = 1;i <= n;i++) p[i] = i;
78     std::vector<Edge>edge;
79     for(int i = 1;i <= m;i++){
80         int a,b,c;std::cin >> a >> b >> c;
81         edge.push_back({a,b,c});
82     }
83     std::sort(edge.begin(),edge.end());
84
85     std::vector<Edge>T2;
86     int cnt = 0;
87     long long ans1 = 0;
88     for(auto &[a,b,c]:edge){
89         int pa = find(a),pb = find(b);
90         if(pa == pb) {
91             T2.push_back({a,b,c});
92         }
93         else{
94             p[pa] = pb;
95             ans1 += c;
96             cnt++;
97             add(a,b,c); add(b,a,c);
98         }
99     }
100
101     // if(cnt != n-1) {return 0;} //无最小生成树
102     // if(T2.empty()) {return 0;} //无次小生成树
103
104     dfs(1,0,0);
105     long long ans2 = 1e18;
106     for(auto &[a,b,c]:T2){
107         long long w2 = lca(a,b,c); //查找生成树中(a,b)路径上边权严格小于c的最大边权
108         if(w2 == -INF) continue; //不存在严格次大边
109
110         long long now = ans1 + c - w2;
111         ans2 = std::min(ans2,now);
112     }
113     // if(ans2 == 1e18) {return 0;} //无严格次小生成树
114     std::cout << ans2;
115 }

```

生成树计数

无向图的生成树计数

[UVA10766 Organising the Organisation - 洛谷 \(luogu.com.cn\)](https://vjudge.net/problem/UVA10766)

给定n个点，两两之间有一条无向边，现在切断m条边，求剩下的图中有多少种不同的生成树

矩阵树定理, 时间复杂度 $O(N^3)$

权值设为1求得的是不同生成树的个数。权值设为原边权求得的是不同生成树的权值之和。

```
1  #include <iostream>
2  #include <vector>
3
4  const int N = 55;
5  int n,m,root;
6  int g[N][N];
7
8  struct Mat{
9      std::vector<std::vector<long long>>>a;
10
11      Mat(int n){
12          a = std::vector<std::vector<long long>>(n+1,std::vector<long long>
13      (n+1));
14      }
15
16      void add(int x,int y,int w){//无向图add(x,y,w),add(y,x,w); 度数矩阵-邻接矩阵
17          if(x == y) return;
18          a[y][y] = (a[y][y] + w);
19          a[x][y] = (a[x][y] - w);
20      }
21
22      long long det(int n){//高斯消元求n*n行列式的值(1_idx)
23          long long ans = 1;
24          for(int i = 2;i <= n;i++){//删除根节点1所在行列,下标从2开始即可(无向图可任选根)
25              for(int j = i+1;j <= n;j++){
26                  while(a[j][i]){
27                      long long t = a[i][i]/a[j][i];//貌似不求逆元都无所谓?
28                      for(int k = i;k <= n;k++) {
29                          a[i][k] -= a[j][k] * t;
30                      }
31                      std::swap(a[i],a[j]);
32                      ans = -ans;
33                  }
34              }
35              if(!a[i][i]) return 0;
36              ans *= a[i][i];
37          }
38          return std::abs(ans);
39      }
40
41      void sol(){
42          Mat M(n);
43          for(int i = 1;i <= n;i++){
44              for(int j = 1;j <= n;j++){
45                  g[i][j] = 1;
46              }
47          }
48      }
49  };
50
51 void sol(){
52     Mat M(n);
53     for(int i = 1;i <= n;i++){
54         for(int j = 1;j <= n;j++){
55             g[i][j] = 1;
56         }
57     }
58 }
```

```

48     for(int i = 1;i <= m;i++){
49         int x,y;std::cin >> x >> y;
50         g[x][y] = g[y][x] = 0;
51     }
52
53     for(int i = 1;i <= n;i++){
54         for(int j = i+1;j <= n;j++){
55             if(g[i][j]) {
56                 M.add(i,j,1);
57                 M.add(j,i,1);
58             }
59         }
60     }
61     std::cout << M.det(n) << '\n';
62 }
63
64 int main(){
65     while(std::cin >> n >> m >> root) sol();
66 }

```

有向图的生成树计数

[P4455 [CQOI2018](https://www.luogu.com.cn/problem/P4455)] [社交网络 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P4455)

给定 n 个点 m 条边的有向图，求以1为根节点的生成树个数。

如果要是给定root，不一定为1，将第root行和第n行交换，第root列和第n列交换，然后删除第n行n列再计算即可。

```

1  #include <iostream>
2  #include <vector>
3
4  const int mod = 1e4+7;
5
6  struct Mat{
7      std::vector<std::vector<long long>>>a;
8
9      Mat(int n){
10         a = std::vector<std::vector<long long>>(n+1,std::vector<long long>
(n+1));
11     }
12
13     void add(int x,int y,int w){//有向图add(x,y,w);
14         if(x == y) return;
15         a[y][y] = (a[y][y] + w) % mod;
16         a[x][y] = (a[x][y] - w) % mod;
17     }
18
19     long long det(int n){//高斯消元求n*n行列式的值(1_idx)

```



```

20     long long ans = 1;
21     for(int i = 2; i <= n; i++){//下标从2开始,相当于删除根节点1所在行列,
22         for(int j = i+1; j <= n; j++){
23             while(a[j][i]){
24                 long long t = a[i][i]/a[j][i];//貌似不求逆元都无所谓?
25                 for(int k = i; k <= n; k++) {
26                     a[i][k] = (a[i][k] - a[j][k] * t % mod + mod) % mod;
27                 }
28                 std::swap(a[i], a[j]);
29                 ans = -ans;
30             }
31         }
32         if(!a[i][i]) return 0;
33         ans = (ans * a[i][i] % mod + mod) % mod;
34     }
35     return std::abs(ans);
36 }
37 };
38
39 int main(){
40     int n,m; std::cin >> n >> m;
41     Mat M(n);
42     for(int i = 1; i <= m; i++){
43         int a,b; std::cin >> a >> b;
44         M.add(b,a,1);//本题题意是b向a连边
45     }
46     std::cout << M.det(n);
47 }

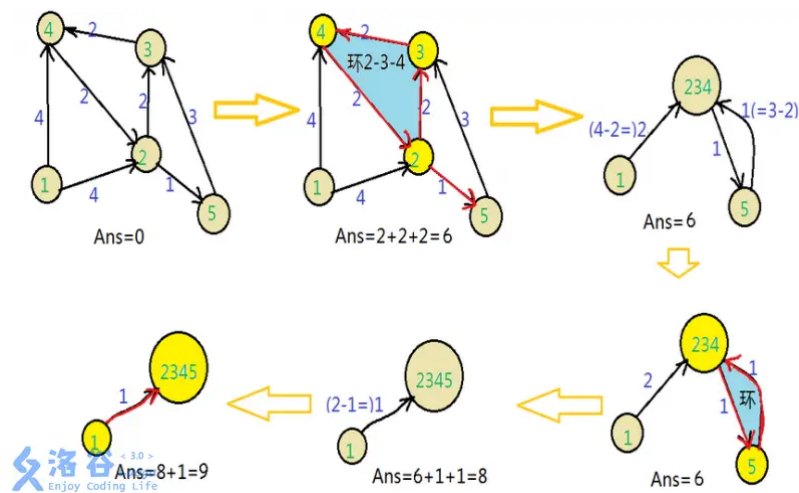
```

最小树形图

有向图上的最小生成树 (Directed Minimum Spanning Tree) 称为最小树形图。

所有从图中生成一颗 **根节点能到达其它节点** 的树, 边权和最小的一颗树

常用的算法是**朱刘算法** (也称 Edmonds 算法) , 可以在 $O(nm)$ 时间内解决最小树形图问题



算法流程，对于每一次循环：

- 初始化
- 对于每个点，选择指向它的边权最小的一条边
- 将选出的边累加到答案里，如果出现孤立点则无解
- 如果没有环，算法完成，退出循环
- 把所有不是环上的点全部设置为自己是一个独立的环(大小为1的新环)
- 进行缩环并更新其它点到环的距离
- 重新设置n和root

```

1 //https://www.luogu.com.cn/problem/P4716
2 //给定一个n个点m条边根节点为root的有向图,试求出一颗以root为根的最小树形图
3 #include <iostream>
4
5 const int INF = 0x3f3f3f3f;
6 const int N = 3003,M = 10004;
7
8 int mn[N],fa[N]; //每个节点的最小入边权值及其父节点
9 int loop[N]; //节点所属的环编号(0表示不在环中)
10 int vis[N]; //临时标记数组(用于环检测)
11
12 struct Edge{
13     int a,b,c;
14 }edge[M];
15
16 int zhuliu(int n,int m,int root){
17     int ans = 0;
18     while(1){
19         int tot = 0; //记录当前迭代检测到的环的数量
20         for(int i = 1;i <= n;i++){ //初始化数组
21             mn[i] = INF;
22             fa[i] = vis[i] = loop[i] = 0;
23         }
24
25         for(int i = 1;i <= m;i++){ //贪心找到最小入边图,并记录该图上每个节点的最小入边权
            //值及其父节点

```

```

26     auto &[a,b,c] = edge[i];
27     if(a != b && mn[b] > c){//处理自环(缩环后存在自环)
28         mn[b] = c;
29         fa[b] = a;
30     }
31 }
32 mn[root] = 0;//根节点没有入边,权值设为0
33
34 for(int i = 1;i <= n;i++){
35     if(mn[i] == INF) return -1;//若存在孤立节点,无解
36     ans += mn[i];//累加当前选择边的权值
37 }
38
39 for(int a = 1;a <= n;a++){//环检测
40     int b = a;
41     //沿着父链回溯,直到遇到根节点、已标记节点或已经在环上的节点
42     while(b != root && !vis[b] && !loop[b]) {
43         vis[b] = a;//标记当前节点
44         b = fa[b];//回溯父节点
45     }
46     if(b != root && vis[b] == a){//发现新环
47         loop[b] = ++tot;//记录环上的点在哪个环上
48         for(int k = fa[b];k != b;k = fa[k]){
49             loop[k] = tot;
50         }
51     }
52 }
53 if(!tot) return ans;//如果没有环,算法完成,返回答案
54
55 for(int i = 1;i <= n;i++){//处理剩余节点,所有非环上的独立点,每个点自己视为一个环
56     if(!loop[i]) loop[i] = ++tot;
57 }
58 for(int i = 1;i <= m;i++){//重新设置边权,缩环
59     auto &[a,b,c] = edge[i];
60     edge[i] = {loop[a],loop[b],c-mn[b]};//边权减去已选择的最小入边权值
61 }
62 n = tot;//完成缩环,重新设置n和root
63 root = loop[root];
64 }
65 }
66
67 int main(){
68     int n,m,root; std::cin >> n >> m >> root;
69     for(int i = 1;i <= m;i++){
70         std::cin >> edge[i].a >> edge[i].b >> edge[i].c;
71     }
72     std::cout << zhuliu(n,m,root);
73 }

```

输出 最小树形图权值 和 选定的根，若根不唯一，输出编号最小的。

我们建立一个虚拟的超级源点，并向其它所有点连边，边权为大于原图所有边权之和和一个数(假设为 `sum`)，若最后 `ans - sum >= sum` 说明虚拟源点建的边用了超过一次，则图不连通无解。否则最终答案就是 `ans-sum`。

主要在于如何找编号最小的根。如果有多个根，则他们最终会成一个环，这个环缩点后会与虚根相连，找出节点编号最小的一个根即可，因为我们从虚根向节点建边时是从 `m+1` 到 `m+n` 建边的，所以在下一次循环时，第一次访问到连向该环的边时，其编号(减去 `m`)即为编号最小的根。

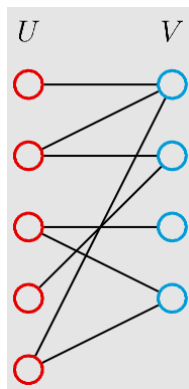
```
1  #include <iostream>
2
3  const int INF = 0x3f3f3f3f;
4  const int N = 2003,M= 20004;
5  int n,m,root,real_root;
6  int fa[N],mn[N],vis[N],loop[N];
7
8  struct Edge{
9      int a,b,c;
10 }edge[M];
11
12 int zhuliu(int n,int m,int root){
13     int ans = 0;
14     while(1){
15         int cnt = 0;
16         for(int i = 1;i <= n;i++){
17             mn[i] = INF;
18             fa[i] = vis[i] = loop[i] = 0;
19         }
20         for(int i = 1;i <= m;i++){
21             auto &[a,b,c] = edge[i];
22             if(a != b && mn[b] > c){
23                 mn[b] = c;
24                 fa[b] = a;
25                 if(a == root) real_root = i;//只需要在这里更新要找的根
26             }
27         }
28         mn[root] = 0;
29         for(int i = 1;i <= n;i++){
30             if(mn[i] == INF) return -1;
31             ans += mn[i];
32         }
33         for(int a = 1;a <= n;a++){
34             int b = a;
35             while(b != root && !vis[b] && !loop[b]){
36                 vis[b] = a;
37                 b = fa[b];
38             }
39             if(b != root && vis[b] == a){
40                 loop[b] = ++cnt;
```

```

41         for(int k = fa[b]; k != b; k = fa[k]){
42             loop[k] = cnt;
43         }
44     }
45 }
46 if(!cnt) return ans;
47 for(int i = 1; i <= n; i++){
48     if(!loop[i]) loop[i] = ++cnt;
49 }
50 for(int i = 1; i <= m; i++){
51     auto &[a,b,c] = edge[i];
52     edge[i] = {loop[a], loop[b], c-mn[b]};
53 }
54 n = cnt;
55 root = loop[root];
56 }
57 }
58
59 void sol(){
60     int sum = 0;
61     for(int i = 1; i <= m; i++){
62         int a,b,c; std::cin >> a >> b >> c;
63         edge[i] = {++a, ++b, c}; //本题节点编号从0_idx开始
64         sum += c;
65     }
66     sum++;
67     for(int i = 1; i <= n; i++){
68         edge[m+i] = {n+1, i, sum};
69     }
70     int ans = zhuliu(n+1, n+m, n+1);
71
72     if(ans == -1 || ans - sum >= sum) std::cout << "impossible\n";
73     else std::cout << ans-sum << ' ' << real_root - m - 1 << '\n';
74 }
75
76 int main(){
77     std::ios::sync_with_stdio(false); std::cin.tie(0);
78     while(std::cin >> n >> m) {
79         sol();
80         std::cout << '\n';
81     }
82 }

```

二分图



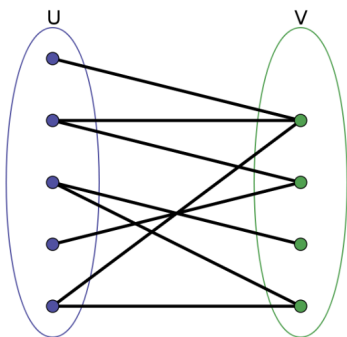
性质:

- 不存在边数为奇数的环
- 如果两个集合中的点分别染成黑色和白色，可以发现二分图中的每一条边都一定是连接一个黑色点和一个白色点。

二分图判定

染色法判二分图

判断是否是二分图（不存在奇环）



```

1 //二分图的判定 https://www.acwing.com/problem/content/862/
2 #include <iostream>
3 #include <queue>
4 #include <cstring>
5 using namespace std;
6 const int N = 200005;
7 int n,m;
8 int h[N],e[N],ne[N],idx;
9 int color[N];//0代表未染色，1/2表示染了两种颜色
10
11 void add(int a,int b){
12     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
13 }
14
15 bool uuz(int be){
16     color[be] = 1;

```

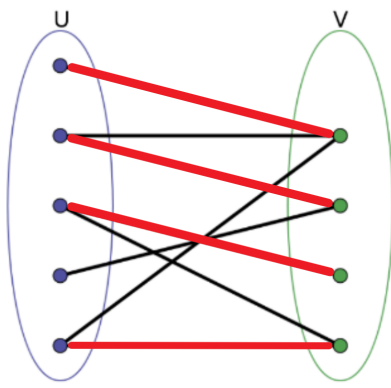
```

17     queue<int>q;
18     q.push(be);
19     while(q.size()){
20         int t = q.front();
21         q.pop();
22         for(int i = h[t];~i;i = ne[i]){
23             int k = e[i];
24             if(!color[k]){//如果当前点k未染色,则将其染为与点t相对的颜色
25                 color[k] = 3 - color[t];
26                 q.push(k);
27             }
28             if(color[k] == color[t]) return 0;//如果已经染色,且相邻节点t与k同色,则不
是二分图
29         }
30     }
31     return 1;
32 }
33
34 int main(){
35     memset(h,-1,sizeof h);
36     cin >> n >> m;
37     for(int i = 1;i <= m;i++){
38         int a,b;cin >> a >> b;
39         add(a,b);add(b,a);
40     }
41
42     for(int i = 1;i <= n;i++){
43         if(!color[i]){
44             if(uuz(i) == 0) {
45                 cout << "No";
46                 return 0;
47             }
48         }
49     }
50     cout << "Yes";
51 }

```

最大匹配

“任意两条边都没有公共端点”的边的集合被称为图的一组**匹配**。在二分图中，包含边数最多的一组匹配被称为二分图的**最大匹配**。



例如图中红边集合为一种最大匹配方案。

匈牙利算法

又称增广路算法，二分图的一组匹配 S 是最大匹配，当且仅当图中不存在 S 的增广路。

时间复杂度 $O(NM)$ ，实际效率较高。

```

1 //https://www.acwing.com/problem/content/863/
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5 const int N = 510, M = 100005;
6 int n1, n2, m;
7 int h[N], e[M], ne[M], idx;
8 int match[N], res; //match[b] = a; 即为a->b的匹配
9 bool st[N];
10
11 void add(int a, int b) {
12     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
13 }
14
15 bool find(int x) {
16     for (int i = h[x]; i != -1; i = ne[i]) {
17         int y = e[i];
18         if (!st[y]) { //如果某个点没被预定
19             st[y] = 1; //预定这个点
20             if (match[y] == 0 || find(match[y])) {
21                 //如果这个点没有匹配的对象或其匹配的对象能匹配其它点
22                 match[y] = x; //则y与x匹配
23                 return 1;
24             }
25         }
26     }
27     return 0; //如果全被预定了，则匹配失败
28 }
29
30 int main() {
31     cin >> n1 >> n2 >> m;
32     memset(h, -1, sizeof h);
33     while (m--){
34         int a, b; cin >> a >> b;

```



```

35     add(a, b); //只连一边(左到右), 只有确定集合A到B只有单向关系, 两边才可以有相同节点编号,
    否则应让b加上一个偏移量
36 }
37
38     for (int i = 1; i <= n1; i++) { //n1轮
39         memset(st, 0, sizeof st);
40         //因为每次模拟匹配的预定情况都是不一样的所以每轮模拟都要初始化
41         if (find(i)) res++;
42     }
43     cout << res;
44 }

```

结果 $match[v_i] = u_i$ 表示右部集合 v 中的点和左部集合 u 中的点完成配对。

无向图的最大匹配

只能求最二分图大匹配(即没有奇环), 若为一般图, 则应换用带花树算法

```

1 //https://vjudge.net/problem/HDU-2444
2 //染色且只匹配当前颜色为1的点
3 //或者最终答案直接除以2(貌似这种具体方案不好求?)
4 #include <iostream>
5 #include <vector>
6 #include <cstring>
7
8 const int N = 205, M = N*N;
9 int n, m;
10
11 int h[N], e[M], ne[M], idx;
12 void add(int a, int b) {
13     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
14 }
15
16 bool vis[N];
17 int match[N], color[N];
18 bool dfs(int x, int c) {
19     color[x] = c;
20     for(int i = h[x]; ~i; i = ne[i]) {
21         int y = e[i];
22         if(!color[y]) {
23             if(dfs(y, 3-c) == 0) return 0;
24         }
25         else if(color[y] == c) return 0;
26     }
27     return 1;
28 }
29
30 bool find(int x) {
31     if(color[x] != 1) return 0; //只匹配颜色为1的点
32     for(int i = h[x]; ~i; i = ne[i]) {

```

```

33     int y = e[i];
34     if(!vis[y]){
35         vis[y] = 1;
36         if(!match[y] || find(match[y])){
37             match[y] = x;
38             return 1;
39         }
40     }
41 }
42 return 0;
43 }
44
45 void sol(){
46     for(int i = 1; i <= m; i++){
47         int a,b; std::cin >> a >> b;
48         add(a,b); add(b,a);
49     }
50     for(int i = 1; i <= n; i++){
51         if(!color[i]){
52             if(dfs(i,1) == 0) {std::cout << "No\n"; return;} //前提必须是二分图
53         }
54     }
55
56     int ans = 0;
57     for(int i = 1; i <= n; i++){
58         std::memset(vis,0,sizeof vis);
59         ans += find(i);
60     }
61     std::cout << ans << '\n';
62 }
63
64 int main(){
65     while(std::cin >> n >> m) {
66         idx = 0;
67         for(int i = 1; i <= n; i++){
68             h[i] = -1;
69             color[i] = match[i] = 0;
70         }
71         sol();
72     }
73 }

```

多重匹配

```

1  const int N = 303;
2  int n1,n2,m;
3  int g[N][N];
4
5  std::set<int> se[N];
6  bool vis[N];

```

```

7  bool find(int x){
8      for(int y = 1;y <= n1;y++){
9          if(g[x][y] && !vis[y]){
10             vis[y] = 1;
11             if(se[y].size() < m){//如果当前组容量有剩余,则直接加入
12                 se[y].insert(x);
13                 return 1;
14             }
15             else if(se[y].size() == m){//如果已经满了
16                 for(auto &x2:se[y]){//尝试从当前组移出一个到其它组
17                     if(find(x2,mid)){
18                         se[y].erase(x2);
19                         se[y].insert(x);
20                         return 1;
21                     }
22                 }
23             }
24         }
25     }
26     return 0;
27 }
28
29 bool check(){
30     for(int i = 1;i <= n1;i++) se[i].clear();
31     for(int i = n1+1;i <= n1+n2;i++){
32         std::memset(vis,0,sizeof vis);
33         if(find(i) == 0) return 0;
34     }
35     return 1;
36 }

```

HK 算法

Hopcroft-Karp算法, 时间复杂度为 $O(M\sqrt{N})$, 实际表现较好

```

1  //【模板】二分图最大匹配 https://www.luogu.com.cn/problem/P3386
2  #include <iostream>
3  #include <queue>
4  #include <cstring>
5
6  struct HK{
7      int n1,n2,dis;
8      std::vector<int>cx,cy,dx,dy,vis;
9      std::vector<std::vector<int>>>g;
10     const int INF = 0x3f3f3f3f;
11
12     HK(int _n1,int _n2){
13         n1 = _n1,n2 = _n2;
14         g.assign(n1+1,{});
15     }

```

```

16
17 void add(int a,int b){
18     g[a].emplace_back(b);
19 }
20
21 bool bfs(){
22     std::queue<int>q;
23     dx = std::vector<int>(n1+1,-1);
24     dy = std::vector<int>(n2+1,-1);
25
26     dis = INF;
27     for(int i = 1;i <= n1;i++){
28         if(cx[i] == -1){
29             q.push(i);
30             dx[i] = 0;
31         }
32     }
33     while(q.size()){
34         int x = q.front();
35         q.pop();
36         if(dx[x] > dis) break;
37         for(auto &y:g[x]){
38             if(dy[y] == -1){
39                 dy[y] = dx[x] + 1;
40                 if(cy[y] == -1) dis = dy[y];
41             }
42             else{
43                 dx[cy[y]] = dy[y]+1;
44                 q.push(cy[y]);
45             }
46         }
47     }
48     return dis != INF;
49 }
50
51 bool dfs(int x){
52     for(auto &y:g[x]){
53         if(!vis[y] && dy[y] == dx[x] + 1){
54             vis[y] = 1;
55             if(cy[y] != -1 && dy[y] == dis) continue;
56             if(cy[y] == -1 || dfs(cy[y])){
57                 cy[y] = x;
58                 cx[x] = y;
59                 return 1;
60             }
61         }
62     }
63     return 0;
64 }
65
66 int sol(){
67     int ans = 0;

```

```

68     cx = std::vector<int>(n1+1,-1);
69     cy = std::vector<int>(n2+1,-1);
70     while(bfs()) {
71         vis = std::vector<int>(n2+1,0);
72         for(int i = 1;i <= n1;i++){
73             if(cx[i] == -1 && dfs(i)) ans++;
74         }
75     }
76     return ans;
77 }
78 };
79
80
81 int main(){
82     int n1,n2,m;
83     std::cin >> n1 >> n2 >> m;
84     HK hk(n1,n2);
85     for(int i = 1;i <= m;i++){
86         int a,b;std::cin >> a >> b;//n1[] -> n2[]
87         hk.add(a,b);
88     }
89     std::cout << hk.sol() << '\n';
90     // for(int i = 1;i <= n1;i++){//具体方案,cx[]为左边的点匹配到的方案,cy[]为右边的点匹配
    到的方案
91     //     if(hk.cx[i] != -1) { //-1则为匹配失败
92     //         std::cout << i << ' ' << hk.cx[i] << '\n';
93     //     }
94     // }
95 }

```

转为网络流模型

边权设为1，源点向左边集合所有点连边，右边集合所有点向汇点连边。求最大流即可。

时间复杂度 $O(M\sqrt{N})$ ，常数较大

```

1 //基于Dinic实现 【模板】二分图最大匹配 https://www.luogu.com.cn/problem/P3386
2 #include <iostream>
3 #include <queue>
4 #include <cstring>
5
6 const int INF = 0x3f3f3f3f;
7 const int N = 1003,M = 400005;
8 int h[N],e[M],ne[M],idx,w[M];
9 int n1,n2,m;
10 int s,t;
11 int maxflow;
12 int d[N],now[N];
13
14 void add(int a,int b,int c){

```

```

15     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
16 }
17
18 bool bfs(){
19     memset(d,0,sizeof d);
20     std::queue<int>q;
21     q.push(s);
22     d[s] = 1;
23     now[s] = h[s];
24     while(q.size()){
25         int x = q.front();
26         q.pop();
27         for(int i = h[x];~i;i = ne[i]){
28             int y = e[i];
29             if(!w[i] || d[y]) continue;
30             d[y] = d[x]+1;
31             now[y] = h[y];
32             q.push(y);
33             if(y == t) return 1;
34         }
35     }
36     return 0;
37 }
38
39 int dfs(int x,int flow){
40     if(x == t) return flow;
41     int res = 0;
42     for(int i = now[x];~i && flow;i = ne[i]){
43         now[x] = i;
44         int y = e[i];
45         if(!w[i] || d[y] != d[x]+1) continue;
46         int k = dfs(y,std::min(flow,w[i]));
47         if(!k) d[y] = 0;
48         w[i] -= k;
49         w[i^1] += k;
50         flow -= k;
51         res += k;
52     }
53     return res;
54 }
55
56 int main(){
57     std::memset(h,-1,sizeof h);
58     std::cin >> n1 >> n2 >> m;
59     s = 0,t = n1+n2+1;
60     for(int i = 1;i <= m;i++){
61         int a,b;std::cin >> a >> b;
62         add(a,b+n1,1);
63         add(b+n1,a,0);
64     }
65     for(int i = 1;i <= n1;i++) add(s,i,1),add(i,s,0);

```

```

66     for(int i = n1+1;i <= n1+n2;i++) add(i,t,1),add(t,i,0);//如果B集合每个点可以容
    下最多m个A集合中的点,则这里边权改为m即可
67
68     while(bfs()) maxflow += dfs(s,INF);
69
70     std::cout << maxflow << '\n';
71 // for(int i = 1;i < idx;i += 2){//具体方案,遍历所有反边
72 //     if(w[i]) {//如果反边还剩流量且端点不为源点或汇点
73 //         if(e[i] == s || e[i^1] == t) continue;
74 //         std::cout << e[i] << ' ' << e[i^1] - n1 << '\n';
75 //     }
76 // }
77 }

```

常见问题模型

二分图匹配的模型有两个要素：

1. 节点能分成独立的两个集合，每个集合内部有0条边。
2. 每个节点只能与1条匹配边相连。

[372. 棋盘覆盖 - AcWing题库](#)

给定一个 N 行 N 列的棋盘，已知某些格子禁止放置。求最多能往棋盘上放多少块的长度为 2、宽度为 1 的骨牌，并且任意两张骨牌都不重叠。

将骨牌看作无向边，在相邻的两个格子连边。如果把棋盘黑白染色(行号加列号为偶数染成白色，否则染成黑色)。那么相邻的两个格子颜色不同，有连边。同色的两个格子不相邻，没有边相连。该图是一张二分图，二分图的最大匹配即为最多能放骨牌的个数。

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  const int N = 105;
5  int n,t;
6  int a[N][N];
7  pair<int,int> match[N][N];
8  bool st[N][N];
9  int dx[] = {-1,1,0,0},dy[] = {0,0,-1,1};
10
11 bool find(pair<int,int>p){
12     auto [x,y] = p;
13     for(int i = 0;i < 4;i++){
14         int nx = x + dx[i],ny = y + dy[i];
15         if(nx < 1 || nx > n || ny < 1 || ny > n || a[nx][ny]) continue;
16         if(!st[nx][ny]){
17             st[nx][ny] = 1;

```

```

18         if(!match[nx][ny].first || find(match[nx][ny])){
19             match[nx][ny] = {x,y};
20             return 1;
21         }
22     }
23 }
24 return 0;
25 }
26
27 int main(){
28     cin >> n >> t;
29     while(t--){
30         int x,y;cin >> x >> y;
31         a[x][y] = 1;
32     }
33     int ans = 0;
34     for(int i = 1;i <= n;i++){
35         for(int j = 1;j <= n;j++){
36             if((i+j)&1 || a[i][j]) continue;
37             memset(st,0,sizeof st);
38             ans += find({i,j});
39         }
40     }
41     cout << ans;
42 }

```

[P10937 車的放置 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P10937)

给定一个 N 行 M 列的棋盘，已知某些格子禁止放置。问棋盘上最多能放多少个不能互相攻击的車（同行或同列只能有一个車）。

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  const int N = 205,M = N*N;
5  int n,m,t;
6  int a[N][N];
7  int h[N],e[M],ne[M],idx;
8  bool st[N];
9  int match[N];
10
11 void add(int a,int b){
12     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
13 }
14
15 bool find(int x){
16     for(int i = h[x];~i;i = ne[i]){
17         int y = e[i];
18         if(!st[y]){
19             st[y] = 1;

```



```

20         if(!match[y] || find(match[y])){
21             match[y] = x;
22             return 1;
23         }
24     }
25 }
26 return 0;
27 }
28
29 int main(){
30     memset(h,-1,sizeof h);
31     cin >> n >> m >> t;
32     while(t--){
33         int x,y;cin >> x >> y;
34         a[x][y] = 1;
35     }
36
37     for(int i = 1;i <= n;i++){
38         for(int j = 1;j <= m;j++){
39             if(a[i][j] == 0) add(i,j);
40         }
41     }
42     int ans = 0;
43     for(int i = 1;i <= n;i++){
44         memset(st,0,sizeof st);
45         ans += find(i);
46     }
47     cout << ans;
48 }

```

最大权匹配

KM算法

时间复杂度 $O(N^3)$ ，常数非常小，在稠密图上表现出色

```

1 //【模板】二分图最大权完美匹配 https://www.luogu.com.cn/problem/P6577
2 #include<bits/stdc++.h>
3 using namespace std;
4
5 const long long INF = 1e18;
6 const int N = 505;
7 int n1,n2,m;
8
9 struct KM {
10     int n;

```

```

11     int pl[N], pr[N], fa[N];
12     long long g[N][N], wl[N], wr[N], sl[N];
13     bitset<N> vis;
14     long long tot;
15
16     void init(int _n) {
17         n = _n;
18         for (int i = 1; i <= n; i++) {
19             fill(g[i] + 1, g[i] + n + 1, -INF); // 本题含有负权边, 初始化为-INF, 否则初始
化为0
20             wl[i] = wr[i] = sl[i] = 0;
21             pl[i] = pr[i] = fa[i] = 0;
22         }
23     }
24
25     void add(int x, int y, long long z) {
26         g[x][y] = max(g[x][y], z);
27     }
28
29     void get(int x) {
30         for (int i = 1; i <= n; i++) {
31             sl[i] = INF;
32             fa[i] = 0;
33         }
34         vis.reset();
35         pr[0] = x;
36         int cr = 0;
37         while (pr[cr]) {
38             long long mn = INF;
39             vis[cr] = 1;
40             int cl = pr[cr];
41             for (int i = 1; i <= n; i++) {
42                 if (!vis[i]) {
43                     const long long t1 = wl[cl] + wr[i] - g[cl][i];
44                     if (sl[i] > t1) {
45                         fa[i] = cr;
46                         sl[i] = t1;
47                     }
48                     mn = min(mn, sl[i]);
49                 }
50             }
51             for (int i = 0; i <= n; i++) {
52                 if (vis[i]) {
53                     wr[i] += mn;
54                     wl[pr[i]] -= mn;
55                 }
56                 else sl[i] -= mn;
57             }
58             cr = 0;
59             for (int i = 1; i <= n; i++) {
60                 if (!vis[i] && sl[i] == 0) {
61                     cr = i;

```

```

62         break;
63     }
64 }
65 }
66 while (cr) {
67     pr[cr] = pr[fa[cr]];
68     cr = fa[cr];
69 }
70 }
71
72 void sol() {
73     tot = 0;
74     for (int i = 1; i <= n; i++) get(i);
75     for (int i = 1; i <= n; i++) pl[pr[i]] = i;
76     for (int i = 1; i <= n; i++) tot += g[i][pl[i]];
77 }
78 } km;
79
80 int main(){
81     ios::sync_with_stdio(0); cin.tie(0);
82     std::cin >> n1 >> m; n2 = n1;
83     km.init(max(n1,n2));
84     while(m--){
85         int a,b,c; cin >> a >> b >> c;
86         km.add(a,b,c);
87     }
88
89     km.sol();
90
91     cout << km.tot << '\n';
92     // for(int i = 1;i <= n1;i++){
93     //     if(km.g[i][km.pl[i]] > -INF) {
94     //         std::cout << i << "->" << km.pl[i] << ' ';
95     //     }
96     // }
97 }

```

```

1 //非完美匹配下的最大权 https://uoj.ac/problem/80
2 #include<bits/stdc++.h>
3 using namespace std;
4
5 const long long INF = 1e18;
6 const int N = 505;
7 int n1,n2,m;
8
9
10 struct KM {
11     int n;
12     int pl[N], pr[N], fa[N];
13     long long g[N][N], wl[N], wr[N], sl[N];
14     bitset<N> vis;

```

```

15     long long tot;
16
17     void init(int _n) {
18         n = _n;
19         for (int i = 1; i <= n; i++) {
20             fill(g[i] + 1, g[i] + n + 1, 0); //本题均为正权边,边权初始化为0
21             wl[i] = wr[i] = sl[i] = 0;
22             pl[i] = pr[i] = fa[i] = 0;
23         }
24     }
25
26     void add(int x, int y, long long z) {
27         g[x][y] = max(g[x][y], z);
28     }
29
30     void get(int x) {
31         for (int i = 1; i <= n; i++) {
32             sl[i] = INF;
33             fa[i] = 0;
34         }
35         vis.reset();
36         pr[0] = x;
37         int cr = 0;
38         while (pr[cr]) {
39             long long mn = INF;
40             vis[cr] = 1;
41             int cl = pr[cr];
42             for (int i = 1; i <= n; i++) {
43                 if (!vis[i]) {
44                     const long long t1 = wl[cl] + wr[i] - g[cl][i];
45                     if (sl[i] > t1) {
46                         fa[i] = cr;
47                         sl[i] = t1;
48                     }
49                     mn = min(mn, sl[i]);
50                 }
51             }
52             for (int i = 0; i <= n; i++) {
53                 if (vis[i]) {
54                     wr[i] += mn;
55                     wl[pr[i]] -= mn;
56                 }
57                 else sl[i] -= mn;
58             }
59             cr = 0;
60             for (int i = 1; i <= n; i++) {
61                 if (!vis[i] && sl[i] == 0) {
62                     cr = i;
63                     break;
64                 }
65             }
66         }

```

```

67         while (cr) {
68             pr[cr] = pr[fa[cr]];
69             cr = fa[cr];
70         }
71     }
72
73     void sol() {
74         tot = 0;
75         for (int i = 1; i <= n; i++) get(i);
76         for (int i = 1; i <= n; i++) pl[pr[i]] = i;
77         for (int i = 1; i <= n; i++) tot += g[i][pl[i]];
78     }
79 } km;
80
81 int main(){
82     ios::sync_with_stdio(0); cin.tie(0);
83     cin >> n1 >> n2 >> m;
84     km.init(max(n1,n2)); //相当于在节点较少的集合添加了虚拟点
85     while(m--){
86         int a,b,c; cin >> a >> b >> c;
87         km.add(a,b,c);
88     }
89
90     km.sol();
91
92     // cout << km.tot << '\n';
93     // for(int i = 1;i <= n1;i++){
94     //     if(km.g[i][km.pl[i]]) {
95     //         std::cout << i << "->" << km.pl[i] << '\n';
96     //     }
97     // }
98 }

```

```

1 //https://vjudge.net/problem/HDU-2813
2 //求最小权匹配
3 struct KM {
4     int n;
5     int pl[N], pr[N], fa[N];
6     long long g[N][N], wl[N], wr[N], sl[N];
7     bitset<N> vis;
8     long long tot;
9
10    void init(int _n) {
11        n = _n;
12        for (int i = 1; i <= n; i++) {
13            fill(g[i] + 1, g[i] + n + 1, -INF); //初始化为-INF
14            wl[i] = wr[i] = sl[i] = 0;
15            pl[i] = pr[i] = fa[i] = 0;
16        }

```

```

17     }
18
19     void add(int x, int y, long long z) {
20         g[x][y] = max(g[x][y], -z); //存负值然后求最大
21     }
22
23     void get(int x) {
24         for (int i = 1; i <= n; i++) {
25             sl[i] = INF;
26             fa[i] = 0;
27         }
28         vis.reset();
29         pr[0] = x;
30         int cr = 0;
31         while (pr[cr]) {
32             long long mn = INF;
33             vis[cr] = 1;
34             int cl = pr[cr];
35             for (int i = 1; i <= n; i++) {
36                 if (!vis[i]) {
37                     const long long t1 = wl[cl] + wr[i] - g[cl][i];
38                     if (sl[i] > t1) {
39                         fa[i] = cr;
40                         sl[i] = t1;
41                     }
42                     mn = min(mn, sl[i]);
43                 }
44             }
45             for (int i = 0; i <= n; i++) {
46                 if (vis[i]) {
47                     wr[i] += mn;
48                     wl[pr[i]] -= mn;
49                 }
50                 else sl[i] -= mn;
51             }
52             cr = 0;
53             for (int i = 1; i <= n; i++) {
54                 if (!vis[i] && sl[i] == 0) {
55                     cr = i;
56                     break;
57                 }
58             }
59         }
60         while (cr) {
61             pr[cr] = pr[fa[cr]];
62             cr = fa[cr];
63         }
64     }
65
66     void sol() {
67         tot = 0;
68         for (int i = 1; i <= n; i++) get(i);

```

```

69     for (int i = 1; i <= n; i++) p1[pr[i]] = i;
70     for (int i = 1; i <= n; i++) {
71         if (g[i][p1[i]] != -INF) tot += g[i][p1[i]]; //只计算实际存在的边
72     }
73     tot = -tot; //取反得到最小权
74 }
75 } km;

```

转为费用流模型

- 新建源点s1和汇点s2。
- 源点 向每个 左部节点 连一条流量为1，费用为0的边。
- 每个 右部节点 向 汇点 连一条流量为1，费用为0的边。
- 对于二分图中每条左部到右部的边，连一条流量为1，费用为边权的边。
- 另外如果考虑非完美匹配，对于每个 左部节点 还需要向 汇点 连一条流量为1，费用为0的边。最大费用的前提是最大流。
- 求这个网络的**最大费用最大流**即可，此时网络的最大流量一定为左部节点的数量，而最大流量下的最大费用即对应一个最大权匹配方案。

时间复杂度较高 $O(N^2M)$ ，比KM算法慢了大概几十倍，如果数据范围较大建议换KM算法。

```

1 //模板 二分图最大权完美匹配 https://www.luogu.com.cn/problem/P6577
2 //基于Dinic实现,实际并不能通过本题,只作实现参考,一般只能通过节点总数<=500的图
3 #include <iostream>
4 #include <queue>
5 #include <cstring>
6
7 const long long INF = 1e18;
8 const int N = 1005,M = 1000006;//N开n1+n2+5,M开n1*n2*3
9 int n1,n2,m;
10
11 int h[N],e[M],ne[M],idx;
12 long long c[M],w[M];
13 void add(int a,int b,int x,int y){
14     c[idx] = y,w[idx] = x,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
15 }
16
17 int s1,s2;
18 long long maxflow,maxcost;
19 long long dist[N];
20 int now[N];
21 bool vis[N];
22 bool spfa(){
23     std::memset(vis,0,sizeof vis);
24     for(int i = 0;i <= n1+n2+1;i++) dist[i] = -INF;
25     std::queue<int>q;

```

```

26     q.push(s1);
27     dist[s1] = 0;
28     now[s1] = h[s1];
29     while(q.size()){
30         int x = q.front();
31         q.pop();
32         vis[x] = 0;
33         for(int i = h[x];~i;i = ne[i]){
34             int y = e[i];
35             if(w[i] && dist[y] < dist[x] + c[i]) {
36                 dist[y] = dist[x] + c[i];
37                 now[y] = h[y];
38                 if(!vis[y]) {
39                     vis[y] = 1;
40                     q.push(y);
41                 }
42             }
43         }
44     }
45     return dist[s2] > -INF;
46 }
47
48 long long dinic(int x,long long flow){
49     if(x == s2) return flow;
50     long long res = 0;
51     vis[x] = 1;
52     for(int i = now[x];~i && flow;i = ne[i]){
53         int y = e[i];
54         now[x] = i;
55         if(w[i] && dist[y] == dist[x] + c[i] && !vis[y]){
56             long long k = dinic(y,std::min(flow,w[i]));
57             if(!k) dist[y] = 0;
58             w[i] -= k;
59             w[i^1] += k;
60             res += k;
61             flow -= k;
62             maxcost += k * c[i];
63         }
64     }
65     vis[x] = 0;
66     return res;
67 }
68
69 int main(){
70     std::ios::sync_with_stdio(false);std::cin.tie(0);
71     std::memset(h,-1,sizeof h);
72     std::cin >> n1 >> m; n2 = n1;
73     s1 = 0,s2 = n1+n2+1;
74     for(int i = 1;i <= m;i++){
75         int x,y,z;std::cin >> x >> y >> z;
76         add(x,y+n1,1,z); add(y+n1,x,0,-z);
77     }

```



```

78
79     for(int i = 1; i <= n1; i++){
80         add(s1, i, 1, 0); add(i, s1, 0, 0);
81         // add(i, s2, 1, 0); add(s2, i, 0, 0); //加上即为非完美匹配下的最大权,使保证满流,因为费
            用流的前提是最大流
82     }
83
84     for(int i = 1; i <= n2; i++){
85         add(i+n1, s2, 1, 0); add(s2, i+n1, 0, 0);
86     }
87
88     while(spfa()) maxflow += dinic(s1, INF);
89
90     std::cout << maxcost << '\n';
91
92     // for(int i = 1; i < idx; i += 2){
93     //     if(w[i]){
94     //         if(e[i] == s1 || e[i^1] == s2) continue;
95     //         std::cout << e[i] << "->" << e[i^1] - n1 << '\n';
96     //     }
97     // }
98 }

```

最小点覆盖

求出一个最小的点集 S ，使得任意一条边都至少有一个端点属于 S

König定理：最小点覆盖=最大匹配

二分图最小覆盖的模型特点是：每条边有2个端点，二者至少选择一个。

[P6062 [USACO05JAN](#)] [Muddy Fields G](#) - 洛谷 ([luogu.com.cn](#))

给定一个 $N \times M$ 的网格，其中一些地面是泥泞的，你需要用一些宽度为1，长度任意的木板将所有泥地盖住，同时不能盖住干净的地面，木板可以重叠。 $N, M \leq 50$ 。

每块泥地要么被第 i 行的一个横着的木板盖住，要么被第 j 列的一个竖着的木板盖住，二者至少选择一个，满足二分图最小覆盖特点。把行木板作为左边，列木板作为右边，对于每块泥地，在它所属的行木板与列木板之间连边，求出二分图的最小覆盖，即为最少得木板覆盖所有泥地的答案。

```
1 #include <iostream>
```

```

2  #include <vector>
3  #include <cstring>
4  using namespace std;
5  const int N = 55;
6  int n,m;
7  string s[N];
8  int r[N][N],c[N][N],cnt;
9  bool st[N*N];
10 int match[N*N];
11 vector<int>e[N*N];
12
13 void add(int a,int b){
14     e[a].emplace_back(b);
15 }
16
17 bool find(int x){
18     for(int y:e[x]){
19         if(!st[y]){
20             st[y] = 1;
21             if(!match[y] || find(match[y])){
22                 match[y] = x;
23                 return 1;
24             }
25         }
26     }
27     return 0;
28 }
29
30 int main(){
31     cin >> n >> m;
32     for(int i = 1;i <= n;i++){
33         cin >> s[i];s[i] = ' ' + s[i];
34     }
35     for(int i = 1;i <= n;i++){
36         for(int j = 1;j <= m;j++){
37             if(!r[i][j] && s[i][j] == '*'){
38                 if(r[i][j-1]) r[i][j] = r[i][j-1];
39                 else r[i][j] = ++cnt;
40             }
41         }
42     }
43     int cnt1 = cnt;
44     for(int j = 1;j <= m;j++){
45         for(int i = 1;i <= n;i++){
46             if(!c[i][j] && s[i][j] == '*'){
47                 if(c[i-1][j]) c[i][j] = c[i-1][j];
48                 else c[i][j] = ++cnt;
49                 add(r[i][j],c[i][j]);
50             }
51         }
52     }
53     int ans = 0;

```

```

54     for(int i = 1;i <= cnt1;i++){
55         memset(st,0,sizeof st);
56         ans += find(i);
57     }
58     cout << ans;
59 }

```

最大独立集

选最多的点，满足两两之间没有边相连。

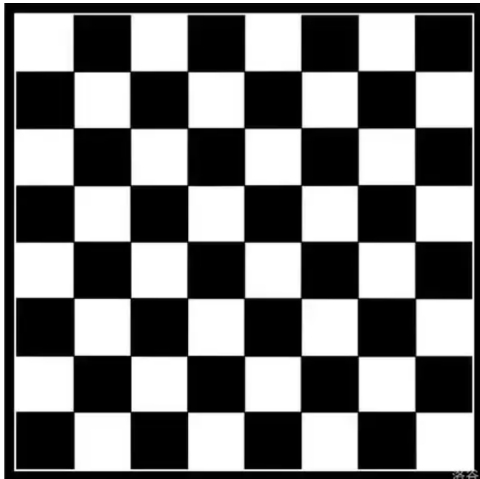
定理：设G是有N个点的二分图，则G的最大独立集等于**顶点数N-最大匹配数**

求解一个图中的最大独立集等价于求解其反图的 **最大团** (即两两之间都有连边)。

[P10939 骑士放置 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P10939)

给点N*M的棋盘，其中有一些格子禁止放旗子，问棋盘上最多能放多少个互不攻击的骑士(与象棋的马类似，攻击‘日’字)。N,M <= 100。

将棋盘黑白染色后，可以发现，一匹马可以跳到的格子的颜色一定与当前所在格子颜色相反。于是我们可以将每个位置与能跳到的位置连边，这样就构成了一个二分图（所有白色格子是一部分，所有黑色格子是一部分）。如果想让所有的马都不能互相吃，那么这个二分图里一条边的两个点最多只能选一个点放马，所以这题就是让我们在一个二分图里相邻的两个点只能选一个，问最多能选多少个点。



```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  const int N = 105;
5  int n,m,t;
6  int g[N][N];
7  pair<int,int> match[N][N];
8  bool st[N][N];
9  int dx[] = {-2,-1,1,2,2,1,-1,-2},dy[] = {1,2,2,1,-1,-2,-2,-1};

```

```

10
11 bool find(pair<int,int>p){
12     auto &[x,y] = p;
13     for(int i = 0;i < 8;i++){
14         int nx = x + dx[i],ny = y + dy[i];
15         if(nx < 1 || nx > n || ny < 1 || ny > m || g[nx][ny]) continue;
16         if(!st[nx][ny]){
17             st[nx][ny] = 1;
18             if(!match[nx][ny].first || find(match[nx][ny])){
19                 match[nx][ny] = {x,y};
20                 return 1;
21             }
22         }
23     }
24     return 0;
25 }
26
27 int main(){
28     cin >> n >> m >> t;
29     for(int i = 1;i <= t;i++){
30         int x,y;cin >> x >> y;
31         g[x][y] = 1;
32     }
33
34     int ans = 0;
35     for(int i = 1;i <= n;i++){
36         for(int j = 1;j <= m;j++){
37             if(g[i][j] || (i+j)%2) continue;
38             memset(st,0,sizeof st);
39             ans += find({i,j});
40         }
41     }
42     cout << n*m - t - ans;
43 }

```

最小路径覆盖

在一个有无环向图中，找出最少得路径，使得这些路径经过了所有点。特别的，每个点自己也可以称为是路径覆盖，只不过路径的长度是0。

最小不相交路径覆盖

每一条路径经过的顶点各不相同。

原图G中每个点如果a能直接到达b，就加边a->b这样就得到了一个G的拆点二分图G2。

定理：DAG图G的最小不相交路径覆盖包含的路径条数 = n - 拆点二分图G2的最大匹配数

一开始每个点都是独立的为一条路径，总共有n条不相交路径。我们每次在二分图里找一条匹配边就相当于把两条路径合成了一条路径，也就相当于路径数减少了1。所以找到了几条匹配边，路径数就减少了多少。所以最小路径覆盖=原图的结点数-新图的最大匹配数

```
1 //街道清理 https://www.luogu.com.cn/problem/UVA1184
2 #include <iostream>
3 #include <cstring>
4 #include <vector>
5 using namespace std;
6 const int N = 205;
7 int n,m;
8 vector<int>e[N];
9 int match[N];
10 bool st[N];
11
12 bool find(int x){
13     for(int y:e[x]){
14         if(!st[y]){
15             st[y] = 1;
16             if(!match[y] || find(match[y])){
17                 match[y] = x;
18                 return 1;
19             }
20         }
21     }
22     return 0;
23 }
24
25 void sol(){
26     memset(match,0,sizeof match);
27     cin >> n >> m;
28     for(int i = 1;i <= n;i++) e[i].clear();
29     for(int i = 1;i <= m;i++){
30         int a,b;cin >> a >> b;
31         e[a].emplace_back(b);
32     }
33     int ans = 0;
34     for(int i = 1;i <= n;i++){
35         memset(st,0,sizeof st);
36         ans += find(i);
37     }
38     cout << n - ans << '\n';
39 }
40
41 int main(){
42     int T;cin >> T;
43     while(T-->0) sol();
44 }
```

网络流实现+输出具体路径: [P2764 最小路径覆盖问题 - 洛谷](#)

连边：每个点拆为 i 和 $i+n$ ，源点 $s1$ 向所有点 i 连容量为1的边，所有点 $i+n$ 向汇点 $s2$ 连容量为1的边，原图连边 $(x, y+n, 1)$

路径条数 = 节点个数 - 最大流。

在残量网络中，如果左侧节点 x 到右侧节点 $y+n$ 的边容量为0，说明这条边被匹配了，记录 $next[x] = y, last[y] = x$ ，重建匹配关系。遍历所有点，找到所有没有前驱节点的点 ($prev[i] == 0$)，以此节点为起点，沿着 $next$ 一直走，输出路径。

最小可相交路径覆盖

每一条路径经过的顶点可以相同。

思路：先用floyd求出原图的传递闭包，即如果 a 能直接/间接到达 b ，那么就加边 $a \rightarrow b$ 。得到有向无环图 $G2$ ，再在 $G2$ 上求最小不相交路径覆盖即可。

```
1 //宝藏探索 https://vjudge.net/problem/OpenJ\_Bailian-2594
2 #include <iostream>
3 #include <cstring>
4
5 const int N = 505;
6 int n,m;
7 int g[N][N];
8
9 void floyd(){
10     for(int k = 1;k <= n;k++){
11         for(int i = 1;i <= n;i++){
12             for(int j = 1;j <= n;j++){
13                 g[i][j] |= g[i][k]&g[k][j];
14             }
15         }
16     }
17 }
18
19 bool vis[N];
20 int match[N];
21 bool find(int x){
22     for(int y = 1;y <= n;y++){
23         if(g[x][y] && !vis[y]){
24             vis[y] = 1;
25             if(!match[y] || find(match[y])) {
26                 match[y] = x;
27                 return 1;
28             }
29         }
30     }
31     return 0;
32 }
33
34 void sol(){
```

```

35     std::memset(g,0,sizeof g);
36     for(int i = 1;i <= m;i++){
37         int a,b;std::cin >> a >> b;
38         g[a][b] = 1;
39     }
40
41     floyd();
42
43     int ans = 0;
44     std::memset(match,0,sizeof match);
45     for(int i = 1;i <= n;i++){
46         std::memset(vis,0,sizeof vis);
47         ans += find(i);
48     }
49     std::cout << n - ans << '\n';
50 }
51
52 int main(){
53     while(std::cin >> n >> m,n || m) sol();
54 }

```

一般图最大匹配

一般图匹配和二分图匹配不同的是，图可能存在奇环。一般图的最大匹配可以用**带花树算法** (blossom algorithm) 解决。

时间复杂度在 $O(NM)$ 到 $O(N^3)$ 左右

```

1 //【模板】一般图最大匹配 https://www.luogu.com.cn/problem/P6113
2 //模版来自sse
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 namespace blossom_tree {
7     const int N = 1003;
8     vector<int> e[N];
9     int lk[N],rt[N],f[N],dfn[N],typ[N],q[N];
10    int id,h,t,n;
11    int lca(int u,int v) {
12        ++id;
13        while(1) {
14            if(u){
15                if (dfn[u]==id) return u;
16                dfn[u]=id;u=rt[f[lk[u]]];
17            }
18            swap(u,v);
19        }
20    }
21    void blm(int u,int v,int a) {
22        while (rt[u]!=a) {

```

```

23         f[u]=v;
24         v=lk[u];
25         if (typ[v]==1) typ[q[++t]=v]=0;
26         rt[u]=rt[v]=a;
27         u=f[v];
28     }
29 }
30 void aug(int u) {
31     while (u) {
32         int v=lk[f[u]];
33         lk[lk[u]=f[u]]=u;
34         u=v;
35     }
36 }
37 void bfs(int root) {
38     memset(typ+1,-1,n*sizeof typ[0]);
39     iota(rt+1,rt+n+1,1);
40     typ[q[h=t=1]=root]=0;
41     while (h<=t) {
42         int u=q[h++];
43         for (int v:e[u]) {
44             if (typ[v]==-1) {
45                 typ[v]=1;f[v]=u;
46                 if (!lk[v]) return aug(v);
47                 typ[q[++t]=lk[v]]=0;
48             } else if (!typ[v]&&rt[u]!=rt[v]) {
49                 int a=lca(rt[u],rt[v]);
50                 blm(v,u,a);blm(u,v,a);
51             }
52         }
53     }
54 }
55 int max_general_match(int N,vector<pair<int,int>> edges){
56     n=N;id=0;
57     memset(f+1,0,n*sizeof f[0]);
58     memset(dfn+1,0,n*sizeof dfn[0]);
59     memset(lk+1,0,n*sizeof lk[0]);
60     for (int i=1;i<=n;i++) e[i].clear();
61     mt19937_64 rnd(114514);
62     shuffle(edges.begin(),edges.end(),rnd);
63     for (auto [u,v]:edges) {
64         e[u].push_back(v),e[v].push_back(u);
65         if (!(lk[u]||lk[v])) lk[u]=v,lk[v]=u;
66     }
67     int res = 0;
68     for (int i=1;i<=n;i++) if (!lk[i]) bfs(i);
69     for (int i=1;i<=n;i++) res += !!lk[i];
70     return res/2;
71 }
72 }
73 using blossom_tree::max_general_match,blossom_tree::lk;
74

```



```

75 int main(){
76     int n,m; std::cin >> n >> m;
77     std::vector<std::pair<int,int>>e;
78     while(m--){
79         int a,b;std::cin >> a >> b;
80         e.push_back({a,b});
81     }
82
83     std::cout << max_general_match(n,e) << '\n';//N:1_idx; edges:0_idx
84
85     std::vector<int>ans(n+1);
86     for(int i = 1;i <= n;i++){//if(lk[i]) i <-> lk[i]
87         ans[i] = lk[i];
88     }
89
90     for(int i = 1;i <= n;i++){
91         std::cout << ans[i] << ' ';//if(ans[i] == 0) not_match
92     }
93 }

```

一般图最大权匹配

时间复杂度 $O(N^3)$

```

1 //https://www.luogu.com.cn/problem/P6699
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 namespace Graph {
6     const int N = 403 * 2; //两倍点数
7     typedef int T; //权值大小
8     const T inf = numeric_limits<int>::max() >> 1;
9     struct Q { int u, v; T w; } e[N][N];
10    T lab[N];
11    int n, m = 0, id, h, t, lk[N], sl[N], st[N], f[N], b[N][N], s[N], ed[N],
    q[N];
12    vector<int> p[N];
13    #define dvd(x) (lab[x.u] + lab[x.v] - e[x.u][x.v].w * 2)
14    #define FOR(i, b) for (int i = 1; i <= (int)(b); i++)
15    #define ALL(x) (x).begin(), (x).end()
16    #define ms(x, i) memset(x + 1, i, m * sizeof x[0])
17    void upd(int u, int v) {
18        if (!sl[v] || dvd(e[u][v]) < dvd(e[sl[v]][v])) {
19            sl[v] = u;
20        }
21    }
22    void ss(int v) {
23        sl[v] = 0;
24        FOR(u, n) {

```

```

25         if (e[u][v].w > 0 && st[u] != v && !s[st[u]]) {
26             upd(u, v);
27         }
28     }
29 }
30 void ins(int u) {
31     if (u <= n) { q[++t] = u; }
32     else {
33         for (int v : p[u]) ins(v);
34     }
35 }
36 void mdf(int u, int w) {
37     st[u] = w;
38     if (u > n) {
39         for (int v : p[u]) mdf(v, w);
40     }
41 }
42 int gr(int u, int v) {
43     v = find(ALL(p[u]), v) - p[u].begin();
44     if (v & 1) {
45         reverse(1 + ALL(p[u]));
46         return (int)p[u].size() - v;
47     }
48     return v;
49 }
50 void stm(int u, int v) {
51     lk[u] = e[u][v].v;
52     if (u <= n) return;
53     Q w = e[u][v];
54     int x = b[u][w.u], y = gr(u, x);
55     for (int i = 0; i < y; i++) {
56         stm(p[u][i], p[u][i ^ 1]);
57     }
58     stm(x, v);
59     rotate(p[u].begin(), y + ALL(p[u]));
60 }
61 void aug(int u, int v) {
62     int w = st[lk[u]];
63     stm(u, v);
64     if (!w) return;
65     stm(w, st[f[w]]), aug(st[f[w]], w);
66 }
67 int lca(int u, int v) {
68     for (++id; u | v; swap(u, v)) {
69         if (!u) continue;
70         if (ed[u] == id) return u;
71         ed[u] = id;
72         if (u == st[lk[u]]) u = st[f[u]];
73     }
74     return 0;
75 }
76 void add(int u, int a, int v) {

```

```

77     int x = n + 1, i, j;
78     while (x <= m && st[x]) ++x;
79     if (x > m) ++m;
80     lab[x] = s[x] = st[x] = 0;
81     lk[x] = lk[a];
82     p[x].clear();
83     p[x].push_back(a);
84     for (i = u; i != a; i = st[f[j]]) {
85         p[x].push_back(i);
86         p[x].push_back(j = st[lk[i]]);
87         ins(j);
88     }
89     reverse(1 + ALL(p[x]));
90     for (i = v; i != a; i = st[f[j]]) {
91         p[x].push_back(i);
92         p[x].push_back(j = st[lk[i]]);
93         ins(j);
94     }
95     mdf(x, x);
96     FOR(i, m) {
97         e[x][i].w = e[i][x].w = 0;
98     }
99     memset(b[x] + 1, 0, n * sizeof b[0][0]);
100    for (int u : p[x]) {
101        FOR(v, m) {
102            if (!e[x][v].w || dvd(e[u][v]) < dvd(e[x][v])) {
103                e[x][v] = e[u][v], e[v][x] = e[v][u];
104            }
105        }
106        FOR(v, n) {
107            if (b[u][v]) { b[x][v] = u; }
108        }
109    }
110    ss(x);
111 }
112 void ex(int u) {
113     for (int x : p[u]) mdf(x, x);
114     int a = b[u][e[u][f[u]].u], r = gr(u, a);
115     for (int i = 0; i < r; i += 2) {
116         int x = p[u][i], y = p[u][i + 1];
117         f[x] = e[y][x].u;
118         s[x] = 1;
119         s[y] = sl[x] = 0;
120         ss(y), ins(y);
121     }
122     s[a] = 1, f[a] = f[u];
123     for (int i = r + 1; i < p[u].size(); i++) {
124         s[p[u][i]] = -1;
125         ss(p[u][i]);
126     }
127     st[u] = 0;
128 }

```

```

129 bool on(const Q &e) {
130     int u = st[e.u], v = st[e.v];
131     if (s[v] == -1) {
132         f[v] = e.u, s[v] = 1;
133         int a = st[lk[v]];
134         sl[v] = sl[a] = s[a] = 0;
135         ins(a);
136     } else if (!s[v]) {
137         int a = lca(u, v);
138         if (!a) {
139             return aug(u, v), aug(v, u), 1;
140         } else {
141             add(u, a, v);
142         }
143     }
144     return 0;
145 }
146 bool bfs() {
147     ms(s, -1), ms(sl, 0);
148     h = 1, t = 0;
149     FOR(i, m) {
150         if (st[i] == i && !lk[i]) {
151             f[i] = s[i] = 0;
152             ins(i);
153         }
154     }
155     if (h > t) return 0;
156     while (1) {
157         while (h <= t) {
158             int u = q[h++];
159             if (s[st[u]] == 1) continue;
160             FOR(v, n) {
161                 if (e[u][v].w > 0 && st[u] != st[v]) {
162                     if (dvd(e[u][v])) upd(u, st[v]);
163                     else if (on(e[u][v])) return 1;
164                 }
165             }
166         }
167         T x = inf;
168         for (int i = n + 1; i <= m; i++) {
169             if (st[i] == i && s[i] == 1) {
170                 x = min(x, lab[i] >> 1);
171             }
172         }
173         FOR(i, m) {
174             if (st[i] == i && sl[i] && s[i] != 1) {
175                 x = min(x, dvd(e[sl[i]][i]) >> s[i] + 1);
176             }
177         }
178         FOR(i, n) {
179             if (~s[st[i]]) {
180                 if ((lab[i] += (s[st[i]] * 2 - 1) * x) <= 0) return 0;

```

```

181     }
182 }
183 for (int i = n + 1; i <= m; i++) {
184     if (st[i] == i && ~s[st[i]]) {
185         lab[i] += (2 - s[st[i]] * 4) * x;
186     }
187 }
188 h = 1, t = 0;
189 FOR(i, m) {
190     if (st[i] == i && sl[i] && st[sl[i]] != i && !dvd(e[sl[i]][i])
&& on(e[sl[i]][i])) {
191         return 1;
192     }
193 }
194 for (int i = n + 1; i <= m; i++) {
195     if (st[i] == i && s[i] == 1 && !lab[i]) ex(i);
196 }
197 }
198 return 0;
199 }
200 template<typename TT> long long work(int N, const vector<tuple<int, int,
TT>> &edges) {
201     ms(ed, 0), ms(lk, 0);
202     n = m = N; id = 0;
203     iota(st + 1, st + n + 1, 1);
204     T wm = 0; long long r = 0;
205     FOR(i, n) FOR(j, n) {
206         e[i][j] = {i, j, 0};
207     }
208     for (auto [u, v, w] : edges) {
209         wm = max(wm, e[v][u].w = e[u][v].w = max(e[u][v].w, (T)w));
210     }
211     FOR(i, n) { p[i].clear(); }
212     FOR(i, n) FOR(j, n) {
213         b[i][j] = i * (i == j);
214     }
215     fill_n(lab + 1, n, wm);
216     while (bfs()) {};
217     FOR(i, n) if (lk[i]) {
218         r += e[i][lk[i]].w;
219     }
220     return r / 2;
221 }
222 auto match() {
223     vector<array<int, 2>> ans;
224     FOR(i, n) if (lk[i]) {
225         ans.push_back({i, lk[i]});
226     }
227     return ans;
228 }
229 } // namespace Graph
230 using Graph::work, Graph::match;

```

```

231
232 void soviet(){
233     int n,m; std::cin >> n >> m;
234     std::vector<std::tuple<int,int,long long>>edges(m);
235     for(auto &[a,b,c]:edges){
236         std::cin >> a >> b >> c;
237     }
238     std::cout << work(n,edges) << '\n';//n:1_idx; edges:0_idx
239
240     std::vector<int>ans(n+1);
241     auto v = match();
242
243     for(auto [x,y]:v){// x <=> y
244         ans[x] = y;
245     }
246     for(int i = 1;i <= n;i++){
247         std::cout << ans[i] << ' ';//if(ans[i] = 0) not_match
248     }
249 }
250
251 int main() {
252     int M_T = 1;std::ios::sync_with_stdio(false),std::cin.tie(0);
253     // std::cin >> M_T;
254     while(M_T--){ soviet(); }
255     return 0;
256 }

```

基环树

n个点n条边组成的无向连通图。若不保证连通，也有可能是内/外向树/森林。

找环

可以以拓扑排序或者dfs实现：[P8655 发现环 - 洛谷](#)

基环树的直径

直径有两种情况，二者取最大值即为答案：

1. 直径处于以环上某一点为根节点，且不经过环边的子树中。对每一颗子树进行树形DP求直径，取最大值。
2. 经过环边，链接两个根节点x,y的子树。 $ans = \max\{d1[x]+d1[y]+dis(x,y)\}$ 。 $d1[i]$ 表式以i为根节点向下能达到的最大距离， $dis(i,j)$ 表示环上两点之间的最远距离，断链成环，复制拼接，再用前缀和+单调队列优化即可O(N)求得答案。

一般处理方法

1. 找到唯一的环；

2. 对环之外的部分按照若干棵树处理;
3. 考虑与环一起计算。

负环

SPFA判负环

如果某点的最短路所包含的边数大于等于n, 则说明存在负环

```
1 //https://www.acwing.com/activity/content/problem/content/921/
2 //判断有向图中是否存在负环
3 #include <iostream>
4 #include <cstring>
5 #include <queue>
6 using namespace std;
7 const int N = 100005;
8 int n, m;
9 queue<int>q;
10 int h[N], e[N], ne[N], idx, w[N];
11 int dist[N], cnt[N]; //dist记录1~k最短距离, cnt记录1~k边的数量
12 bool st[N];
13
14 void add(int a, int b, int c) {
15     w[idx] = c, e[idx] = b, ne[idx] = h[a], h[a] = idx++;
16 }
17
18 int SPFA() {
19     //判负环不需要初始化dist, 如果存在负环, 那么dist不管初始化为多少, 都会被更新
20     //将所有点进入队列。如果只加源点可能到不了有负环的点, 只能说明从源点出发不能抵达负环, 而不能
    说明图上不存在负环。
21     for (int i = 1; i <= n; i++) {
22         st[i] = 1;
23         q.push(i);
24     }
25
26     while (q.size()){
27         int t = q.front();
28         q.pop();
29         st[t] = 0;
30         for (int i = h[t]; i != -1; i = ne[i]) {
31             int k = e[i];
32             if (dist[k] > dist[t] + w[i]) {
33                 dist[k] = dist[t] + w[i];
34                 cnt[k] = cnt[t] + 1; //更新
35                 if (cnt[k] >= n) return 1; //如果到k经过的边的数量>=总点数说明存在负环
36                 if (!st[k]) {
37                     q.push(k);
38                     st[k] = 1;
```

```

39         }
40     }
41 }
42 }
43 return 0;
44 }
45
46 int main() {
47     memset(h, -1, sizeof h);
48     cin >> n >> m;
49     while (m--){
50         int a, b, c; cin >> a >> b >> c;
51         add(a, b, c);
52     }
53     if(SPFA()) puts("Yes");
54     else puts("No");
55 }

```

bellman-ford判负环

bellman-ford判负环，据bellman-ford的性质最后得出的是通过不超过n-1条边的从起点到其他点的最短距离

但是如果在n-1次循环之后仍然存在边可以被松弛，那么就存在负环（因为如果没有负环n-1次就已经确定了最短距离，具体可参考bellman-ford证明，已经是最短距离了还能被松弛，必然是存在负环）

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  const int N = 2003,M = 10004;
5  int n,m;
6  int dist[N];
7
8  struct Edge{
9      int a,b,c;
10 }e[M];
11
12 bool bellman(){
13     memset(dist,0x3f,sizeof dist);
14     dist[1] = 0;
15
16     for(int i = 1;i <= n-1;i++){
17         for(int j = 1;j <= m;j++){
18             auto &[a,b,c] = e[j];
19             dist[b] = min(dist[b],dist[a] + c);
20         }
21     }
22     for(int i = 0;i <= m;i++){//如果在n-1次循环之后仍然存在边可以被松弛，那么就存在负环

```



```

23     auto &[a,b,c] = e[i];
24     if(dist[b] > dist[a] + c) return 1;
25 }
26 return 0;
27 }
28 int main(){
29     cin >> n >> m;
30     for(int i = 1; i <= m; i++){
31         cin >> e[i].a >> e[i].b >> e[i].c;
32     }
33     if(bellman()) cout << "Yes";
34     else cout << "No";
35 }

```

差分约束

求解差分约束系统，有 m 条约束条件，每条都为形如 $x_a - x_b \geq c_k$, $x_a - x_b \leq c_k$ 或 $x_a = x_b$ 的形式，判断该差分约束系统有没有解。

题意	转化	连边
$x_a - x_b \geq c$	$x_b - x_a \leq -c$	<code>add(a, b, -c);</code>
$x_a - x_b \leq c$	$x_a - x_b \leq c$	<code>add(b, a, c);</code>
$x_a = x_b$	$x_a - x_b \leq 0, x_b - x_a \leq 0$	<code>add(b, a, 0), add(a, b, 0);</code>

判断负环，如果存在负环，则无解。

建立一个**虚拟的源点**0或n+1，设 $dist[0] = 0$ 并向每一个点连一条权重为 0 边，跑单源最短路，若图中存在负环，则给定的差分约束系统无解，否则， $x_i = dist[i]$ 为该差分约束系统的一组解。

注意到，如果 $\{x_1, x_2, \dots, x_n\}$ 是该差分约束系统的一组解，那么对于任意的常数 d ， $\{x_1 + d, x_2 + d, \dots, x_n + d\}$ 显然也是该差分约束系统的一组解，因为这样做差后 d 刚好被消掉。

一般使用 SPFA 判断图中是否存在负环，最坏时间复杂度为 $O(nm)$ 。

```

1 //小 K 的农场 https://www.luogu.com.cn/problem/P1993
2 #include <iostream>
3 #include <cstring>
4 #include <queue>
5 using namespace std;
6 const int N = 20004;
7 int n,m;
8 int h[N],e[N],ne[N],w[N],idx;
9 int dist[N],cnt[N];
10 bool st[N];
11
12 void add(int a,int b,int c){

```

```

13     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
14 }
15
16 bool spfa(){
17     std::memset(dist,0x3f,sizeof dist);
18     queue<int>q;
19     dist[0] = 0;
20     q.push(0);
21     while(q.size()){
22         int t = q.front();
23         q.pop();
24         st[t] = 0;
25         for(int i = h[t];~i;i = ne[i]){
26             int k = e[i];
27             if(dist[k] > dist[t] + w[i]){
28                 dist[k] = dist[t] + w[i];
29                 cnt[k] = cnt[t] + 1;
30                 if(cnt[k] >= n+1) return 1;//存在负环,无解(加入源点0后有n+1个点)
31                 if(!st[k]){
32                     q.push(k);
33                     st[k] = 1;
34                 }
35             }
36         }
37     }
38     return 0;
39 }
40
41 int main(){
42     memset(h,-1,sizeof h);
43     cin >> n >> m;
44     for(int i = 1;i <= m;i++){
45         int op,a,b,c;cin >> op;
46         if(op == 1){
47             cin >> a >> b >> c;
48             add(a,b,-c);
49         }
50         else if(op == 2){
51             cin >> a >> b >> c;
52             add(b,a,c);
53         }
54         else{
55             cin >> a >> b;
56             add(a,b,0);add(b,a,0);
57         }
58     }
59     for(int i = 1;i <= n;i++) add(0,i,0);//建立超级源点0,跑单源spfa
60     //也可以不建源点,直接将所有结点入队跑全源spfa
61
62     if(spfa()) cout << "No";//若存在负环,则无解
63     else cout << "Yes";//否则dist[]为一组解
64 }

```

连通性相关

割点和割边

dfn[x]为每个点dfs第一次被访问的时间戳

low[x]为以下节点的时间戳的最小值

1.x的子树中的节点

2.通过一条不在搜索树上的边，能够到达x的子树的节点

从根开始的一条路径上的 dfn 严格递增，low 非严格递增

```
1 //Tarjan
2 void Tarjan(int u) {
3     low[u] = dfn[u] = ++dn;           // low 初始化为当前节点 dfn
4     for (int v : G[u]) {             // 遍历 u 的相邻节点
5         if (!dfn[v]) {                // 如果未访问过
6             Tarjan(v);                // 递归
7             low[u] = std::min(low[u], low[v]); // 未访问的和 low 取 min
8         } else
9             low[u] = std::min(low[u], dfn[v]); // 已访问的和 dfn 取 min
10    }
11 }
```

割点

对于一个无向图，如果把一个点删除后这个图的极大连通分量数增加了，那么这个点就是这个图的割点（又称割顶）。

若x不是搜索树的根节点root(dfs的起点)，则x是割点当且仅当搜索树上存在一个x的子节点满足 $dfn[x] \leq low[y]$

若x是搜索树的根节点，则x是割点当且仅当搜索树上存在至少两个子节点 y_1, y_2 满足上式

```
1 //P3388 【模板】割点（割顶） https://www.luogu.com.cn/problem/P3388
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5 const int N = 20004, M = 200005;
6
7 int h[N], e[M], ne[M], idx;
8 void add(int a, int b){
9     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
10 }
11
```

```

12 int dfn[N], low[N], cut[M], dn, root;
13 void tarjan(int x){
14     dfn[x] = low[x] = ++dn;
15     int son = 0;
16     for (int i = h[x]; ~i; i = ne[i]){
17         int y = e[i];
18         if (!dfn[y]){
19             tarjan(y);
20             son++;
21             low[x] = std::min(low[x], low[y]);
22             if (x != root && dfn[x] <= low[y]) cut[x] = 1;
23         }
24         else low[x] = std::min(low[x], dfn[y]);
25     }
26     if (x == root && son >= 2) cut[x] = 1;
27 }
28
29 int main(){
30     memset(h, -1, sizeof h);
31     int n, m; cin >> n >> m;
32     for (int i = 1; i <= m; i++) {
33         int a, b; cin >> a >> b;
34         add(a,b); add(b,a);
35     }
36     int cnt = 0;
37     for (int i = 1; i <= n; i++) {
38         if (!dfn[i]) root = i, tarjan(i);
39         if (cut[i]) cnt++;
40     }
41     cout << cnt << endl;
42     for (int i = 1; i <= n; i++) {
43         if(cut[i]) cout << i << ' ';
44     }
45 }

```

割边

对于一个无向图，如果删掉一条边后图中的连通分量数增加了，则称这条边为桥或者割边。

无向边(x,y)是割边，当且仅当搜索树上存在x的一个子节点y满足 $dfn[x] < low[y]$

```

1 //https://www.luogu.com.cn/problem/U63110
2 //可处理重边
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6 const int N = 100005;
7 int n,m;
8

```

```

9  int h[N],e[N],ne[N],idx;
10 void add(int a,int b){//(0 1),(2 3),(4 5)...(x y)  x和x^1为两条方向相反的边
11     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
12 }
13
14 int dfn[N],low[N],cut[N],dn;
15 void tarjan(int x,int edge){
16     dfn[x] = low[x] = ++dn;
17     for (int i = h[x] ;~i; i = ne[i]) {
18         int y = e[i];
19         if (!dfn[y]) {
20             tarjan(y,i);
21             low[x] = std::min(low[x],low[y]);
22             if (dfn[x] < low[y]) {
23                 cut[i] = cut[i^1] = 1;
24             }
25         }
26         else if (i != (edge^1)) low[x] = std::min(low[x], dfn[y]);
27     }
28 }
29
30 void sol(){
31     memset(h,-1,sizeof h);
32     for(int i = 1;i <= m;i++){
33         int a,b;cin >> a >> b;
34         add(a,b);add(b,a);//正反边一定要同时添加
35     }
36     for(int i = 1;i <= n;i++){
37         if(!dfn[i]) tarjan(i,0);
38     }
39     int cnt = 0;
40     for(int i = 0;i < idx;i+=2){
41         if(cut[i]) cnt++;//e[i]到e[i^1]为一条割边
42     }
43     cout << cnt << '\n';
44 }
45
46 int main(){
47     while(cin >> n >> m,n && m){
48
49         for(int i = 1;i <= n;i++) dfn[i] = low[i] = 0;
50         for(int i = 0;i < idx;i++) cut[i] = 0;//注意cut清空范围
51         idx = dn = 0;
52         sol();
53     }
54 }

```

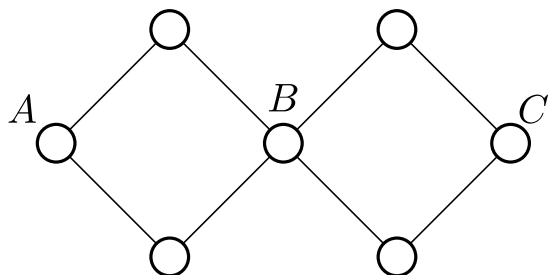
双连通分量

在一张连通的无向图中，对于两个点 u 和 v ，如果无论删去哪条边（只能删去一条）都不能使它们不连通，我们就说 u 和 v **边双连通**。

在一张连通的无向图中，对于两个点 u 和 v ，如果无论删去哪个点（只能删去一个，且不能删 u 或 v 自己）都不能使它们不连通，我们就说 u 和 v **点双连通**。

边双连通具有传递性，即，若 x, y 边双连通， y, z 边双连通，则 x, z 边双连通。

点双连通不具有传递性，反例如下图， A, B 点双连通， B, C 点双连通，而 A, C 不点双连通。



对于一个无向图中的 **极大** 边双连通的子图，我们称这个子图为一个 **边双连通分量**。

对于一个无向图中的 **极大** 点双连通的子图，我们称这个子图为一个 **点双连通分量**。

边双连通分量

不存在割边的连通分量。

e-DCC的求法比较简单，先求出**无向图**中的所有割边，把割边删除后，剩下的图会分成若干个连通块，每一个连通块就是一个“边双连通分量”。

```
1 //https://www.luogu.com.cn/problem/P8436
2 #include <iostream>
3 #include <cstring>
4 #include <vector>
5 using namespace std;
6 const int N = 500005, M = 4000006;
7 vector<int> v[N];
8
9 int h[N], e[M], ne[M], idx;
10 void add(int a, int b){
11     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
12 }
13
14 int dfn[N], low[N], dn;
15 bool cut[M];
16 void tarjan(int x, int edge){
17     dfn[x] = low[x] = ++dn;
18     for(int i = h[x]; ~i; i = ne[i]){
```

```

19     int y = e[i];
20     if(!dfn[y]){
21         tarjan(y,i);
22         low[x] = min(low[x],low[y]);
23         if(dfn[x] < low[y]){
24             cut[i] = cut[i^1] = 1;
25         }
26     }
27     else if(i != (edge^1)) low[x] = min(low[x],dfn[y]);
28 }
29 }
30
31 bool st[N];
32 void dfs(int u){
33     st[u] = 1;
34     for(int i = h[u];~i;i = ne[i]){
35         int k = e[i];
36         if(st[k] || cut[i]) continue;
37         dfs(k);
38         v[cnt].emplace_back(k);
39     }
40 }
41
42 int main(){
43     memset(h,-1,sizeof h);
44     int n,m; cin >> n >> m;
45     for(int i = 1;i <= m;i++){
46         int a,b;scanf("%d %d",&a,&b);
47         add(a,b);add(b,a);
48     }
49
50     for(int i = 1;i <= n;i++){
51         if(!dfn[i]) tarjan(i,0);
52     }
53
54     int cnt = 0;
55     for(int i = 1;i <= n;i++){
56         if(!st[i]) {
57             cnt++;
58             v[cnt] = {i};
59             dfs(i);
60         }
61     }
62     cout << cnt << endl;
63
64     for(int i = 1;i <= cnt;i++){
65         printf("%d ",v[i].size());
66         for(auto x:v[i]){
67             printf("%d ",x);
68         }
69         printf("\n");
70     }

```

点双连通分量

不存在割点的连通分量。

一张**无向图**是点双连通图，当且仅当满足下列**两个条件之一**：

1. 图的顶点数不超过 2。
2. 图中任意两点都同时包含在**至少一个**简单环中。其中“简单环”指的是不自交的环，也就是我们通常画出的环。

```

1  //【模板】点双连通分量 https://www.luogu.com.cn/problem/P8435
2  #include <iostream>
3  #include <cstring>
4  #include <vector>
5  #include <stack>
6  using namespace std;
7  const int N = 500005,M = 4000006;
8  int n,m;
9  int h[N],e[M],ne[M],idx;
10 int dfn[N],low[N],dn,root;
11 bool cut[N];
12 stack<int>sk;
13 vector<int>v[N];
14 int cnt;
15
16 void add(int a,int b){
17     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
18 }
19
20 void tarjan(int x){
21     dfn[x] = low[x] = ++dn;
22     sk.emplace(x);
23     if(x == root && h[x] == -1) { //处理孤立点
24         v[++cnt].emplace_back(x);
25         return;
26     }
27     int son = 0;
28     for(int i = h[x];~i;i = ne[i]){
29         int y = e[i];
30         if(!dfn[y]){
31             tarjan(y);
32             son++;
33             low[x] = min(low[x],low[y]);
34             if(dfn[x] <= low[y]){
35                 if(x != root || son >= 2) cut[x] = 1;
36                 v[++cnt].emplace_back(x);

```



```

37         v[cnt].emplace_back(y);
38         while(sk.top() != y){
39             v[cnt].emplace_back(sk.top());
40             sk.pop();
41         }
42         sk.pop();
43     }
44 }
45 else low[x] = min(low[x],dfn[y]);
46 }
47 }
48
49 int main(){
50     memset(h,-1,sizeof h);
51     cin >> n >> m;
52     for(int i = 1;i <= m;i++){
53         int a,b;scanf("%d %d",&a,&b);
54         if(a == b) continue;
55         add(a,b);add(b,a);
56     }
57
58     for(int i = 1;i <= n;i++){
59         if(!dfn[i]) root = i,tarjan(i);
60     }
61     cout << cnt << endl;
62     for(int i = 1;i <= cnt;i++){
63         printf("%d ",v[i].size());
64         for(auto x:v[i]){
65             printf("%d ",x);
66         }
67         printf("\n");
68     }
69 }

```

强连通分量

有向图强连通分量的Tarjan算法(byvoid.com)

给点一张有向图，若对于图中任意两个节点相互可达，则称该有向图是“强连通图”

有向图的极大强连通子图被称为“强连通分量”，简记为SCC。

一个环一定是强连通图。Tarjan算法基本思路就是对于每个点，尽可能找到与它一起能构成环的所有节点。每个强连通分量为搜索树中的一棵子树。搜索时，把当前搜索树中未处理的节点加入一个栈，回溯时可以判断栈顶到栈中的节点是否为一个强连通分量

```

2  #include <iostream>
3  #include <algorithm>
4  #include <cstring>
5  #include <stack>
6  #include <vector>
7  using namespace std;
8  const int N = 10004,M = 200005;
9  int n,m;
10
11 int h[N],e[M],ne[M],idx;
12 void add(int a,int b){
13     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
14 }
15
16 vector<int>v[N];//第i个强连通分量包含的节点
17 int dfn[N],low[N],dn;
18 int id[N],idn;//节点i所在强连通分量scc
19 stack<int>sk;
20 void tarjan(int x){
21     dfn[x] = low[x] = ++dn;
22     sk.push(x);
23     for(int i = h[x];~i;i = ne[i]){
24         int y = e[i];
25         if(!dfn[y]){//如果y未被搜索过,递归搜索y
26             tarjan(y);
27             low[x] = min(low[x],low[y]);
28         }
29         else if(!id[y]){//如果y被搜索过,且不属于任何一个强连通分量(则y在栈中),用dfn[y]更新low[x]
30             low[x] = min(low[x],dfn[y]);
31         }
32     }
33     if(dfn[x] == low[x]){
34         id[x] = ++idn;
35         v[idn].emplace_back(x);
36         while(sk.top() != x){
37             id[sk.top()] = idn;
38             v[idn].emplace_back(sk.top());
39             sk.pop();
40         }
41         sk.pop();
42     }
43 }
44
45 int main(){
46     memset(h,-1,sizeof h);
47     cin >> n >> m;
48     for(int i = 1;i <= m;i++){
49         int a,b;cin >> a >> b;
50         add(a,b);
51     }
52

```

```

53     for(int i = 1; i <= n; i++){
54         if(!dfn[i]) tarjan(i);
55     }
56
57     for(int i = 1; i <= idn; i++) sort(v[i].begin(), v[i].end());
58     cout << idn << '\n';
59     vector<int> st(n+1);
60     for(int i = 1; i <= n; i++){
61         if(st[i]) continue;
62         for(auto x: v[id[i]]){
63             cout << x << ' ';
64             st[x] = 1;
65         }
66         cout << '\n';
67     }
68 }

```

常应用缩点消除环的影响，转化为DAG图解决

在缩点后的DAG中，强连通分量的标号顺序是其拓扑序的逆序，出度为0的点标号越小，即反图的拓扑序。

2-SAT

有N个变量，每个变量只有两种可能得取值。再给点M个条件，每个条件都是对两个变量的取值限制。求是是否存在对N个变量的合法赋值，使得M个条件均得到满足。

设一个变量 A_i 的两种取值分别是 $A_{i,0}$ 和 $A_{i,1}$ 。每个条件可以转化为统一的形式：

“诺 A_i 的赋值为 $A_{i,p}$ ，则 A_j 的赋值必须为 $A_{j,q}$ ”，其中 $p, q \in \{0, 1\}$ 。

2-SAT问题的判断方法如下（时间复杂度 $O(N+M)$ ）：

1. 建立 $2N$ 个节点的有向图，每个变量 A_i 对应两个节点 i 和 $i+n$ 。
2. 考虑每个条件，形如“诺 A_i 的赋值为 $A_{i,p}$ ，则 A_j 的赋值必须为 $A_{j,q}$ ”， $p, q \in \{0, 1\}$ ，从 $i+p*n$ 到 $j+q*n$ 连一条有向边。
注意上述条件蕴含着它的逆否命题“诺 A_j 的赋值为 $A_{j,1-q}$ ，则 A_i 的赋值必须为 $A_{i,1-p}$ ”。如果在给点的M个限制条件中，原命题和逆否命题不一定成对出现，应该从 $j+(1-q)*n$ 到 $i+(1-p)*n$ 也连一条有向边。
3. 用 **Tarjan** 算法求出有向图中所有的强连通分量。
4. 诺存在 $i \in [1, n]$ 满足 i 和 $i+n$ 属于同一个强连通分量，则出现了矛盾：诺变量 A_i 赋值为 $A_{i,p}$ ，则变量 A_i 必须赋值为 $A_{i,1-p}$ ，说明问题无解。

5. 诺问题还要求具体解，Tarjan后同一元素拆点强连通分量编号小的点即为合法点，如果一个元素拆成的两个点之间没有任何路径相连，即使是有向路径，那么这两点都可以成为合法点，这两点的分量编号不同，选小的即可

P4782 【模板】2-SAT - 洛谷(luogu.com.cn)

N个变量，M个条件：「 x_i 为 a 或 x_j 为 b 」($a, b \in 0, 1$)

条件可转化为诺 x_i 为 $1 - a$ 则 x_j 必为 b ，诺 x_j 为 $1 - b$ 则 x_i 必为 a 。以此建边

```
1  #include <iostream>
2  #include <cstring>
3  #include <stack>
4  using namespace std;
5  const int N = 2000006; // N开两倍
6  int n,m;
7
8  int h[N], e[N], ne[N], idx;
9  void add(int a, int b) {
10     e[idx] = b, ne[idx] = h[a], h[a] = idx++;
11 }
12
13 int dfn[N], low[N], dn;
14 int id[N], idn;
15 stack<int>sk;
16 void tarjan(int x) {
17     dfn[x] = low[x] = ++dn;
18     sk.push(x);
19     for (int i = h[x]; ~i; i = ne[i]) {
20         int y = e[i];
21         if (!dfn[y]) {
22             tarjan(y);
23             low[x] = std::min(low[x], low[y]);
24         }
25         else if (!id[y]) low[x] = std::min(low[x], dfn[y]);
26     }
27     if (low[x] == dfn[x]) {
28         ++idn;
29         id[x] = idn;
30         while (sk.top() != x) {
31             id[sk.top()] = idn;
32             sk.pop();
33         }
34         sk.pop();
35     }
36 }
37
38 int main(){
39     memset(h,-1,sizeof h);
40     scanf("%d %d",&n,&m);
41     while(m--){
```

```

42     int a,x,b,y;scanf("%d %d %d %d",&a,&x,&b,&y);
43     if(x == 0 && y == 0){
44         add(a+n,b);
45         add(b+n,a);
46     }
47     if(x == 0 && y == 1){
48         add(a+n,b+n);
49         add(b,a);
50     }
51     if(x == 1 && y == 0){
52         add(a,b);
53         add(b+n,a+n);
54     }
55     if(x == 1 && y == 1){
56         add(a,b+n);
57         add(b,a+n);
58     }
59 }
60 for(int i = 1;i <= n << 1;i++){//2n层循环
61     if(!dfn[i]) tarjan(i);
62 }
63
64 for(int i = 1;i <= n;i++){
65     if(id[i] == id[i+n]){//属于同一个强连通分量,出现矛盾,无解
66         cout << "IMPOSSIBLE";
67         return 0;
68     }
69 }
70 cout << "POSSIBLE\n";
71 for(int i = 1;i <= n;i++){//这里a[i]为0,a[i+n]为1,选所在强连通分量小的为合法解
72     cout << (id[i] > id[i+n]) << ' ';
73 }
74 }

```

[P10969 Katu Puzzle - 洛谷\(luogu.com.cn\)](https://www.luogu.com.cn/problem/P10969)

网络流

[网络流与线性规划24题\(luogu.com.cn\)](https://www.luogu.com.cn/problem/P3386)

一个网络 $G=(V,E)$ ，是一张有向图，图中每条边 $(x,y) \in E$ 都有一个给定的权值 $c(x,y)$ ，称为边的容量，特别的若 $(x,y) \notin E$ ，则 $c(x,y) = 0$ 。图中还有两个指定的特殊节点 **源点S** 和 **汇点T** ($S \neq T$)。

设 $f(x,y)$ 是定义在节点二元组 $(x \in V, y \in V)$ 三的实数函数，且满足：

1. 容量限制： $f(x,y) \leq c(x,y)$
2. 斜对称： $f(x,y) = -f(y,x)$
3. 流量守恒： c

$f()$ 称为网络的流量函数, 对于 $(x, y) \in E$, $f(x, y)$ 称为边的**流量**, $c(x, y) - f(x, y)$ 称为边的**剩余容量**

$\sum_{(S,v) \in E} f(S, v)$, 为整个网络的流量(其中S表示源点)

最大流

对于一个给定的网络, 合法流函数 $f()$ 有很多。其中使得整个网络的流量最大的流函数被称为网络的**最大流**。

建边时反边容量全为0

Edmonds-Karp

时间复杂度 $O(NM^2)$, 实际效率较高, 一般能处理 $10^3 \sim 10^4$ 规模的网络

每次bfs遍历整个**残量网络**, 找出任意一条增广路, 同时计算路径上各边剩余流量的最小值 $\min f$, 显然可以让一股流沿着增广路从S流向T, 网络的流量就可以增加 $\min f$ 。直到网络上不存在增广路为止。

```
1 //模版 https://www.luogu.com.cn/problem/P3376
2 #include <iostream>
3 #include <queue>
4 #include <cstring>
5 using namespace std;
6 const int N = 205, M = 10004;
7 const int INF = 0x3f3f3f3f;
8 int n, m, s, t;
9 long long maxflow;
10 int h[N], e[M], ne[M], idx;
11 long long w[M]; //容量
12 long long rest[N]; //记录各边剩余容量的最小值
13 int pre[M]; //记录点x是从哪条边过来的
14 bool vis[N];
15
16 void add(int x, int y, int z){
17     w[idx] = z, e[idx] = y, ne[idx] = h[x], h[x] = idx++;
18 }
19
20 bool bfs(){
21     memset(vis, 0, sizeof vis);
22     queue<int> q;
23     q.push(s);
24     vis[s] = 1;
25     rest[s] = INF;
26     while(q.size()){
27         int x = q.front();
28         q.pop();
```

```

29         for(int i = h[x]; ~i; i = ne[i]){
30             int y = e[i];
31             if(vis[y] || w[i] == 0) continue;
32             rest[y] = min(rest[x], w[i]);
33             pre[y] = i;
34             q.push(y);
35             vis[y] = 1;
36             if(y == t) return 1;
37         }
38     }
39     return 0;
40 }
41
42 int update(){//将流过的路径容量减少minf,反边增加minf
43     int x = t;
44     while(x != s){
45         int i = pre[x];
46         w[i] -= rest[t];
47         w[i^1] += rest[t];
48         x = e[i^1];
49     }
50     return rest[t]; //网络的流量可以增加最小值minf
51 }
52
53 int main(){
54     memset(h, -1, sizeof h);
55     cin >> n >> m >> s >> t;
56     for(int i = 1; i <= m; i++){
57         int x, y, z; cin >> x >> y >> z;
58         add(x, y, z); add(y, x, 0);
59     }
60     while(bfs()) maxflow += update();
61     cout << maxflow;
62 }

```

Dinic

时间复杂度 $O(N^2M)$ ，实际效率较高，能处理 $10^4 \sim 10^5$ 规模的网络。特别地，在求解[二分图最大匹配](#)时间复杂度为 $O(M\sqrt{N})$ ，边权设为1，源点向A集合连边，B集合向汇点连边。经典且比较常用的网络流算法。

EK算法每次只找出一条增广路，而Dinic可以一次找出多条。

1. 在[残量网络]上bfs求出节点的层次，构造[分层图]。
2. 在分层图上dfs寻找增广路，回溯时更新边权。

$w[i]$ 表示残量网络中边*i*的剩余容量：

正向边(*idx* 为偶)： $w[i] = \text{原边容量} - \text{已流过的流量}$

方向边(*idx* 为奇)： $w[i] = \text{已经流过的流量}$ (即可以回退的流量)

求完最大流后，对于从源点*s*可达的点，到汇点*t*的所有路径上至少有一条边的 $w[i] = 0$ ，这些边构成了最

小割

$d[i]$ 表示从源点 s 到节点 i 在残量网络中的最短距离 (层次)

如果 $d[i] \neq 0$: 表示节点 i 在残量网络中从源点 s 可达 (属于源点 s 所在集合)

如果 $d[i] == 0$: 表示节点 i 在残量网络中从源点 s 不可达 (属于汇点 t 所在集合)

```
1  #include <iostream>
2  #include <queue>
3  #include <cstring>
4  using namespace std;
5  const int INF = 0x3f3f3f3f;
6  const int N = 205,M = 10004;
7  int n,m,s,t;
8  long long maxflow;
9  int h[N],e[M],ne[M],w[M],idx;
10 int d[N];
11 int now[N]; //当前弧优化
12
13 void add(int a,int b,int c){
14     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
15 }
16
17 bool bfs(){ //在残量网络上构造分层图
18     memset(d,0,sizeof d);
19     queue<int>q;
20     q.push(s);
21     d[s] = 1;
22     now[s] = h[s];
23     while(q.size()){
24         int x = q.front();
25         q.pop();
26         for(int i = h[x];~i;i = ne[i]){
27             int y = e[i];
28             if(!w[i] || d[y]) continue;
29             d[y] = d[x] + 1;
30             now[y] = h[y];
31             q.push(y);
32             if(y == t) return 1;
33         }
34     }
35     return 0;
36 }
37
38 int dfs(int x,int flow){ //flow表示经过该点的剩余流量
39     if(x == t) return flow;
40     int res = 0; //res表示经过该点的所有流量和
41     for(int i = now[x];~i && flow;i = ne[i]){ //只有当前还有流时才继续搜索
42         now[x] = i; //当前弧优化
43         int y = e[i];
44         if(!w[i] || d[y] != d[x] + 1) continue;
45         int k = dfs(y,min(flow,w[i]));
46         if(k == 0) d[y] = 0; //剪枝,去掉增广完毕的点
```



```

47     w[i] -= k;
48     w[i^1] += k;
49     res += k;
50     flow -= k;
51 }
52 return res;
53 }
54
55 int main(){
56     memset(h,-1,sizeof h);
57     cin >> n >> m >> s >> t;
58     for(int i = 1;i <= m;i++){
59         int a,b,c;cin >> a >> b >> c;
60         add(a,b,c);add(b,a,0);
61     }
62     while(bfs()) maxflow += dfs(s,INF);
63     cout << maxflow;
64 }

```

最小割

若删除一个边集 $E' \in E$ 后, S 与 T 将不再连通, 则 E' 为该网络的一个割。容量之和最小的割称为网络的**最小割**。

最大流最小割定理: 任何一个网络的最大流量等于最小割中边的容量之和, 即“最大流=最小割”

求具体方案

在求完最大流后, 从源点 S 出发, BFS 遍历残量网络中还能到达的点 ($w[i] > 0$), 这些点属于 S 集合, 剩下无法到达的点属于 T 集合。所有连接 S 和 T 的边即为属于最小割的边。

```

1  bool st[N] = {}; // st[x]=1 则属于 S, 否则属于 T
2  void uuz(){
3      queue<int> q;
4      q.push(s);
5      st[s] = 1;
6      while(q.size()){
7          int x = q.front();
8          q.pop();
9          for(int i = h[x]; ~i; i = ne[i]){
10             int y = e[i];
11             if(!st[y] && w[i]){
12                 q.push(y);
13                 st[y] = 1;
14             }

```

```

15     }
16 }
17 for(int i = 0; i < idx; i += 2){
18     int x = e[i^1], y = e[i];
19     if(st[x] && !st[y]){ //此时w[i] == 0
20         cout << x << ' ' << y << endl; //边(x,y)属于最小割
21     }
22 }
23 }

```

求割边数量

如果要在最小割的前提下最小化个边数量，那么先求出最小割，然后把没有满流的边容量改为INF，满流的边容量改为1，再跑一边最小割就可以求出最小割边数量；

如果没有最小割的前提，直接把所有边的容量设为1，求一遍最小割即可。

常见问题模型

有 n 个物品和两个集合A,B，每个物品必须且只能放入一个集合里，如果一个物品没有放入A集合会花费 a_i ，如果没有放入B集合会花费 b_i ，还有若干个条件如 (u_i, v_i, w_i) 表示如果 u_i 和 v_i 不在同一个集合里会花费 w_i ，求最小代价。

这是一个经典的二者选一的最小割题目。我们设立一个超级源点 s 和超级汇点 t ，对每个点 i 连边

`add(s, i, a[i])` 和 `add(i, t, b[i])`，反边容量均为0。对每个限制条件连双向边

`add(u, v, w), add(v, u, w)`。

注意到，但源点和汇点不相连时，代表这些点都选择了一个其中集合，最小割就是最小花费。如果要求最大花费，将所有花费减去最小割即可。

例题：[P1361 小M的作物 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P1361)

有两块耕地A和B， n 种作物，每种一个。第 i 种作物种在A中可以获得 a_i 的收益，种在B中可以获得 b_i 的收益。另外还有 m 种组合，第 i 个组合中的作物共同种在A中可以获得 $C_{1,i}$ 的额外收益，共同种在B中可以获得 $C_{2,i}$ 的额外收益。求种植的最大收益。 $1 \leq k < n \leq 10^3, 1 \leq m \leq 10^3$

二选其一很容易联想到最小割模型，若不考虑额外收益，容易建图：

- 源点 S 向 i 连一条边权为 a_i 的边
- i 向汇点 T 连一条边权为 b_i 的边

跑一遍最小割（即删去最小），剩下的就是最大的。

考虑额外收益，我们可以对每组作物新建一个节点 na 和 nb ，添加以下边

- 源点 S 向 na 连一条边权为 $C_{1,i}$ 的边， na 向该组作物连一条边权为 INF 的边。
- nb 向汇点 T 连一条边权为 $C_{2,i}$ 的边，该组作物向 nb 连一条边权为 INF 的边。

跑最小割，再用所有收益减去最小割。

```

1  #include <iostream>
2  #include <cstring>
3  #include <queue>
4  using namespace std;
5  const int N = 40004,M = 2000006;
6  int n,m,tot;
7
8  const int INF = 0x3f3f3f3f;
9  template<typename T>
10 struct Dinic{
11     int n,s1,s2,idx;
12     T maxflow;
13
14     std::vector<int>h,e,ne,d,now;
15     std::vector<T>w;
16
17     Dinic(int _n,int _s1,int _s2){
18         n = _n+2; s1 = _s1; s2 = _s2; idx = 0;
19         h = now = std::vector<int>(n,-1);
20         e.clear(); ne.clear(); w.clear();
21     }
22
23     void add(int a,int b,T c){
24         w.emplace_back(c),e.emplace_back(b),ne.emplace_back(h[a]),h[a] =
idx++;
25         w.emplace_back(0),e.emplace_back(a),ne.emplace_back(h[b]),h[b] =
idx++;
26     }
27
28     bool bfs(){
29         d = std::vector<int>(n);
30         std::queue<int>q;
31         q.push(s1);
32         d[s1] = 1;
33         now[s1] = h[s1];
34         while(q.size()){
35             int x = q.front();
36             q.pop();
37             for(int i = h[x];~i;i = ne[i]){
38                 int y = e[i];
39                 if(w[i] && !d[y]){
40                     d[y] = d[x] + 1;
41                     now[y] = h[y];
42                     q.push(y);
43                     if(y == s2) return 1;
44                 }
45             }
46         }
47         return 0;
48     }
49
50     T dfs(int x,T flow){

```

```

51     if(x == s2) return flow;
52     T res = 0;
53     for(int i = now[x]; ~i && flow; i = ne[i]){
54         now[x] = i;
55         int y = e[i];
56         if(w[i] && d[y] == d[x] + 1){
57             T k = dfs(y, std::min(flow, w[i]));
58             if(k == 0) d[y] = 0;
59             w[i] -= k;
60             w[i^1] += k;
61             res += k;
62             flow -= k;
63         }
64     }
65     return res;
66 }
67
68 T sol(){
69     maxflow = 0;
70     while(bfs()) maxflow += dfs(s1, INF);
71     return maxflow;
72 }
73 };
74
75 int main(){
76     cin >> n;
77     long long ans = 0;
78     Dinic<int> t(n+1 + 1003*2, 0, n+1); //(最大节点数, 源点s1, 汇点s2)
79     tot = n+1;
80     for(int i = 1; i <= n; i++){
81         int x; cin >> x;
82         ans += x;
83         t.add(t.s1, i, x);
84     }
85     for(int i = 1; i <= n; i++){
86         int x; cin >> x;
87         ans += x;
88         t.add(i, t.s2, x);
89     }
90     cin >> m;
91     for(int i = 1; i <= m; i++){
92         int k, c1, c2; cin >> k >> c1 >> c2;
93         ans += c1+c2;
94         t.add(t.s1, tot+1, c1);
95         t.add(tot+2, t.s2, c2);
96
97         for(int j = 1; j <= k; j++){
98             int x; cin >> x;
99             t.add(tot+1, x, INF);
100             t.add(x, tot+2, INF);
101         }
102         tot += 2;

```

```

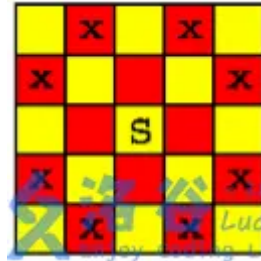
103     }
104     cout << ans - t.sol();
105 }

```

P3355 骑士共存问题 - 洛谷

$N \times N$ 的棋盘上，马（骑士）可以攻击如图所示的八个点。棋盘上有 M 个点设置了障碍，无法放置

棋子。求问棋盘上最多可以放多少个马？ $N \leq 200$ 。



对原图所有点进行奇偶分类，建立源点 s_1 、汇点 s_2 。源点向所有奇数点连一条容量为1的边，所有偶数点向汇点连一条边，奇数点向不能共存的八个偶数点连容量为INF的边。求扔掉最少得骑士使得奇偶两部分无冲突（网络流为0），即跑最小割，答案即为 $n^2 - m - maxflow$ 。即二分图最大独立集模型。

最大权闭合图

给定一张有向图，每个点都有一个权值 $a[i]$ （可能 ≤ 0 ），你需要选择权值和最大的一组点集，且满足：每个点的后继节点都必须包含在点集中。

结论：建立超级源点 s 和超级汇点 t ，若节点 u 权值为正，则连边 $(s, u, a[i])$ ；若节点 u 权值为负，则连边 $(u, t, -a[i])$ ；权值为0不用连，原图上所有边权改为 ∞ 。跑最大流，答案即为**所有正权点之和减去最小割**。

例题：[Get More Money \(★7\) - AtCoder typical90 an - Virtual Judge \(vjjudge.net\)](#)

也可处理不连通的有向森林

```

1  #include <iostream>
2  #include <cstring>
3  #include <queue>
4  using namespace std;
5  const int INF = 0x3f3f3f3f;
6  const int N = 105, M = N*N;
7  int n, m;
8  int h[N], e[M], ne[M], idx, w[M];
9  int a[N];
10
11 int main(){
12     memset(h, -1, sizeof h);

```

```

13     cin >> n >> m;
14     s = 0, t = n+1;
15     for(int i = 1; i <= n; i++){
16         cin >> a[i];
17         a[i] -= m;
18         if(a[i] > 0){
19             ans += a[i];
20             add(s, i, a[i]);
21             add(i, s, 0);
22         }
23         if(a[i] < 0){
24             add(i, t, -a[i]);
25             add(t, i, 0);
26         }
27     }
28
29     for(int i = 1; i <= n; i++){
30         int k; cin >> k;
31         while(k--){
32             int j; cin >> j;
33             add(j, i, INF);
34             add(i, j, 0);
35         }
36     }
37     while(bfs()) maxflow += dfs(s, INF); //Dinic最大流板子省略
38     cout << ans - maxflow;
39 }

```

费用流

给定一个网络 $G = (V, E)$ ，每条边除了有容量限制 $c(x, y)$ 外，还有一个单位流量的费用 $w(x, y)$ ，当 (x, y) 的流量为 $f(x, y)$ 时，需要花费 $f(x, y) * w(x, y)$ 的费用。 w 也满足斜对称性质，即 $w(x, y) = -w(y, x)$ 。

使该网络中花费最小的最大流称为**最小费用最大流**。注意：费用流的前提是最大流。

SSP算法 (Successive Shortest Path)：每次寻找单位费用最小的增广路进行增广，直到图上不存在增广路为止。时间复杂度最坏为 $O(nmf)$ ，其中 f 表示网络的最大流。实现时，只需将 EK 算法或 Dinic 算法中找增广路的过程，替换为用最短路算法寻找单位费用最小的增广路即可。需要保证网络没有负环。

[P3381 【模板】最小费用最大流 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P3381)

基于EK算法实现

```

1  #include <iostream>
2  #include <cstring>
3  #include <queue>

```

```

4  using namespace std;
5  const int INF = 0x3f3f3f3f;
6  const int N = 5003,M = 100005;
7  int n,m,s,t;
8  int maxflow,mincost;
9  int h[N],e[M],ne[M],w[M],c[M],idx;//w为容量,c为费用
10 bool vis[N];
11 int pre[M];
12 int rest[N];
13 int dist[N];
14
15 void add(int u,int v,int x,int y){
16     c[idx] = y,w[idx] = x,e[idx] = v,ne[idx] = h[u],h[u] = idx++;
17 }
18
19 bool spfa(){
20     memset(vis,0,sizeof vis);
21     memset(dist,0x3f,sizeof dist);
22     queue<int>q;
23     q.push(s);
24     vis[s] = 1;
25     rest[s] = INF;
26     dist[s] = 0;
27     while(q.size()){
28         int x = q.front();
29         q.pop();
30         vis[x] = 0;
31         for(int i = h[x];~i;i = ne[i]){
32             int y = e[i];
33             if(w[i] && dist[y] > dist[x] + c[i]){
34                 dist[y] = dist[x] + c[i];
35                 rest[y] = min(rest[x],w[i]);
36                 pre[y] = i;
37                 if(!vis[y]){
38                     q.push(y);
39                     vis[y] = 1;
40                 }
41             }
42         }
43     }
44     return dist[t] < INF;
45 }
46
47 void update(){
48     int x = t;
49     while(x != s){
50         int i = pre[x];
51         w[i] -= rest[t];
52         w[i^1] += rest[t];
53         x = e[i^1];
54     }
55     mincost += rest[t] * dist[t];

```

```

56     maxflow += rest[t];
57 }
58
59 int main(){
60     memset(h,-1,sizeof h);
61     cin >> n >> m >> s >> t;
62     for(int i = 1;i <= m;i++){
63         int u,v,x,y;cin >> u >> v >> x >> y;
64         add(u,v,x,y);add(v,u,0,-y); //反边 流量取0,费用取反
65     }
66     while(spfa()) update();
67     cout << maxflow << ' ' << mincost;
68 }

```

基于Dinic算法实现

```

1  #include <iostream>
2  #include <cstring>
3  #include <queue>
4  using namespace std;
5  const int INF = 0x3f3f3f3f;
6  const int N = 5003,M = 100005;
7  int n,m,s,t;
8  int maxflow,mincost;
9  int h[N],e[M],ne[M],w[M],c[M],idx;
10 int now[N],dist[N];
11 bool vis[N];
12
13 void add(int a,int b,int x,int y){w[] = x为容量,c[] = y为费用
14     c[idx] = y,w[idx] = x,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
15 }
16
17 bool spfa(){
18     memset(vis,0,sizeof vis);
19     memset(dist,0x3f,sizeof dist);//如果要求大费用,则dist应初始化为-INF而非0
20     queue<int>q;
21     q.push(s);
22     now[s] = h[s];
23     dist[s] = 0;
24     while(q.size()){
25         int x = q.front();
26         q.pop();
27         vis[x] = 0;
28         for(int i = h[x];~i;i = ne[i]){
29             int y = e[i];
30             if(w[i] && dist[y] > dist[x] + c[i]){
31                 dist[y] = dist[x] + c[i];
32                 now[y] = h[y];
33                 if(!vis[y]){
34                     q.push(y);

```



```

35         vis[y] = 1;
36     }
37 }
38 }
39 }
40 return dist[t] < INF;
41 }
42
43 int dfs(int x,int flow){
44     if(x == t) return flow;
45     vis[x] = 1;//剪枝优化
46     int res = 0;
47     for(int i = now[x];~i && flow;i = ne[i]){
48         now[x] = i;
49         int y = e[i];
50         if(w[i] && dist[y] == dist[x] + c[i] && !vis[y]){
51             int k = dfs(y,min(w[i],flow));
52             if(!k) dist[y] = 0;
53             w[i] -= k;
54             w[i^1] += k;
55             res += k;
56             flow -= k;
57             mincost += k*c[i];//最小费用 += 当前流量*当前费用
58         }
59     }
60     vis[x] = 0;
61     return res;
62 }
63
64 int main(){
65     memset(h,-1,sizeof h);
66     cin >> n >> m >> s >> t;
67     for(int i = 1;i <= m;i++){
68         int a,b,x,y;cin >> a >> b >> x >> y;
69         add(a,b,x,y);add(b,a,0,-y);//流量取0,费用取反
70     }
71     while(spfa()) maxflow += dfs(s,INF);
72     cout << maxflow << ' ' << mincost;
73 }

```

```

1 //Dinic封装实现,最小费用最大流
2 const int INF = 0x3f3f3f3f;
3 template<typename T>
4 struct Dinic{
5     int n,s1,s2,idx;
6     T maxflow,mincost;
7
8     std::vector<int>h,e,ne,now,vis;
9     std::vector<T>w,c,d;

```

```

10
11     Dinic(int _n,int _s1,int _s2){ //Dinic<int> t(节点总数,源点s1,汇点s2);
12         n = _n+2; s1 = _s1; s2 = _s2; idx = 0;
13         h = now = std::vector<int>(n,-1);
14         e.clear(); ne.clear(); w.clear(); c.clear();
15     }
16
17     void add(int a,int b,T x,T y){
18
19         c.emplace_back(y),w.emplace_back(x),e.emplace_back(b),ne.emplace_back(h[a]),h[a]
20         = idx++;
21         c.emplace_back(-
22         y),w.emplace_back(0),e.emplace_back(a),ne.emplace_back(h[b]),h[b] = idx++;
23     }
24
25     bool spfa(){
26         d = std::vector<T>(n,INF);
27         vis = std::vector<int>(n);
28         std::queue<int>q;
29         q.push(s1);
30         d[s1] = 0;
31         now[s1] = h[s1];
32         while(q.size()){
33             int x = q.front();
34             q.pop();
35             vis[x] = 0;
36             for(int i = h[x];~i;i = ne[i]){
37                 int y = e[i];
38                 if(w[i] && d[y] > d[x] + c[i]){
39                     d[y] = d[x] + c[i];
40                     now[y] = h[y];
41                     if(!vis[y]){
42                         vis[y] = 1;
43                         q.push(y);
44                     }
45                 }
46             }
47         }
48         return d[s2] < INF;
49     }
50
51     T dinic(int x,T flow){
52         if(x == s2) return flow;
53         vis[x] = 1;
54         T res = 0;
55         for(int i = now[x];~i && flow;i = ne[i]){
56             now[x] = i;
57             int y = e[i];
58             if(w[i] && d[y] == d[x] + c[i] && !vis[y]){
59                 T k = dinic(y,std::min(flow,w[i]));
60                 if(k == 0) d[y] = 0;
61                 w[i] -= k;

```

```

59         w[i^1] += k;
60         res += k;
61         flow -= k;
62         mincost += k * c[i];
63     }
64 }
65 vis[x] = 0;
66 return res;
67 }
68
69 std::pair<T,T> sol(){
70     maxflow = mincost = 0;
71     while(spfa()) maxflow += dinic(s1,INF);
72     return {maxflow,mincost};
73 }
74 };

```

欧拉图

欧拉路：给定一张无向图，若存在一条从节点S到节点T的路径，恰好不重不漏地经过每条边一次，则称该路径为S到T的欧拉路

欧拉回路：特别地，若存在一条从节点S出发，恰好不重不漏地经过每一条边一次(可以重复经过图中的节点)，最终回到起点S，则称该路径为欧拉回路。存在欧拉回路的图被称为欧拉图。

欧拉路的存在性判断：一张无向图中存在欧拉路，当且仅当无向图连通，且起点和终点的度数为奇数，其它节点的度数为偶数。

使用DFS寻找欧拉路的基本思想如下：

DFS寻找到第一个无边可走的节点，则这个节点必定为终点。

接下来由于DFS的递归回溯，会退回终点的上一个节点，继续往下搜索，直到寻找到第二个无边可走的节点，则这个节点必定为欧拉路中终点前最后访问的节点。

于是当通过DFS遍历完整张图后，就可以倒序储存下整个欧拉路。该算法时间复杂度为 $O(N + M)$ ，因为一个节点会被遍历多次，且递归层数为 $O(M)$ 级别，容易栈溢出。可以使用栈模拟递归，且每次访问一条边后删除该边(修改表头，令其指向下一条边)

[P6066 [USACO05JAN](#)] [Watchcow S](#) - 洛谷

给定 n 个点 m 条边的无向图(存在重边)，求从1号点出发，每条路必须从两个方向各走恰好一边，最后回到一号农场的任意一种路径。 $2 \leq n \leq 10^4, 1 \leq m \leq 5 \times 10^4$ 。

```

1 // 若欧拉回路存在，则可用dfs和栈求出欧拉回路的一种具体方案
2 #include <iostream>
3 #include <cstring>
4 #include <stack>
5 #include <vector>
6 using namespace std;

```

```

7  const int N = 10004,M = 100005;
8  int n,m;
9  int h[N],e[M],ne[M],idx;
10 bool vis[M];
11 stack<int>sk;
12 vector<int>ans;
13
14 void add(int a,int b){
15     e[idx] = b,ne[idx] = h[a],h[a] = idx++;
16 }
17
18 void euler(){//栈模拟递归
19     sk.emplace(1);
20     while(sk.size()){
21         int u = sk.top();
22         int i = h[u];
23         while(~i && vis[i]) i = ne[i];//找到一条尚未访问的边
24         if(~i){//沿着这条边模拟递归过程，标记该边，并更新表头，避免重复检查已处理边
25             sk.emplace(e[i]);
26             vis[i] = vis[i^1] = 1;//诺将该行注释掉，则为每条边正反恰好各走一次的答案
27             h[u] = ne[i];
28         }
29         else{//u相连的所有边均已访问，模拟回溯过程，记录答案
30             sk.pop();
31             ans.emplace_back(u);
32         }
33     }
34 }
35
36 int main(){
37     memset(h,-1,sizeof h);
38     cin >> n >> m;
39     for(int i = 1;i <= m;i++){
40         int a,b;cin >> a >> b;
41         add(a,b); add(b,a);
42     }
43
44     euler();//这里假设1为起点
45
46     for(int i = ans.size()-1;i >= 0;i--){
47         cout << ans[i] << '\n';
48     }
49 }

```

如果需要输出字典序最小的欧拉路或欧拉回路，则需要将边排序。

